Informe N.º 2 — Diseño de la Solución de Software

Proyecto Integrado — Unidad 2

SGP-Kotoduo: Sistema de Gestión y Seguimiento Productivo

Asignatura:	Proyecto Integrado
Sección:	2026/P TIHI43/IEC-N4-C2/D Temuco IEC
Nombre del docente:	Manuel Alfredo Sanchez Carcamo
Empresa / contraparte:	Kotoduo — agencia publicitaria y taller de confección, Temuco, La Araucanía (kotoduo.cl)
Metodología:	Scrum (Sprint 0 de diseño + 4 Sprints de construcción)
Nombre del estudiante:	Carlos David Mac-Iver Reydet

23 de octubre de 2026

Índice

Guía de lectura **4**

1. Introducción **5**

2. Síntesis de la solución propuesta **5**

2.1. Línea base de requerimientos (Informe N.º 1) 6

2.2. Trazabilidad necesidad → componente → evidencia 8

3. Planificación del proyecto **10**

3.1. Actividades, fases, plazos y responsables 10

3.2. Dependencias y prioridades entre tareas 13

4. Arquitectura de la solución **14**

4.1. Descripción general de la arquitectura 4+1 14

4.2. Vista lógica 15

4.3. Vista de desarrollo o implementación 21

4.4. Vista de procesos 25

4.5. Vista física o de despliegue 30

4.6. Vista de escenarios o casos de uso 32

4.7. Tecnologías y herramientas de infraestructura 36

5. Diseño de interfaces de usuario **40**

5.1. Perfiles y flujos principales de usuario 40

5.2. Navegación y organización de contenidos 40

5.3. Prototipo interactivo funcional 42

5.4. Consistencia visual y adaptabilidad 44

5.5. Retroalimentación de usuarios y mejoras aplicadas 45

6. Diseño e implementación del modelo de datos **47**

6.1. Modelo de datos: entidades y relaciones 47

6.2. Diccionario y reglas de integridad 53

6.3. Bases de implementación de la base de datos estructurada 55

6.4. Bases de implementación de la base de datos no estructurada 58

6.5. Integración entre datos estructurados y no estructurados 60

7. Seguridad y estándares de programación segura **62**

7.1. Roles y perfiles de acceso 62

7.2. Política de autenticación 63

7.3. Política de autorización 64
7.4. Amenazas y medidas de mitigación 65
7.5. Estándares de codificación segura 67
7.6. Evidencia de aplicación de estándares seguros 68
7.7. Marco normativo chileno y estándares de referencia 68

8. Plan de pruebas 74
8.1. Objetivo y alcance del plan de pruebas 74
8.2. Trazabilidad entre requerimientos, casos de uso y casos de prueba 74
8.3. Pruebas unitarias 75
8.4. Pruebas de integración 75
8.5. Pruebas de rendimiento 76
8.6. Pruebas de seguridad 76
8.7. Pruebas de usabilidad 76
8.8. Diseño, ejecución y resultados de casos de prueba 76
8.9. Herramientas manuales y automatizadas 77

9. Conclusiones 80

10. Referencias 80

11. Preparación para la defensa oral 83

12. Manifiesto de uso de inteligencia artificial 86

13. Anexos 88
Anexo E — Lista de cotejo de seguridad (extracto ASVS 4.0.3 L2) 89
Anexo H — Plan detallado de actividades y matriz de dependencias 90
Anexo I — Matriz de tecnologías 96
Anexo J — Diccionario de datos 99
Anexo K — Modelo de amenazas y patrones de codificación segura 106
Anexo L — Especificación de los casos de prueba 114

Guía de lectura

El informe se organiza en dos niveles, para que pueda leerse de corrido sin perder el detalle:

- **Cuerpo (capítulos 1 a 9):** presenta las decisiones, su justificación, los diagramas y tablas de resumen.
- **Anexos (capítulo 13):** reúnen las tablas extensas de consulta (plan de actividades, dependencias, tecnologías, diccionario de datos, amenazas, patrones de codificación segura y casos de prueba) y los artefactos externos (diagramas, scripts de base de datos y prototipo). Cada resumen del cuerpo indica qué tabla del anexo lo detalla.

Los capítulos 10 a 12 contienen las referencias, la preparación para la defensa oral y el manifiesto de uso de inteligencia artificial.

Códigos usados en el informe

Código	Significado	Dónde se define
RF-nn / RNF-nn	Requerimiento funcional / no funcional	Sección 2.1
HU-nn	Historia de usuario	Sección 2.1
M1–M8	Módulo de la solución	Capítulo 2
OT	Orden de trabajo	Capítulo 2
TE-nn	Habilitador técnico del backlog	Sección 3.1 (Tabla 3.1)
A00–A31	Actividad del plan	Sección 3.1 y Anexo H
S0–S4 / Dnn	Sprint / día hábil relativo del proyecto	Sección 3.1
CU-nn	Caso de uso	Sección 4.6
P-nn / IK-n	Pantalla / interacción clave del prototipo	Sección 5.3
FB-nn	Retroalimentación de usuarios	Sección 5.5
R1–R8	Regla de negocio implementada en la base de datos	Sección 6.2
AM-nn	Amenaza del modelo de amenazas	Sección 7.4 y Anexo K
CP-nn	Caso de prueba	Capítulo 8 y Anexo L

1 Introducción

Kotoduo es una agencia publicitaria de Temuco que fabrica ropa deportiva, corporativa y escolar, letreros, souvenirs y papelería. Sus procesos incluyen bordado industrial, sublimación, impresión en gran formato, grabado e instalación. Atiende a colegios, clubes deportivos, municipalidades, fundaciones y empresas. El diagnóstico de la Unidad 1 identificó el problema central confirmado por la contraparte: **los trabajadores no saben con certeza en qué proceso va una fabricación ni a qué área corresponde continuar**. La producción se detiene cuando no están las personas a cargo (Bianca, Berny, René), lo que genera atrasos permanentes. Hoy el seguimiento depende de un Excel y de la memoria de la encargada de producción. La información está duplicada entre ERP, Softland, formularios, correo y WhatsApp, sin integración. A esto se suma una gestión reactiva de insumos, como tinta de plotter o papel de sublimación agotados, que solo se detecta cuando el trabajo ya debía comenzar.

Este informe traduce ese diagnóstico en un **diseño de software verificable**. Cubre la planificación Scrum de la construcción, la arquitectura según el modelo 4+1 de Kruchten, la tecnología e infraestructura, el diseño de interfaces con prototipo navegable, el modelo de datos estructurado y no estructurado, la seguridad y el plan de pruebas. Cada decisión se justifica frente a tres criterios de éxito expresados por el cliente:

1. **Autonomía del trabajador:** el trabajador sabe qué hacer después sin depender de una persona encargada.
2. **Simplicidad:** un sistema "fácil, práctico y amigable", accesible desde el celular.
3. **Alcance acotado:** sin contactos de clientes, sin dinero y con presupuesto no definido.

La expectativa de "una app para el celular" se interpreta como un **requisito de acceso y usabilidad** y no como una tecnología impuesta. Por eso se opta por una Aplicación Web Progresiva (PWA) instalable, cuya justificación se presenta en la sección 4.7.

2 Síntesis de la solución propuesta

SGP-Kotoduo es un sistema web progresivo de apoyo a la producción. Centraliza cada trabajo desde su ingreso hasta su entrega, lo modela como una **orden de trabajo (OT)** compuesta por **etapas** generadas a partir de **plantillas de proceso** por tipo de trabajo, e indica a cada trabajador su **siguiente actividad, área e instrucciones**. Además, anticipa faltantes de insumos y deja trazabilidad de cada acción.

Alcance de la versión 1.0: registro y seguimiento de OT, plantillas de proceso, avance de etapas desde el móvil, aprobación de diseño, insumos con alertas, panel de supervisión, bitácora de auditoría y control de acceso por roles. **Fuera de alcance** (por decisión del cliente o por delimitación): datos de contacto de clientes, precios, pagos y facturación, integración con Softland o el ERP, compras automáticas a proveedores y remuneraciones.

Usuarios: Administrador, Encargada de Producción, Vendedor, Diseñador y Operario. Estos roles formalizan los perfiles "administrador" y "de respuesta" mencionados por la contraparte.

Módulos:

Código	Módulo	Capacidad principal
M1	Autenticación y Usuarios	Inicio de sesión, MFA, gestión de cuentas y roles
M2	Plantillas de Proceso	Tipos de trabajo con etapas, áreas, instrucciones, tiempos e insumos
M3	Órdenes de Trabajo	Registro de OT, generación de etapas y asignación de responsables
M4	Mi Trabajo / Seguimiento de Etapas	Vista personal, iniciar/terminar/bloquear etapa y evidencias
M5	Insumos y Alertas	Stock, kardex de movimientos, alertas automáticas y manuales
M6	Aprobación de Diseño	Versiones de diseño y registro de la aceptación del cliente
M7	Panel de Supervisión	Estado global, avance, órdenes atrasadas o en riesgo
M8	Bitácora y Auditoría	Registro inmutable de eventos, consultable por orden y usuario

2.1 Línea base de requerimientos (Informe N.º 1)

La tabla consolida los requerimientos priorizados en la Unidad 1. La columna "Origen" distingue lo **confirmado (C)** por la contraparte de lo que se trabaja como **supuesto (S)**, pendiente de validación en la actividad A01.

ID	Requerimiento funcional	Origen	Prioridad (MoSCoW)
RF-01	Autenticar usuarios y administrar cuentas con roles y permisos diferenciados	C	Must
RF-02	Definir tipos de trabajo y plantillas con etapas ordenadas, área, instrucciones, duración e insumos	S	Must
RF-03	Registrar órdenes de trabajo con datos de producción (medidas, cantidades, especificaciones), sin contactos ni montos	C	Must
RF-04	Generar las etapas de la OT desde la plantilla y asignar o reasignar responsables	S	Must
RF-05	Mostrar a cada trabajador sus etapas, el estado actual, la siguiente actividad, el área y las instrucciones	C	Must
RF-06	Registrar inicio, término y bloqueo de etapas, habilitando y notificando la siguiente etapa	C	Must
RF-07	Adjuntar evidencias (fotos, notas) a una etapa	S	Could
RF-08	Registrar versiones de diseño y la aceptación o rechazo del cliente	C	Should
RF-09	Mantener insumos con stock mínimo y registrar ingresos y consumos	S	Should
RF-10	Generar alertas de insumo automáticas (bajo mínimo) y manuales (faltante reportado)	S	Should
RF-11	Panel de supervisión con OT en curso, detenidas, atrasadas y fecha estimada de término	S	Should

ID	Requerimiento funcional	Origen	Prioridad (MoSCoW)
RF-12	Registrar bitácora de auditoría (usuario, fecha, acción, estado anterior y nuevo)	C	Must

ID	Requerimiento no funcional	Métrica de aceptación
RNF-01	Usabilidad: interfaz en español, *mobile-first*	Tarea "ver siguiente actividad" en ≤ 3 toques; SUS ≥ 70
RNF-02	Portabilidad: acceso desde celular y PC sin tiendas de apps	PWA instalable en Android 10+ / iOS 16.4+ y navegadores modernos
RNF-03	Rendimiento	p95 < 500 ms en lecturas con 40 usuarios concurrentes; LCP $< 2{,}5$ s en 4G
RNF-04	Seguridad	Controles OWASP ASVS 4.0.3 nivel 2 en autenticación, sesión y control de acceso; TLS 1.2+
RNF-05	Trazabilidad	100 % de cambios de estado registrados en la bitácora; retención de 24 meses
RNF-06	Disponibilidad y respaldo	99 % en horario laboral; RPO 24 h; RTO 4 h
RNF-07	Mantenibilidad	TypeScript estricto, arquitectura en capas, cobertura ≥ 70 % en servicios
RNF-08	Costo	Operación con servicios gestionados de bajo costo ($\approx$ US$ 10/mes), sin licencias
RNF-09	Privacidad (Ley 19.628 / 21.719)	Minimización: sin datos de contacto de clientes; datos personales de trabajadores limitados a lo operativo

ID	Historia de usuario	RF
HU-01	Como **Operario** quiero ver mis etapas y la siguiente actividad con sus instrucciones para continuar sin esperar a la encargada	RF-05
HU-02	Como **Operario** quiero marcar una etapa como iniciada o terminada para que el trabajo avance a la siguiente área	RF-06
HU-03	Como **Operario** quiero reportar un insumo faltante desde la etapa para que se gestione su reposición	RF-10
HU-04	Como **Operario** quiero adjuntar una foto o nota a la etapa para dejar constancia del trabajo	RF-07
HU-05	Como **Vendedor** quiero registrar una orden eligiendo el tipo de trabajo para que producción reciba la información completa	RF-03, RF-04
HU-06	Como **Diseñador** quiero subir versiones del diseño y, como **Vendedor**, registrar la respuesta del cliente, para que producción no comience sin un diseño aprobado	RF-08
HU-07	Como **Encargada de Producción** quiero definir plantillas de proceso por tipo de trabajo para estandarizar las etapas	RF-02

ID	Historia de usuario	RF
HU-08	Como **Encargada de Producción** quiero asignar y reasignar responsables de etapas para distribuir la carga	RF-04
HU-09	Como **Encargada de Producción** quiero recibir alertas de insumos bajo el mínimo antes de que detengan la producción	RF-09, RF-10
HU-10	Como **Encargada / Administrador** quiero un panel con el estado y los atrasos de las órdenes para priorizar	RF-11
HU-11	Como **Administrador** quiero gestionar usuarios y roles para controlar el acceso	RF-01
HU-12	Como **Administrador** quiero consultar la bitácora para saber quién cambió qué y cuándo	RF-12

2.2 Trazabilidad necesidad → componente → evidencia

Necesidad / requerimiento de origen	Componente de la solución que responde	Evidencia en este informe
Trabajadores no saben la siguiente etapa ni el área (RF-05, RF-06; HU-01, HU-02)	M4 Mi Trabajo + máquina de estados de etapa + notificación push	Fig. 4.17, Fig. 4.7, CU-05/CU-06, pantallas P-02/P-03, CP-01, CP-12, CP-33
Proceso en Excel y memoria de una persona (RF-02, RF-04; HU-07)	M2 Plantillas de Proceso + generación automática de etapas	Fig. 4.3, tablas `plantilla_etapa` y `etapa_orden`, P-07, CP-03, CP-11
Información duplicada y dispersa (RF-03; HU-05)	M3 Órdenes de Trabajo como fuente única + PostgreSQL	Sección 6.1–6.3, P-04, CP-06, CP-11
Gestión reactiva de insumos (RF-09, RF-10; HU-03, HU-09)	M5 Insumos y Alertas: kardex con trigger de alerta y faltantes proyectados	Sección 6.2 (R3), Fig. 4.15, P-08, CP-05, CP-14, CP-15
Aceptación del diseño sin registro formal (RF-08; HU-06)	M6 Aprobación de Diseño con versiones y segregación de funciones	Fig. 4.15, tabla `aprobacion_diseno`, P-05, CP-16
Falta de visibilidad global del avance (RF-11; HU-10)	M7 Panel de Supervisión + vista `v_panel_ordenes`	P-06, CP-18, CP-23
Usuarios y permisos diferenciados (RF-01; HU-11)	M1 + RBAC compartido entre front-end y API	Secciones 7.1–7.3, CP-09, CP-10, CP-27, CP-28
Trazabilidad de cambios y responsables (RF-12; HU-12)	M8 Bitácora: outbox transaccional → MongoDB	Secciones 6.4–6.5, Fig. 4.18, CP-11
App práctica para celular (RNF-01, RNF-02)	PWA *mobile-first* instalable	Secciones 4.7 y 5.4, CP-22, CP-34

Necesidad / requerimiento de origen	**Componente de la solución que responde**	**Evidencia en este informe**
Sin presupuesto definido (RNF-08)	Servicios gestionados con capa gratuita o de bajo costo; infraestructura como código como mejora ideal para cambiar de proveedor sin rediseñar	Sección 4.7 (Tablas 4.2 y 4.3)

3 Planificación del proyecto

3.1 Actividades, fases, plazos y responsables

Metodología. Se adopta **Scrum** (Schwaber & Sutherland, 2020). Los requerimientos están parcialmente confirmados (sección 2.1) y la contraparte debe validar flujos, roles y alcance de insumos. Un enfoque iterativo con revisiones frecuentes reduce el riesgo de construir sobre supuestos erróneos. Kanban se descartó como marco principal porque el proyecto tiene hitos académicos fijos que requieren compromisos por iteración. El tablero Kanban se usa, en cambio, para visualizar cada Sprint Backlog.

Desarrollo asistido por IA y supuesto de capacidad. La construcción dispone de aproximadamente un mes (20 días hábiles). El equipo usa asistentes de codificación basados en IA para generar código, migraciones, pruebas automatizadas y documentación técnica a partir del contrato OpenAPI, el DDL y las HU. Con estas herramientas, escribir código deja de ser la restricción del proyecto. El cuello de botella pasa a ser la **verificación**: revisión humana de cada PR, ejecución y análisis de pruebas, pruebas de seguridad, validación con el cliente y pruebas de usabilidad con trabajadores reales. Por eso, las duraciones del plan detallado (Anexo H, Tabla H.1) se estiman según el esfuerzo de verificación de cada actividad, no según el esfuerzo de escribir el código. Las actividades de prueba conservan una mayor proporción del tiempo de cada Sprint. El riesgo de que el código generado introduzca defectos se controla con la Definition of Done: ningún incremento se acepta sin revisión humana, pruebas en verde y análisis de seguridad.

Configuración del marco:

Elemento Scrum	**Definición en el proyecto**
Product Owner	Representante de Kotoduo (por validar: Koto o Bianca). **Integrante 2** actúa como PO proxy: mantiene y prioriza el backlog entre reuniones con el cliente
Scrum Master	**Integrante 1**: facilita los eventos, gestiona impedimentos y vigila el burndown
Equipo de desarrollo	Integrante 1 (back-end, datos, seguridad, infraestructura) y Integrante 2 (UX/UI, front-end, pruebas)
Duración del Sprint	Sprint 0: 10 días hábiles (diseño). Sprints 1–4: 5 días hábiles (1 semana) cada uno; la construcción completa dura 20 días hábiles ($\approx$ 1 mes)
Capacidad	$\approx$ 4 h/día por integrante, dedicadas principalmente a especificar, revisar y verificar el código generado con asistencia de IA; velocidad objetivo de 21–26 story points (SP) por Sprint semanal, limitada por la capacidad de verificación
Plazos	**Relativos, en días hábiles (D1...D30)** desde el inicio del proyecto; sin fechas calendario fijas
Eventos	Planning (día 1 de cada Sprint, 2 h); Daily asíncrona (15 min, mensaje en canal del equipo); Review con el cliente y Retrospectiva (día 5 de cada Sprint)
Definition of Ready	HU con criterios de aceptación, maqueta asociada, dependencias resueltas y estimación en SP

Elemento Scrum	Definición en el proyecto
Definition of Done	Código (incluido el generado con asistencia de IA) revisado por el otro integrante (PR), pruebas unitarias e integración en verde en CI, sin hallazgos High en análisis de seguridad, desplegado en staging y validado por el PO proxy

Asignación por capacidades. Se asigna a cada integrante un responsable principal por actividad para no diluir la responsabilidad.

Integrante	Rol	Capacidad considerada	Responsabilidad principal
Integrante 1	Scrum Master / Arquitecto / Back-end	Modelado de datos, programación de servicios, seguridad e infraestructura cloud	A00, A03, A05, A07, A09–A12, A14, A16, A18, A20, A21, A24, A26, A27, A29
Integrante 2	PO proxy / UX-UI / Front-end / QA	Levantamiento con usuarios, diseño de interfaces, desarrollo web y diseño de pruebas	A01, A02, A04, A06, A08, A13, A15, A17, A19, A22, A23, A25, A28, A30, A31

Product Backlog priorizado (Tabla 3.1).

Orden	Ítem	Descripción	SP	Prioridad	Sprint
1	TE-01	Habilitador técnico: repositorio, CI, ambientes, migraciones	5	Alta	S1
2	HU-11	Gestionar usuarios y roles + inicio de sesión	8	Alta	S1
3	HU-07	Plantillas de proceso	8	Alta	S1
4	HU-05	Registrar orden de trabajo	8	Alta	S2
5	HU-08	Asignar responsables	5	Alta	S2
6	HU-01	Ver Mi Trabajo y siguiente actividad	5	Alta	S2
7	HU-02	Iniciar / terminar etapa y avanzar	8	Alta	S2
8	HU-09	Alertas de insumos	8	Media	S3
9	HU-03	Reportar faltante	3	Media	S3
10	HU-06	Aprobación de diseño	5	Media	S3
11	HU-04	Evidencias de etapa	5	Baja	S3
12	HU-10	Panel de supervisión	8	Media	S4
13	HU-12	Bitácora de auditoría	5	Media	S4
14	TE-02	Habilitador: pruebas no funcionales, endurecimiento y piloto	8	Alta	S4
		Total	**89**		

Plan por Sprint (Tabla 3.2). Resume el cronograma de 30 días hábiles y sustituye a la carta Gantt. El detalle de las 32 actividades (A00–A31), con su entregable, responsable, día de inicio, día de término y duración, está en el **Anexo H (Tabla H.1)**.

	Sprint 0 (D1–D10)	**Sprint 1 (D11–D15)**	**Sprint 2 (D16–D20)**	**Sprint 3 (D21–D25)**	**Sprint 4 (D26–D30)**
Objetivo	Diseño validado de la solución	Acceso seguro y plantillas operativas	Una OT recorre todas sus etapas	Insumos anticipados y diseño aprobado	Supervisión, calidad y puesta en marcha
Actividades	A00–A09	A10–A15	A16–A19	A20–A23	A24–A31
Ítems del backlog	—	TE-01, HU-11, HU-07	HU-05, HU-08, HU-01, HU-02	HU-09, HU-03, HU-06, HU-04	HU-10, HU-12, TE-02
SP comprometidos	—	21	26	21	21
Incremento	Informe N.º 2 + prototipo	Login + MFA + plantillas	Flujo productivo completo	Alertas, aprobación y evidencias	Piloto en producción
Review con cliente	D10 (**Hito H1**)	D15	D20	D25	D30 (**Hito H2**)

Los días D15, D20 y D25 se reservan para Sprint Review y Retrospectiva por la mañana y para **buffer de contingencia** por la tarde. Cada Sprint de construcción tiene así 4 días de trabajo planificado y una holgura explícita de medio día. Los días 1 de cada Sprint (D11, D16, D21, D26) comienzan con la Sprint Planning de 2 h. En el Sprint 4, las actividades de prueba (A26–A28) ocupan la mayor parte del tiempo porque son el cuello de botella del proyecto. El día D30 concentra el despliegue y la capacitación por la mañana, y la Review final por la tarde.

Tablero del Sprint 2 (estado planificado al día D18; Tabla 3.3).

Por hacer	**En progreso**	**En revisión / QA**	**Hecho**
HU-02 UI “Terminar etapa” (A19)	HU-02 API avance de etapa (A18)	HU-05 UI registro OT (A17)	HU-05 API registro OT (A16)
HU-01 UI Mi Trabajo (A19)		HU-08 UI asignación (A17)	HU-08 API asignación (A16)

Burndown planificado del release (Tabla 3.4). Se actualiza al cierre de cada Sprint, comparando el valor real contra el ideal.

Momento	**SP restantes (ideal)**	**SP restantes (real)**
Inicio S1 (D11)	89	[Completar]
Fin S1 (D15)	68	[Completar]
Fin S2 (D20)	42	[Completar]
Fin S3 (D25)	21	[Completar]
Fin S4 (D30)	0	[Completar]

El Sprint 2 concentra 26 SP, por encima de la velocidad media, porque contiene el flujo de valor central. La mitigación es que HU-08 (5 SP) puede pasar al Sprint 3 sin bloquear HU-01/HU-02, ya que la asignación puede hacerse temporalmente por datos semilla.

3.2 Dependencias y prioridades entre tareas

Tipos de dependencia: **FC** (fin-comienzo, la sucesora comienza al terminar la predecesora) y **CC** (comienzo-comienzo, la sucesora puede comenzar cuando comienza la predecesora, porque comparten un contrato ya definido, como el OpenAPI de A03). La prioridad responde al valor para el cliente, al riesgo técnico y a la posición en la ruta crítica, no al orden cronológico.

La matriz completa, con la predecesora, el tipo de dependencia, la prioridad, la justificación del orden y el efecto de un retraso de cada una de las 32 actividades, está en el **Anexo H (Tabla H.2)**. La Tabla 3.5 resume las dependencias que más condicionan el plan.

Tabla 3.5 — Dependencias que condicionan la planificación

Actividad	Depende de	Por qué tiene prioridad	Efecto si se retrasa
A01 Validar la línea base con Kotoduo	—	Todo el diseño descansa en supuestos que el cliente debe validar	Cada día retrasa A02–A04; riesgo de diseñar flujos incorrectos
A03 Arquitectura y contrato OpenAPI	A00, A01	Define los módulos, el contrato API y la tecnología de los que dependen datos, seguridad y pruebas; el contrato permite que el front-end avance en paralelo (dependencias CC)	Ruta crítica: +n d desplaza H1 en n d
A12 API de autenticación y usuarios	A07, A11	Toda la API exige identidad y permisos	Ruta crítica: +1 d consume el buffer D15 y desplaza S2 en medio día
A18 Avance de etapa transaccional	A16	Es el corazón de RF-05/RF-06	Ruta crítica; el problema principal del cliente queda sin resolver
A27 Pruebas de seguridad	A24, A23	Deben cubrir la superficie completa antes de producción	No se autoriza pasar a producción (control de salida)
A29 Despliegue en producción	A26, A27, A28	Solo se despliega lo que pasó las pruebas (DoD)	Desplaza H2

Ruta crítica (cadena del recurso back-end, 0 días de holgura salvo los buffers de Sprint): A00 (2 d) → A03 (3 d) → A05 (2 d) → A07 (2 d) → A09 (1 d) → A10 (1 d) → A11 (0,5 d) → A12 (1,5 d) → A14 (1 d) → [Review y buffer D15] → A16 (2 d) → A18 (2 d) → [Review y buffer D20] → A20 (2 d) → A21 (2 d) → [Review y buffer D25] → A24 (1 d) → A26 (1 d) → A27 (2 d) → A29 (0,5 d) → A31 (0,5 d) = **30 días hábiles** (10 de diseño + 20 de construcción).

Gestión de retrasos: un retraso de hasta medio día por Sprint se absorbe con el buffer del día 5. Si es mayor, el PO proxy reduce el alcance del Sprint y mueve al siguiente el ítem de menor prioridad (HU-04 y HU-08 son los candidatos definidos). Así se protegen los hitos H1 y H2 sin comprometer la DoD.

4 Arquitectura de la solución

Criterio de legibilidad. Ningún diagrama intenta mostrar todo el sistema. Cada vista parte de una **visión general** sin detalle y luego presenta **acercamientos** por tema. Los elementos en gris («ref») se detallan en otra figura y solo se muestran para dar contexto.

4.1 Descripción general de la arquitectura 4+1

SGP-Kotoduo adopta un **monolito modular en capas con cliente PWA** (Figura 4.1). El front-end es una SPA/PWA estática. El back-end es una API REST única organizada en los módulos M1–M8, cada uno con capas *routes* → *controller* → *service* → *repository*. La persistencia es **políglota**: PostgreSQL para los datos transaccionales con integridad referencial y MongoDB para eventos, evidencias y notificaciones de esquema variable. Ambas se integran mediante un **outbox transaccional**.

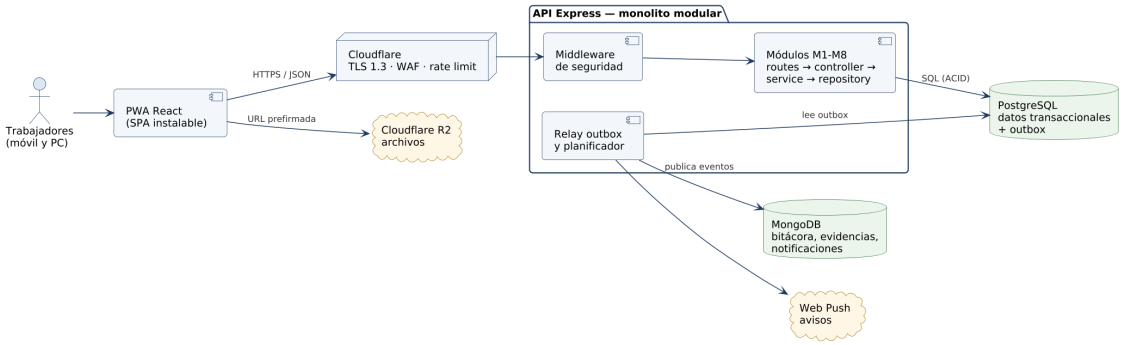
Se descartaron los microservicios: con dos desarrolladores, alrededor de 25 usuarios y cerca de 30 OT por semana, su costo operativo (despliegues, observabilidad distribuida, consistencia eventual generalizada) no aporta valor. Los límites de módulo permiten extraer servicios en el futuro, por ejemplo M5 si se integra con proveedores.

Vista (Kruchten, 1995)	Pregunta que responde	Interesados	Diagramas	Figuras	Sección
Lógica	¿Qué abstracciones y responsabilidades tiene el sistema?	Usuarios finales, analistas	Clases (dominio y servicios), estados	4.2–4.8	4.2
Desarrollo	¿Cómo se organiza el código y sus dependencias?	Programadores, jefe de proyecto	Componentes, paquetes	4.9–4.13	4.3
Procesos	¿Qué flujos, concurrencia y sincronización existen?	Integradores de sistemas	Actividades con carriles (BPMN), secuencia, tabla de procesos	4.14–4.18	4.4
Física	¿Dónde se ejecuta cada elemento y cómo se comunica?	Infraestructura y administración	Despliegue	4.19–4.20	4.5
Escenarios (+1)	¿Qué casos de uso validan y unen las cuatro vistas?	Todos, incluido el cliente	Casos de uso	4.21–4.24	4.6

Coherencia entre vistas. El escenario CU-06 “Actualizar avance de etapa” recorre las cinco vistas:

- **Lógica:** clase `EtapaOrden`, su máquina de estados y el método `EtapaService.terminar()` (Fig. 4.3, 4.6 y 4.7).
- **Desarrollo:** se implementa en el componente M4 y en el paquete `modules/etapas` (Fig. 4.11–4.13).
- **Procesos:** forma parte del bucle de producción (Fig. 4.14 y 4.16), se ejecuta en una transacción (Fig. 4.17) y genera trabajo asíncrono en el relay outbox (Fig. 4.18).
- **Física:** se ejecuta en el contenedor de Render y persiste en Neon y Atlas (Fig. 4.19).
- **Escenarios:** es el caso de uso central del taller (Fig. 4.23) y se valida con CP-01, CP-12, CP-19 y CP-27.

Figura 4.1 — Vista general de capas. Resume el recorrido de una petición: la PWA entra por el borde de Cloudflare, el middleware de seguridad la filtra y los módulos M1–M8 la resuelven contra PostgreSQL. El relay publica después los eventos en MongoDB y Web Push. Los archivos viajan directo de la PWA a R2.



4.2 Vista lógica

La vista lógica se presenta en siete figuras: un mapa del dominio (Fig. 4.2), tres acercamientos por grupo de clases (Fig. 4.3–4.5), los servicios que coordinan esas clases (Fig. 4.6) y las máquinas de estados de la etapa (Fig. 4.7) y de la orden de trabajo (Fig. 4.8).

Simbología común de los diagramas de clases: cada rectángulo es una clase con atributos privados (`-`) y operaciones públicas (`+`). Los rombos rellenos (♦, composición) indican que la parte no existe sin el todo: una `EtapaOrden` no existe sin su `OrdenTrabajo` y una `PlantillaEtapa` no existe sin su `TipoTrabajo`. Los rombos vacíos (◇, agregación) indican pertenencia débil: los `MovimientoInsumo` se agregan al `Insumo`. Las flechas continuas son asociaciones navegables con multiplicidad (`1`, `0..1`, `*`, `1..*`). Las flechas punteadas son dependencias de uso. Los `enum` fijan los estados válidos.

Figura 4.2 — Modelo de dominio: visión general. Muestra las 13 clases del dominio sin atributos, agrupadas en tres paquetes. Cada paquete indica la figura donde se detalla.

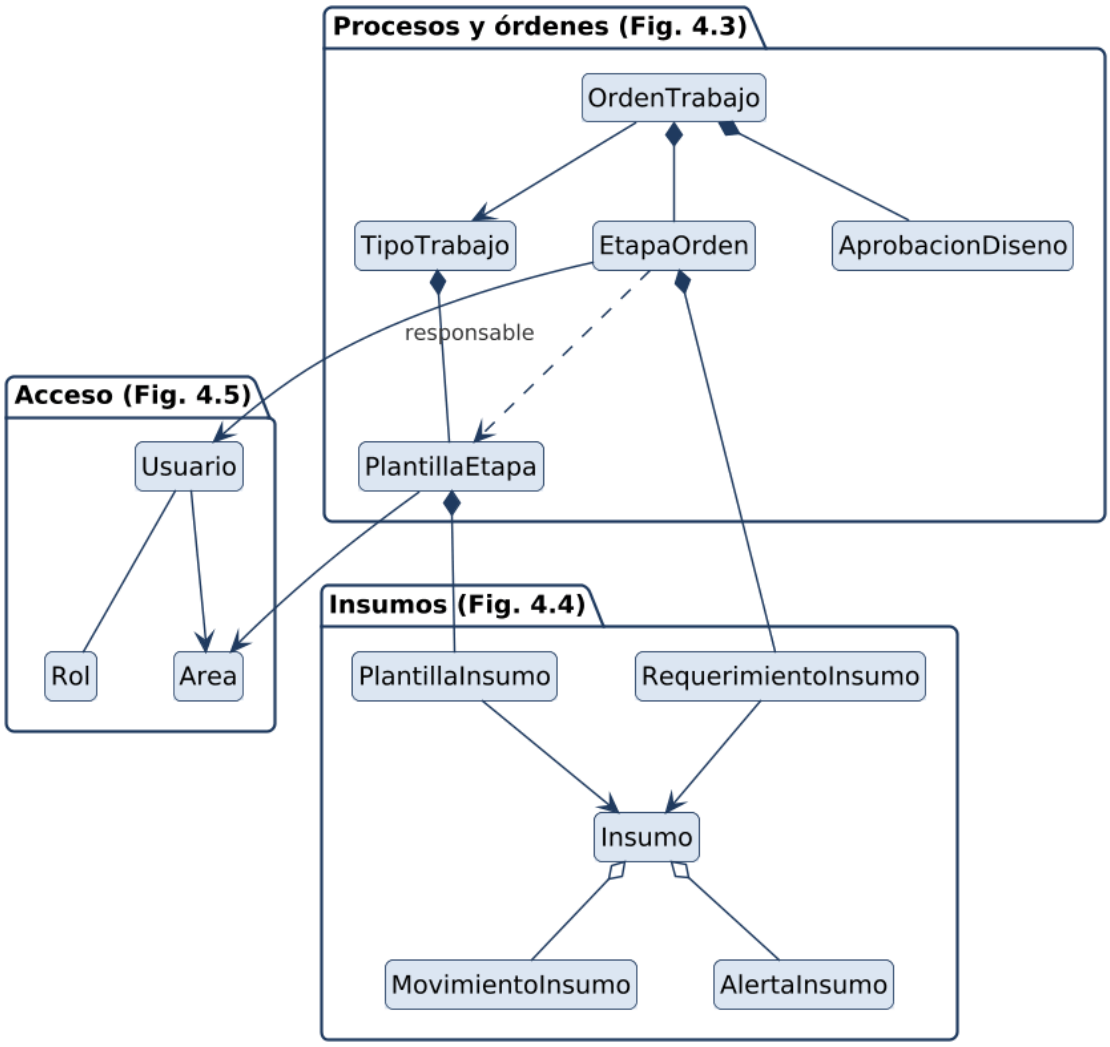


Figura 4.3 — Dominio: procesos y órdenes de trabajo. Es el núcleo del sistema: el tipo de trabajo define sus etapas estándar y cada orden se compone de sus propias etapas y de las versiones del diseño.

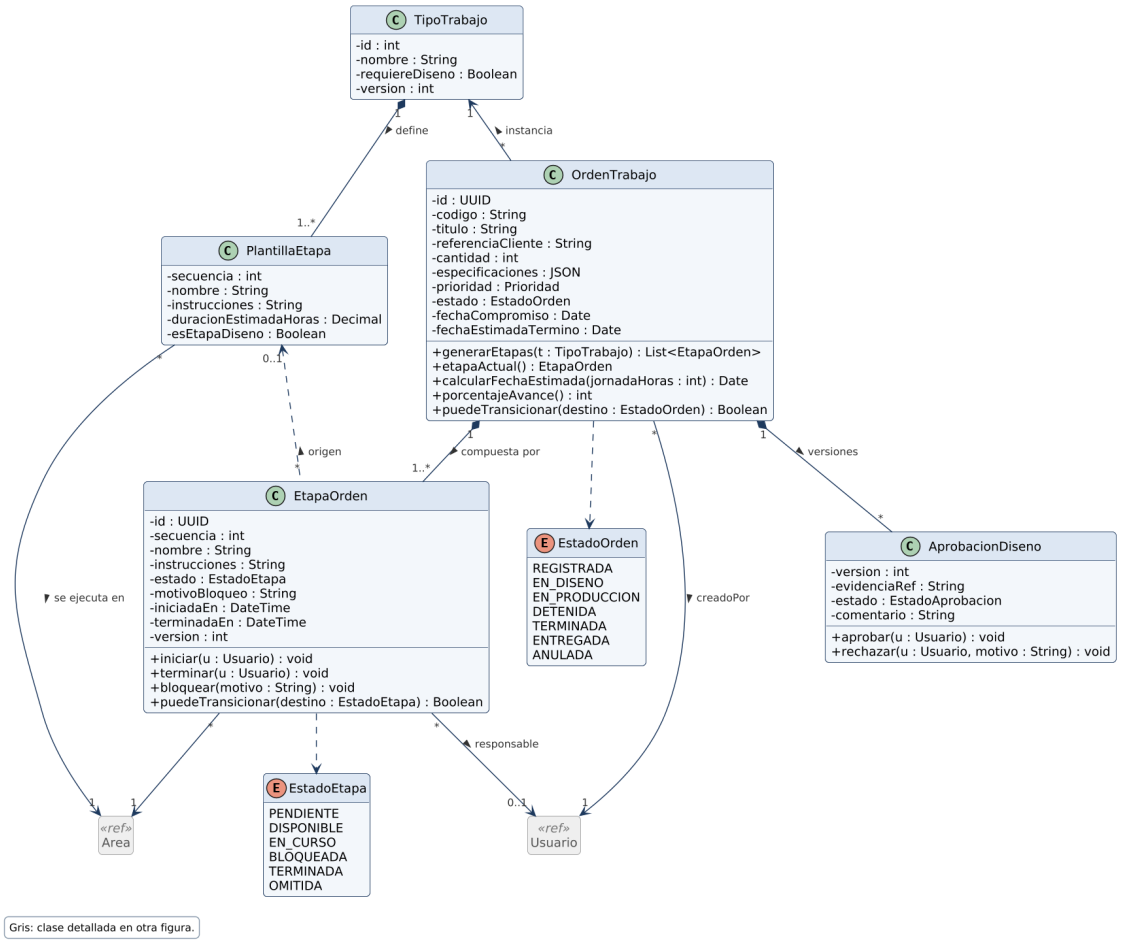


Figura 4.4 — Dominio: insumos, consumo y alertas. Separa el consumo estándar por plantilla (`PlantillaInsumo`) del requerimiento concreto de cada etapa (`Requerimiento Insumo`). El stock se explica por el kardex de movimientos.

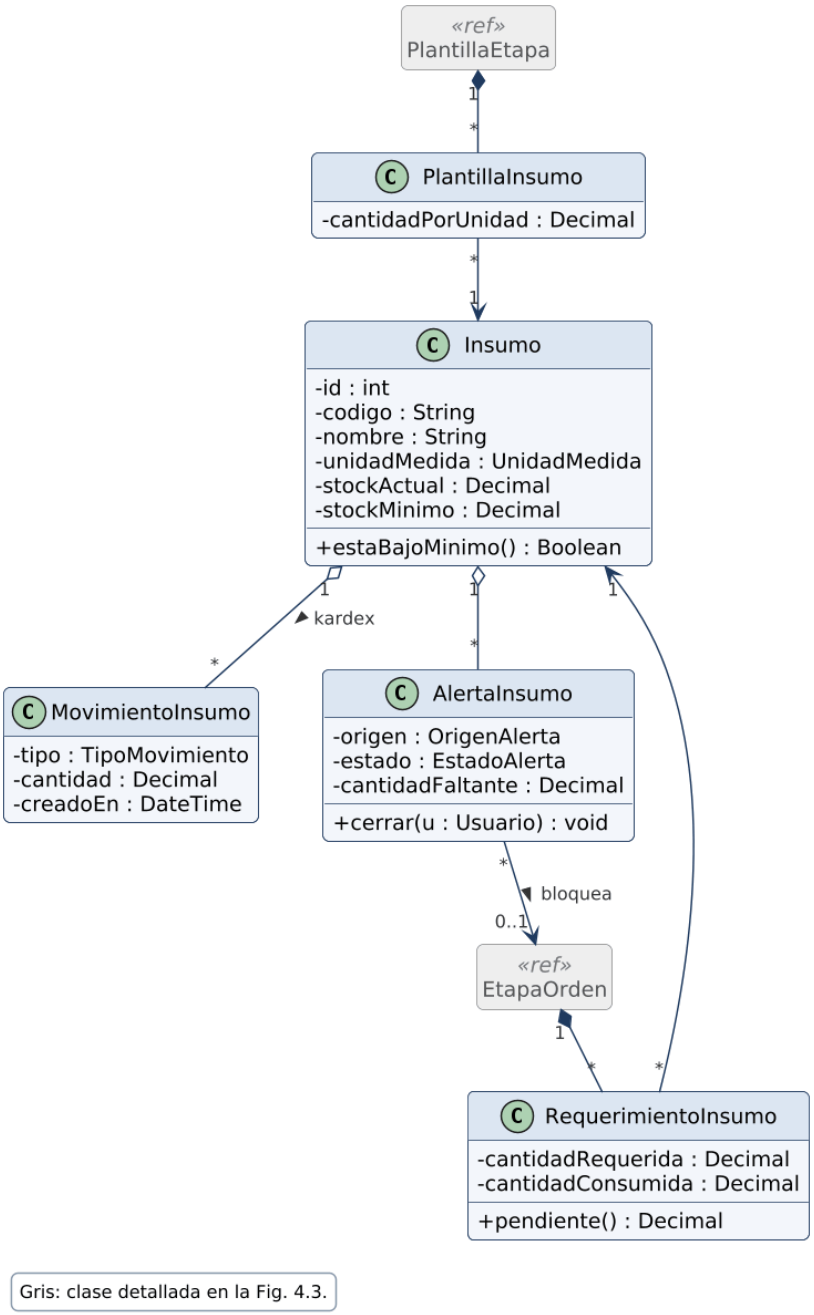


Figura 4.5 — Dominio: usuarios, roles y áreas. Un usuario puede tener varios roles y pertenecer a un área. Los cinco códigos de rol son la base del RBAC (sección 7.1).

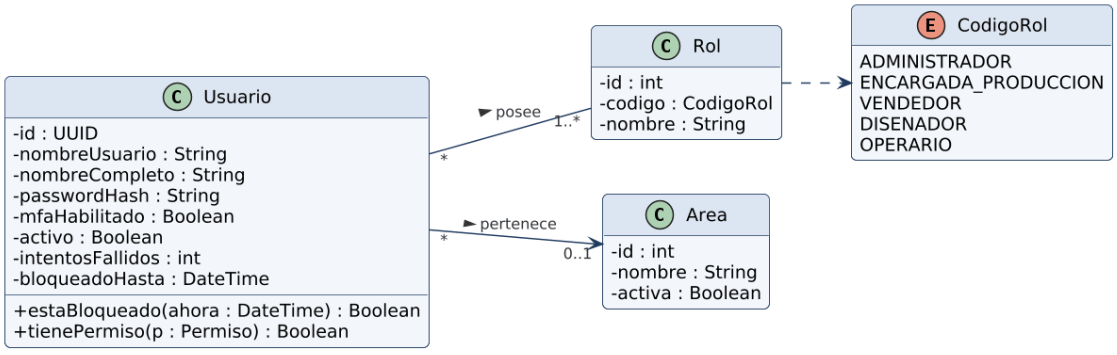
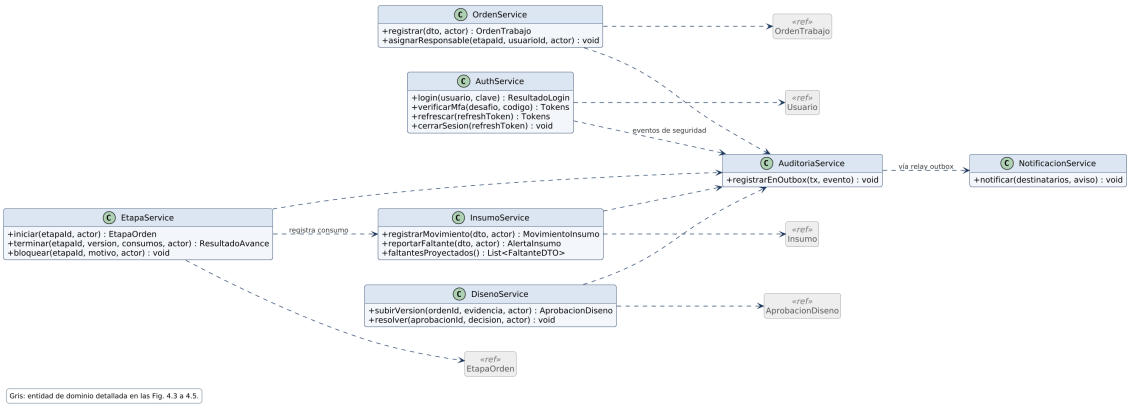


Figura 4.6 — Servicios de aplicación. Cada servicio coordina una entidad del dominio. `EtapaService` delega el consumo en `InsumoService`. Todos los servicios que cambian estado (órdenes, etapas, insumos y aprobaciones de diseño), además de `AuthService` con los eventos de seguridad, registran el evento en `AuditoriaService`, que notifica de forma diferida mediante el relay outbox.



Decisiones de diseño:

1. **`TipoTrabajo` + `PlantillaEtapa`** externalizan el conocimiento hoy concentrado en Bianca, Berny y René. La secuencia, el área, las instrucciones y la duración pasan a ser datos editables (RF-02).
2. **`EtapaOrden` copia** nombre, área, instrucciones y duración desde la plantilla al crearse la OT. Así, modificar una plantilla no altera órdenes en curso, y se mantiene la referencia opcional `origen` para análisis.
3. La lógica de transición vive en `EtapaOrden.puedeTransicionar()` y `OrdenTrabajo.puedeTransicionar()`, con una defensa en profundidad en la base de datos (trigger R2, sección 6.2).
4. Los servicios no se llaman en cadena síncrona hacia MongoDB. `AuditoriaService` solo escribe en el outbox dentro de la misma transacción.

Figura 4.7 — Diagrama de estados de `EtapaOrden`. Cada nodo es un estado y cada arco es una transición etiquetada con *evento [condición] / acción*. El estado compuesto *Abierta* agrupa los estados en que la etapa todavía puede avanzar. El camino principal es PENDIENTE→DISPONIBLE→EN_CURSO→TERMINADA. Una etapa pasa a DISPONIBLE cuando la anterior queda cerrada, sea TERMINADA u OMITIDA, de modo que omitir una etapa no traba la secuencia. La etapa de diseño (`es_etapa_diseno`) no se cierra con el botón **Terminar**: se cierra cuando el cliente aprueba el diseño. **Reportar problema** exige motivo (`ck_etapa_bloqueo`), avisa a la Encargada y detiene la OT, y solo la Encargada desbloquea la etapa. Si la OT se anula, toda etapa abierta pasa a OMITIDA. El mismo grafo se implementa en el dominio (TypeScript) y en el trigger `trg_etapa_transicion`, que además impide iniciar o terminar una etapa mientras la OT está DETENIDA, y se prueba en CP-01 y CP-02.

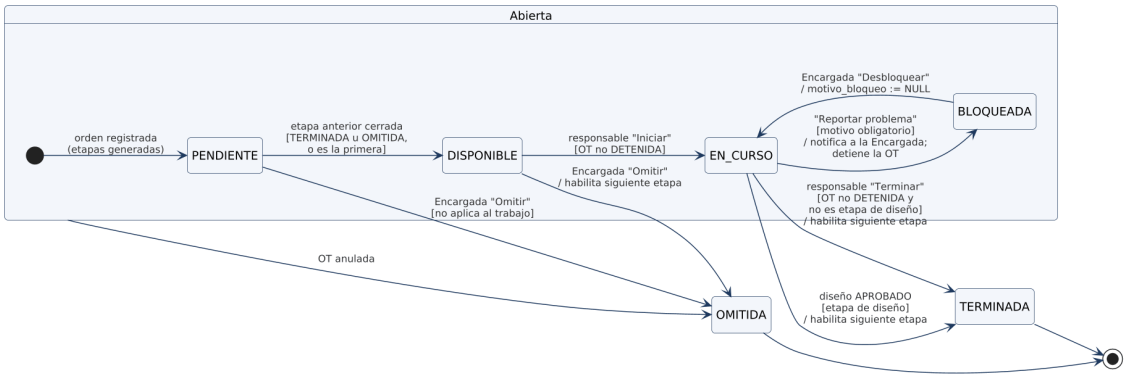
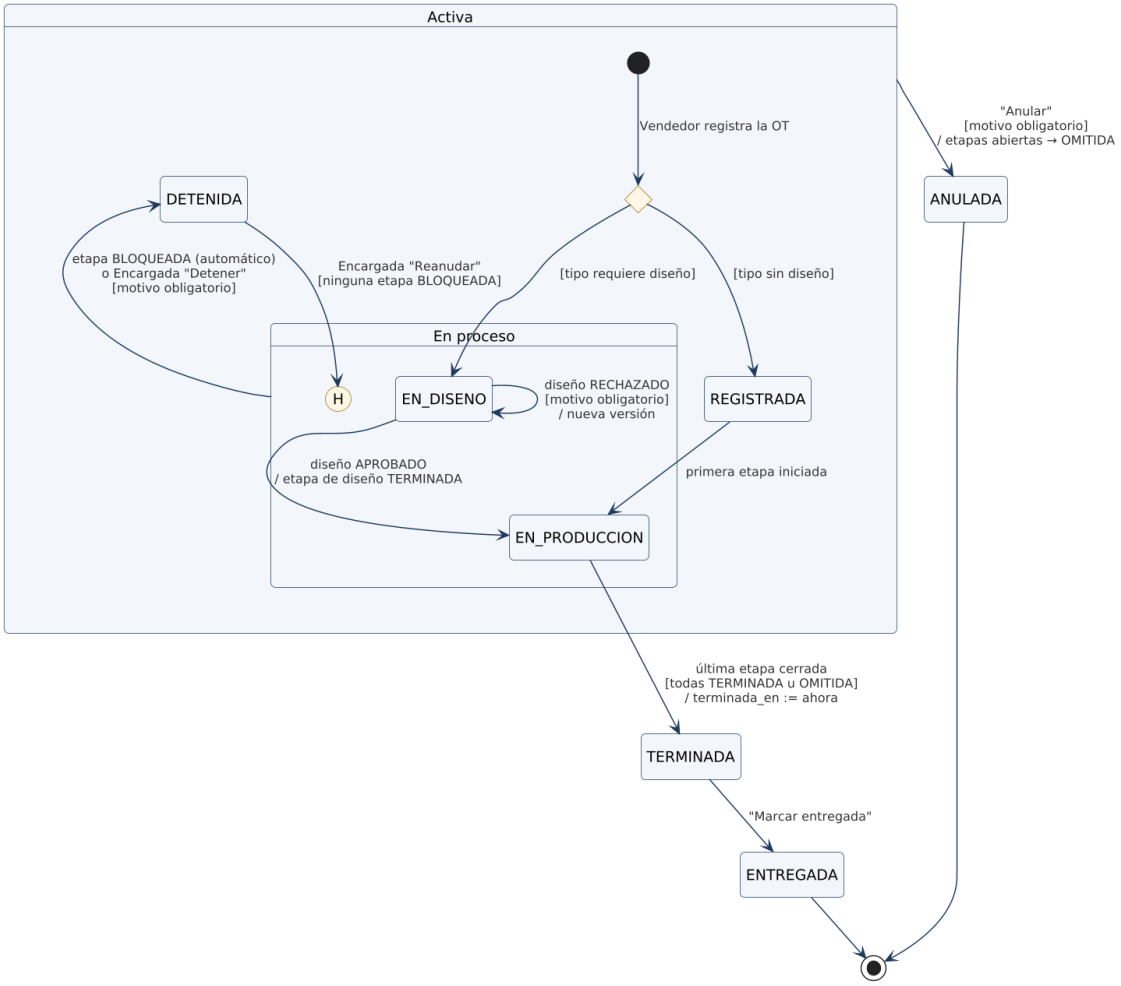


Figura 4.8 — Diagrama de estados de `OrdenTrabajo`. La OT nace en REGISTRADA o, si su tipo de trabajo requiere diseño, en EN_DISENO. Pasa a EN_PRODUCCION cuando el cliente aprueba el diseño o cuando se inicia su primera etapa, y a TERMINADA cuando todas sus etapas quedan cerradas (CP-13). Queda **DETENIDA** en dos casos: de forma automática, cuando una de sus etapas se bloquea, o de forma manual, cuando la Encargada la detiene con un motivo. Solo la Encargada la reanuda, y únicamente si no quedan etapas bloqueadas. El pseudoestado de historia (H) la devuelve al subestado que tenía antes de detenerse (EN_DISENO o EN_PRODUCCION). Mientras está DETENIDA no se inicia ni se termina ninguna etapa (Fig. 4.7). Mientras está activa puede anularse con motivo, y entonces sus etapas abiertas pasan a OMITIDA. El motivo de una detención o de una anulación queda en la bitácora. Las transiciones y sus condiciones (etapas cerradas para terminar, ninguna etapa bloqueada para reanudar) se validan en `OrdenTrabajo.puedeTransicionar()` y en el trigger `trg_orden_transicion`.



Las otras dos entidades con estado tienen ciclos lineales que no requieren diagrama:

- **`AprobacionDiseno`:** PENDIENTE → APROBADO o RECHAZADO, y ambos son finales. Rechazar exige comentario, y quien subió el diseño no puede resolverlo (R6). Una nueva propuesta es una nueva versión (fila nueva), no un cambio de estado.
- **`AlertaInsumo`:** ABIERTA → EN_GESTION → CERRADA (IK-9). CERRADA es final y exige `cerrada_en`.

4.3 Vista de desarrollo o implementación

Simbología (UML 2) de los diagramas de componentes: los componentes (rectángulos con ícono de componente) son unidades desplegables o reemplazables. El puerto "HTTPS /api/v1" es la interfaz provista por la API. Las flechas continuas son conectores de uso en tiempo de ejecución y las punteadas son dependencias de compilación. Los cilindros representan bases de datos y las nubes, servicios externos.

Figura 4.9 — Componentes: visión general. Muestra los tres bloques del monorepo (PWA, `packages/shared` y API) y los servicios externos de los que dependen.

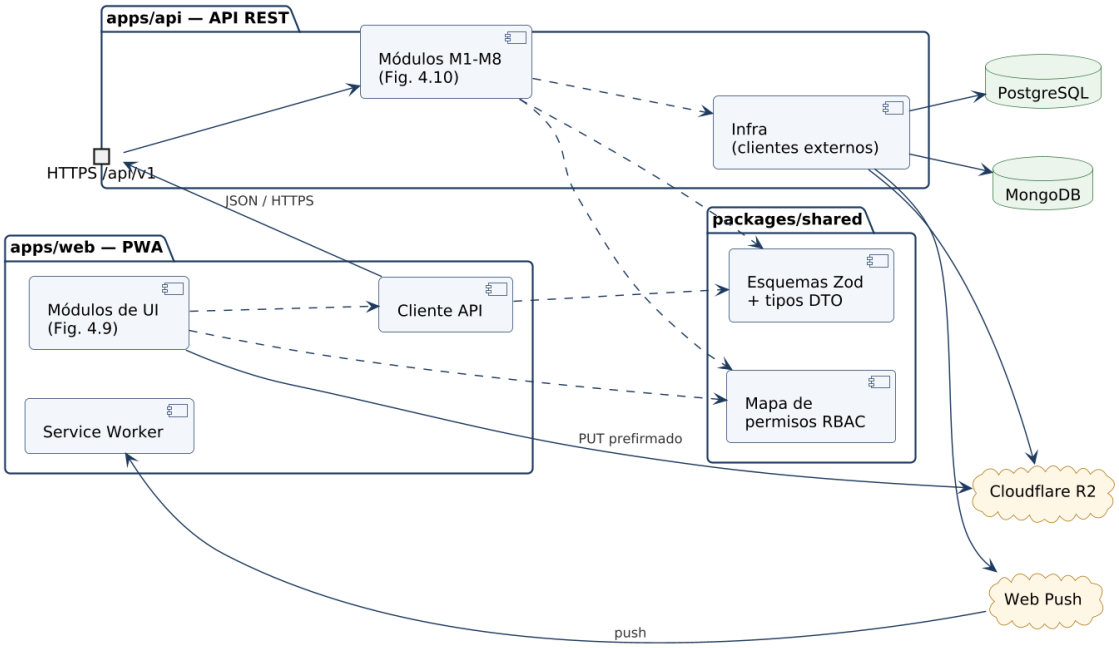


Figura 4.10 — Componentes de la PWA. El *shell* carga los ocho módulos de funcionalidad; todos llaman a la API a través de un único cliente.

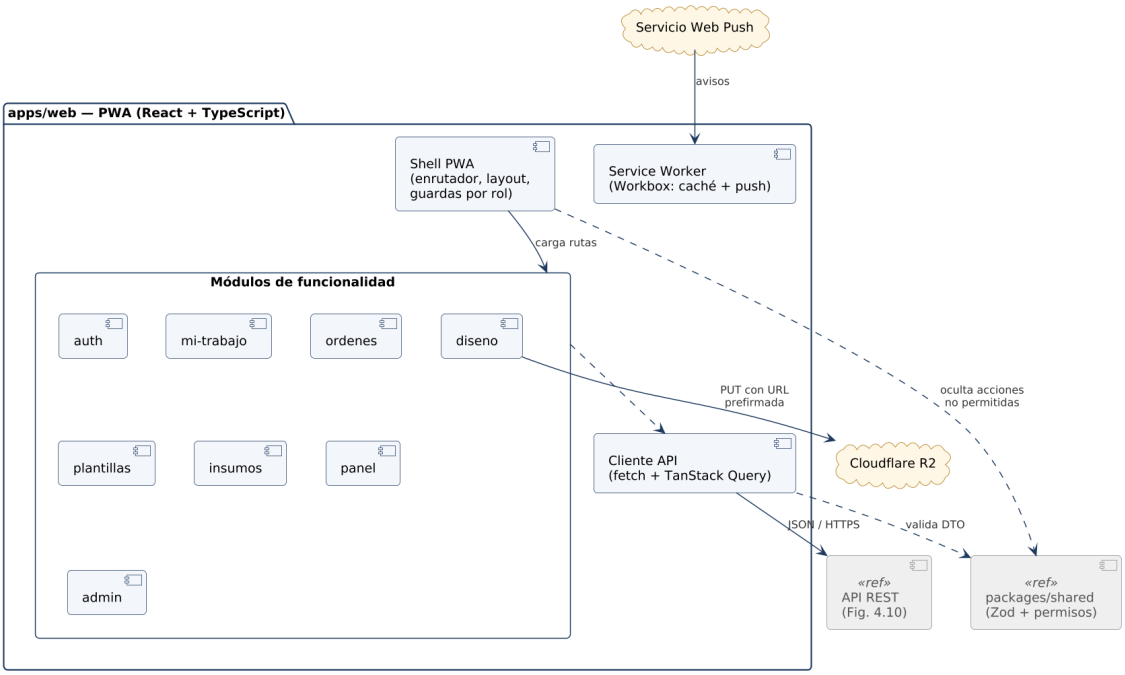
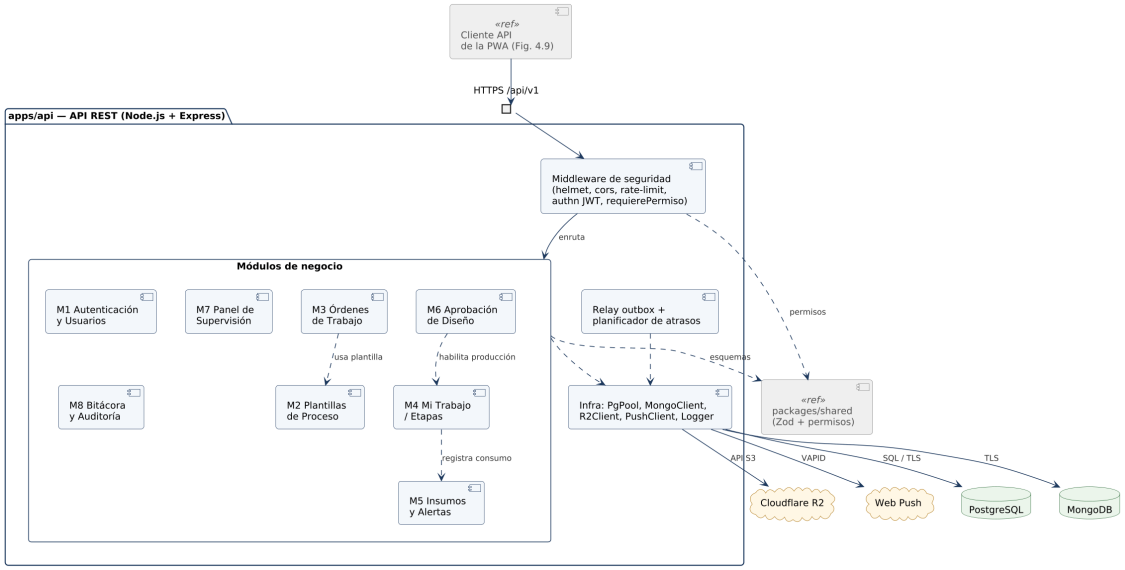


Figura 4.11 — Componentes de la API. Toda petición pasa por el middleware de seguridad antes de llegar a un módulo. Solo existen tres dependencias entre módulos (M3→M2, M4→M5 y M6→M4), y todos acceden a los servicios externos a través de *Infra*.



Organización:

- La PWA tiene un módulo de UI por cada módulo de negocio (`mi-trabajo` ↔ M4) y un **cliente API** único.
- `packages/shared` contiene los **esquemas Zod** y el **mapa de permisos**, que comparten el front-end y la API. Con ello se evita que validaciones y permisos diverjan entre capas.
- El componente *Infra* encapsula los clientes de PostgreSQL, MongoDB, R2 y Web Push. Así los módulos no dependen de proveedores concretos, y migrar de Neon a otro PostgreSQL solo cambia la configuración.

Figura 4.12 — Paquetes del monorepo. Monorepo `sgp-kotoduo` con `apps/web`, `apps/api`, `packages/shared`, `db` (migraciones `node-pg-migrate`, validadores Mongo y semillas) y `tests` (unit, integration, e2e, perf). La PWA y la API solo comparten código a través de `packages/shared`.

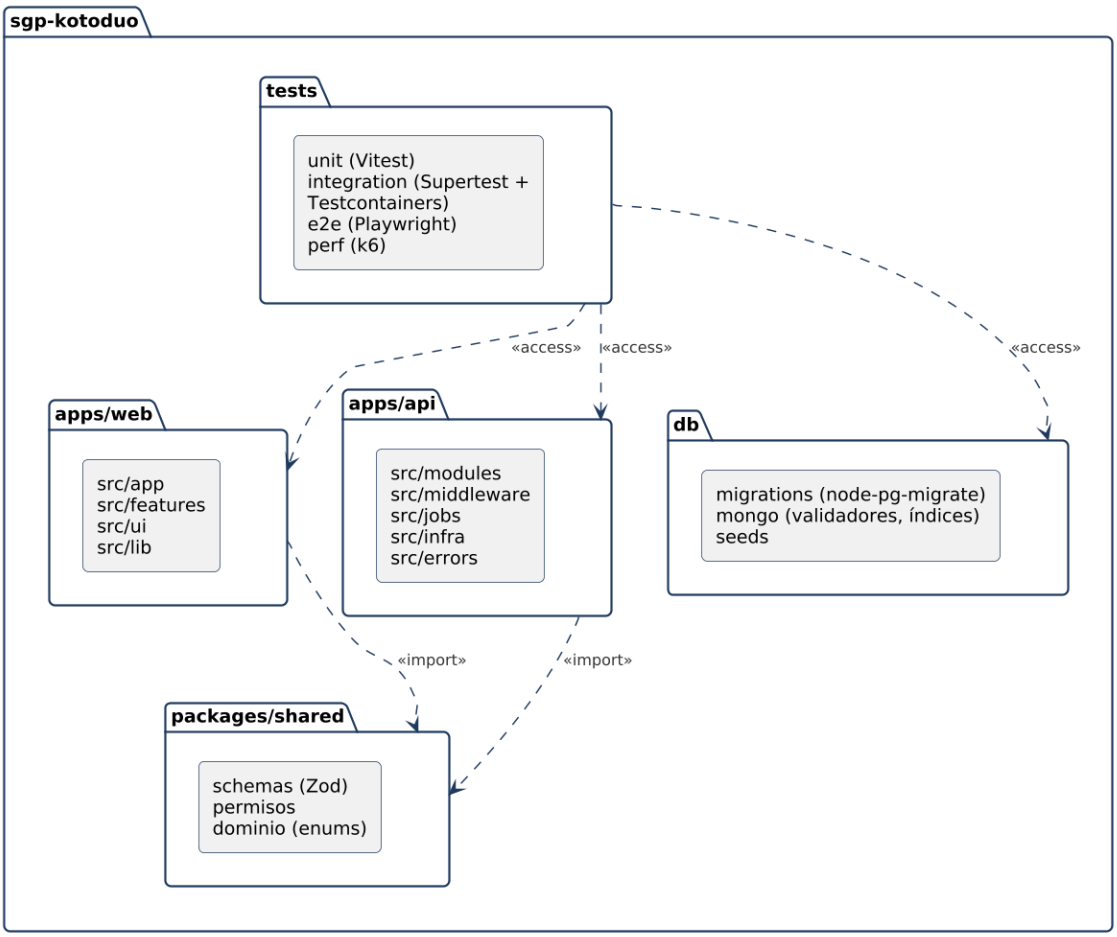
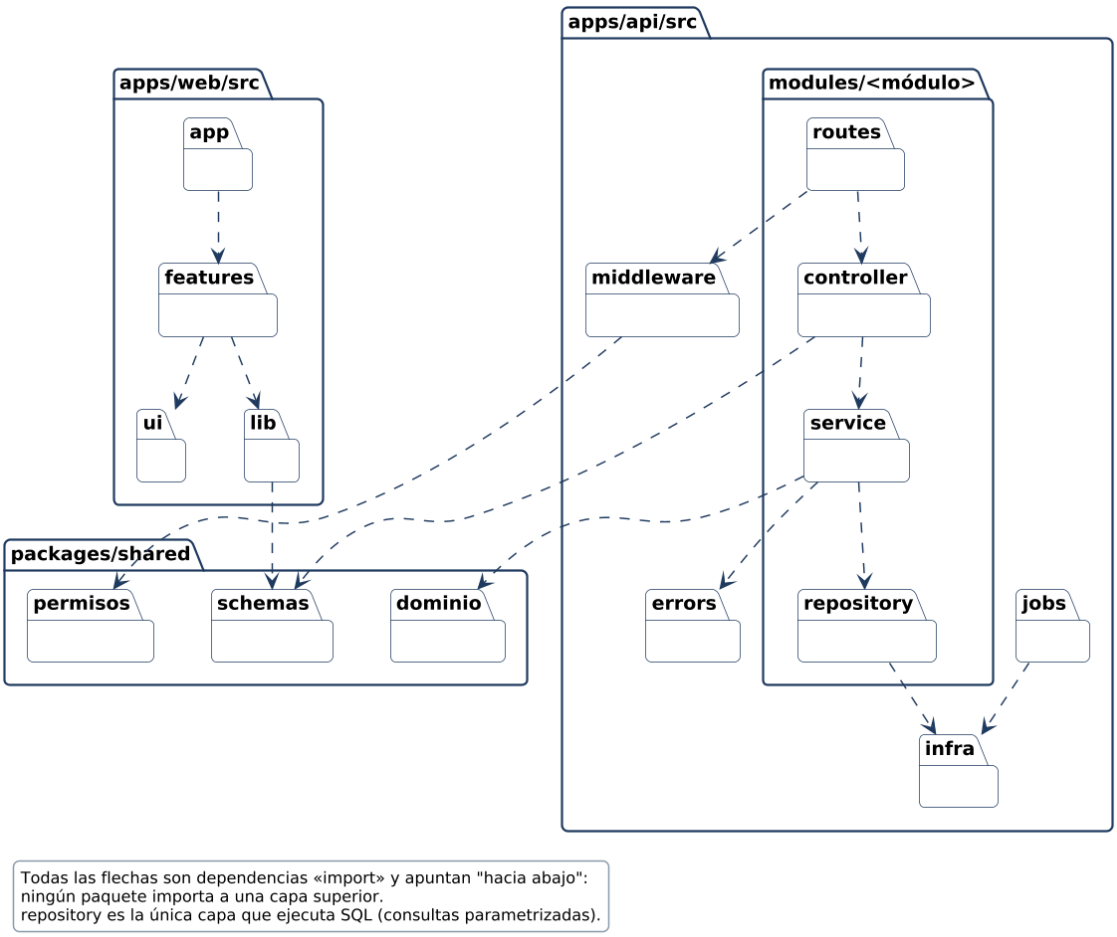


Figura 4.13 — Regla de dependencias por capas. Las relaciones `<<import>>` siguen una única dirección: *routes* → *controller* → *service* → *repository* → *infra* en la API y *app* → *features* → *ui/lib* en la PWA. Esto permite probar los servicios con repositorios simulados y garantiza que solo la capa *repository* ejecute SQL, siempre parametrizado (sección 7.6).



Convención	Regla
Ramas	`main` protegida; `feature/HU-xx-descripcion`; *merge* solo por PR aprobado y CI en verde
Versionado	SemVer en *tags* (`v1.0.0`); migraciones numeradas e inmutables
Estilo	ESLint (con `eslint-plugin-security`) + Prettier; TypeScript `strict: true`
Configuración	Variables de entorno validadas con Zod al inicio; sin secretos en el repositorio

4.4 Vista de procesos

La vista de procesos muestra primero el flujo de negocio (diagramas de actividades, Fig. 4.14–4.16) y luego la interacción técnica del escenario crítico CU-06, separada en su tramo síncrono y su tramo asíncrono (diagramas de secuencia, Fig. 4.17–4.18).

Simbología de los diagramas de actividades: equivalen a un BPMN de colaboración. Los carriles (Vendedor, Diseñador, Encargada de Producción, Operario y SGP-Kotoduo) corresponden a *lanes* y asignan la responsabilidad de cada actividad. Los rectángulos redondeados son actividades; los que tienen doble borde lateral son subprocesos detallados en otra figura. Los rombos son decisiones (*gateways* exclusivos) con guardas. `repeat … repeat while` modela bucles. Las barras `fork / end fork` modelan **concurrencia**.

Figura 4.14 — Flujo productivo: visión general. Recorre la orden de punta a punta:

registro y diseño, asignación, ejecución repetida de etapas, término y entrega.

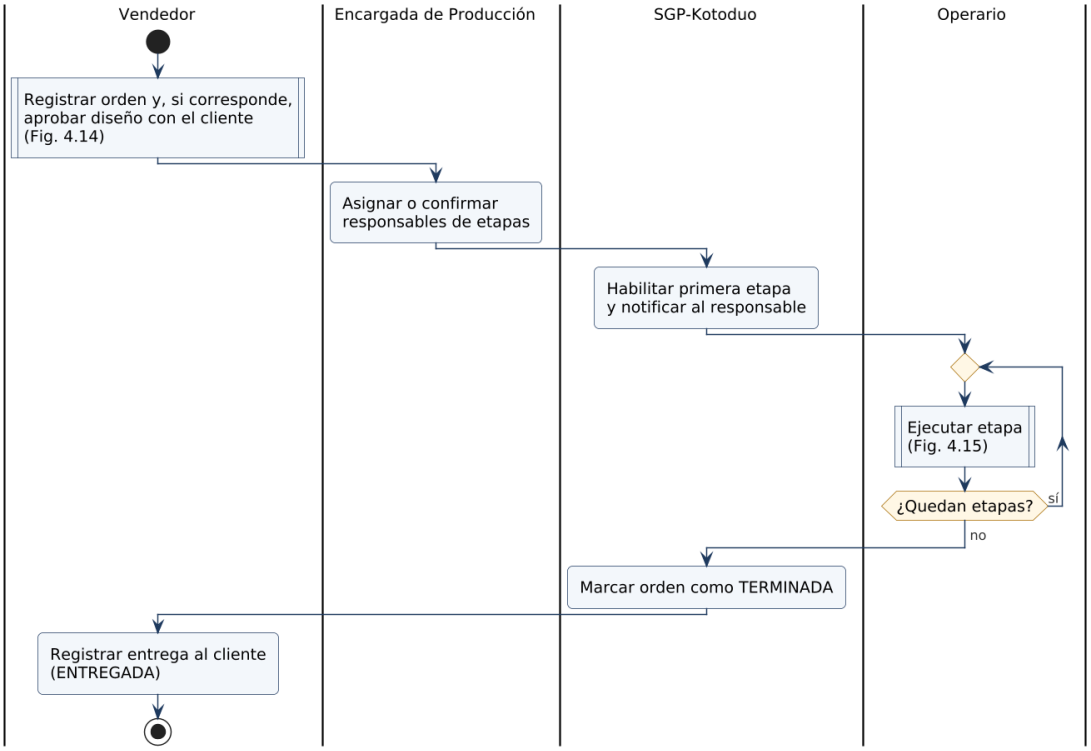


Figura 4.15 — Subproceso: registro de la orden y aprobación de diseño. El bucle entre Diseñador y Vendedor modela las rondas de aprobación del diseño con el cliente. Cada respuesta del cliente queda en la bitácora con la versión, el actor y el motivo (RF-12).

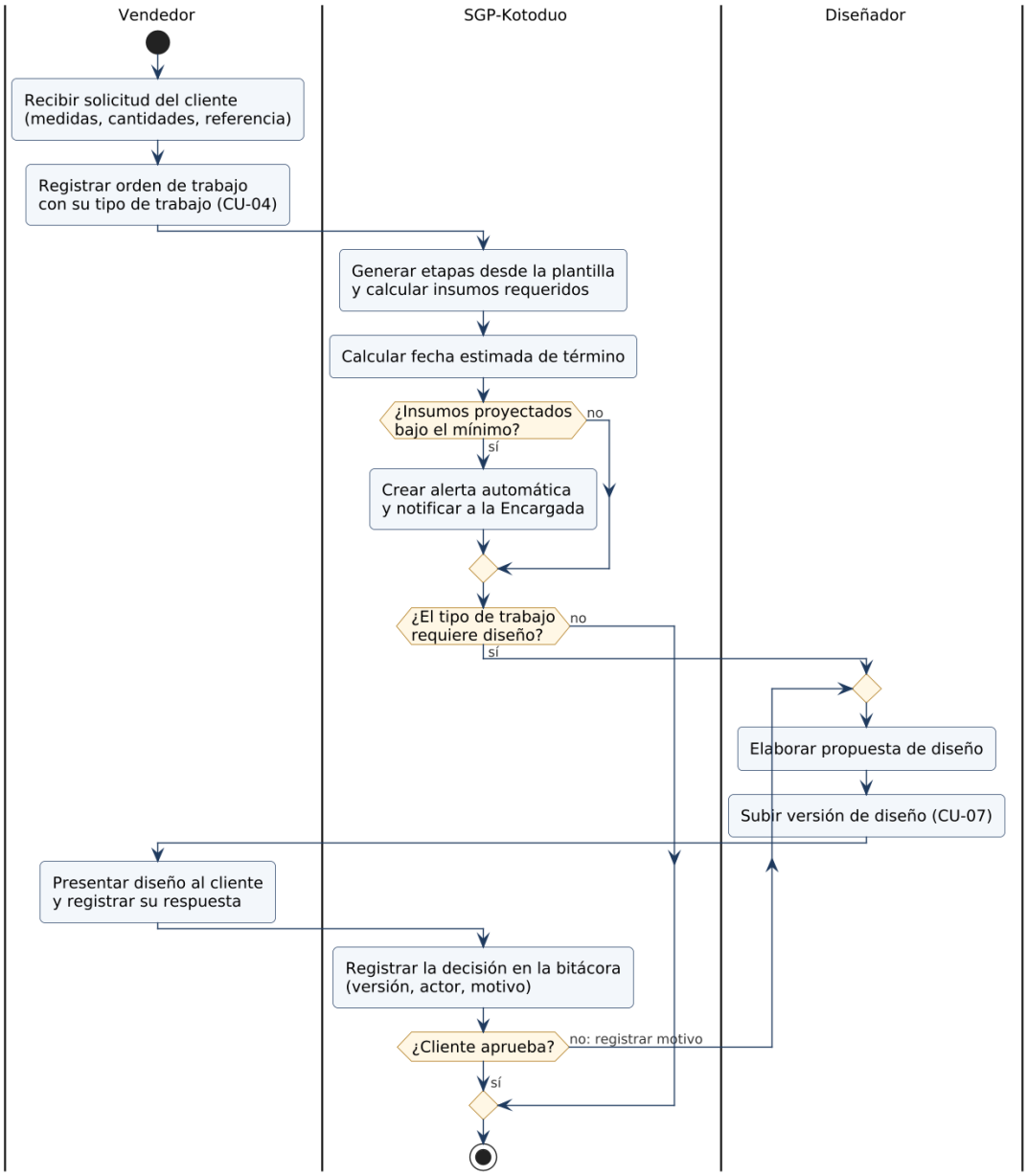
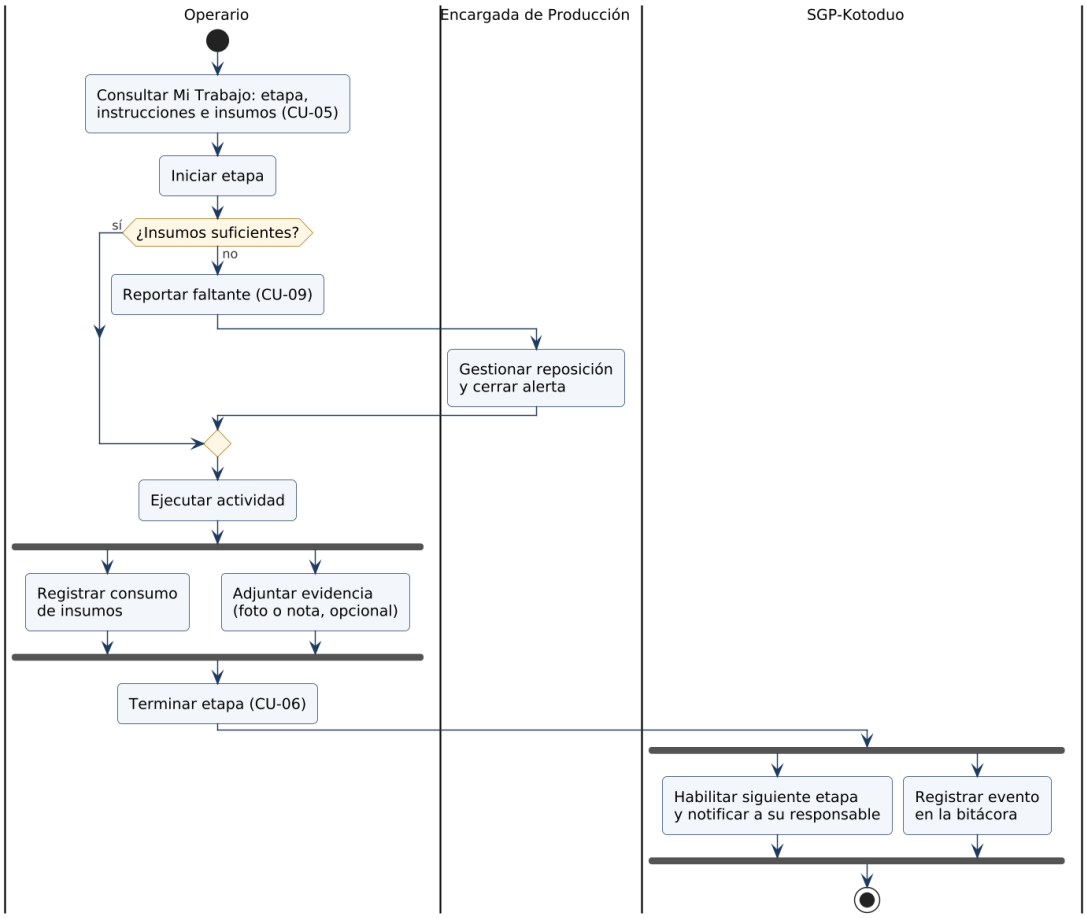


Figura 4.16 — Subproceso: ejecución de una etapa. Se repite por cada etapa de la orden. El consumo y la evidencia pueden registrarse en paralelo, y la habilitación de la siguiente etapa ocurre en paralelo con el registro en la bitácora.



Puntos de decisión relevantes:

1. **Insumos proyectados bajo el mínimo:** la alerta se emite al registrar la OT, antes de producir. Esto ataca la causa del caso "sin tinta el lunes".
2. **Diseño requerido:** la producción no se habilita sin aprobación, lo que formaliza la "aceptación del diseño" mencionada por el cliente.
3. **Insumo insuficiente durante la etapa:** el operario reporta y continúa cuando la Encargada resuelve, sin necesidad de buscarla físicamente.

Simbología de los diagramas de secuencia: las líneas de vida verticales representan participantes (actores, componentes y bases de datos). Las flechas continuas son mensajes síncronos y las punteadas, respuestas. Los fragmentos `alt` modelan alternativas excluyentes, `loop` modela repeticiones y `opt` un paso opcional.

Figura 4.17 — Secuencia CU-06 "Terminar etapa": tramo síncrono. La API autentica el JWT, verifica el permiso `etapas.avanzar_propias` y valida el cuerpo con Zod. Luego bloquea la fila de la etapa (`SELECT … FOR UPDATE`), verifica que el actor sea el responsable (404 si no lo es, para no revelar que la etapa existe) y que la versión coincida (409 si cambió: control de concurrencia optimista), y aplica todos los cambios en **una transacción ACID**: etapa terminada, consumos de insumo (que pueden disparar una alerta), siguiente etapa disponible y evento outbox.

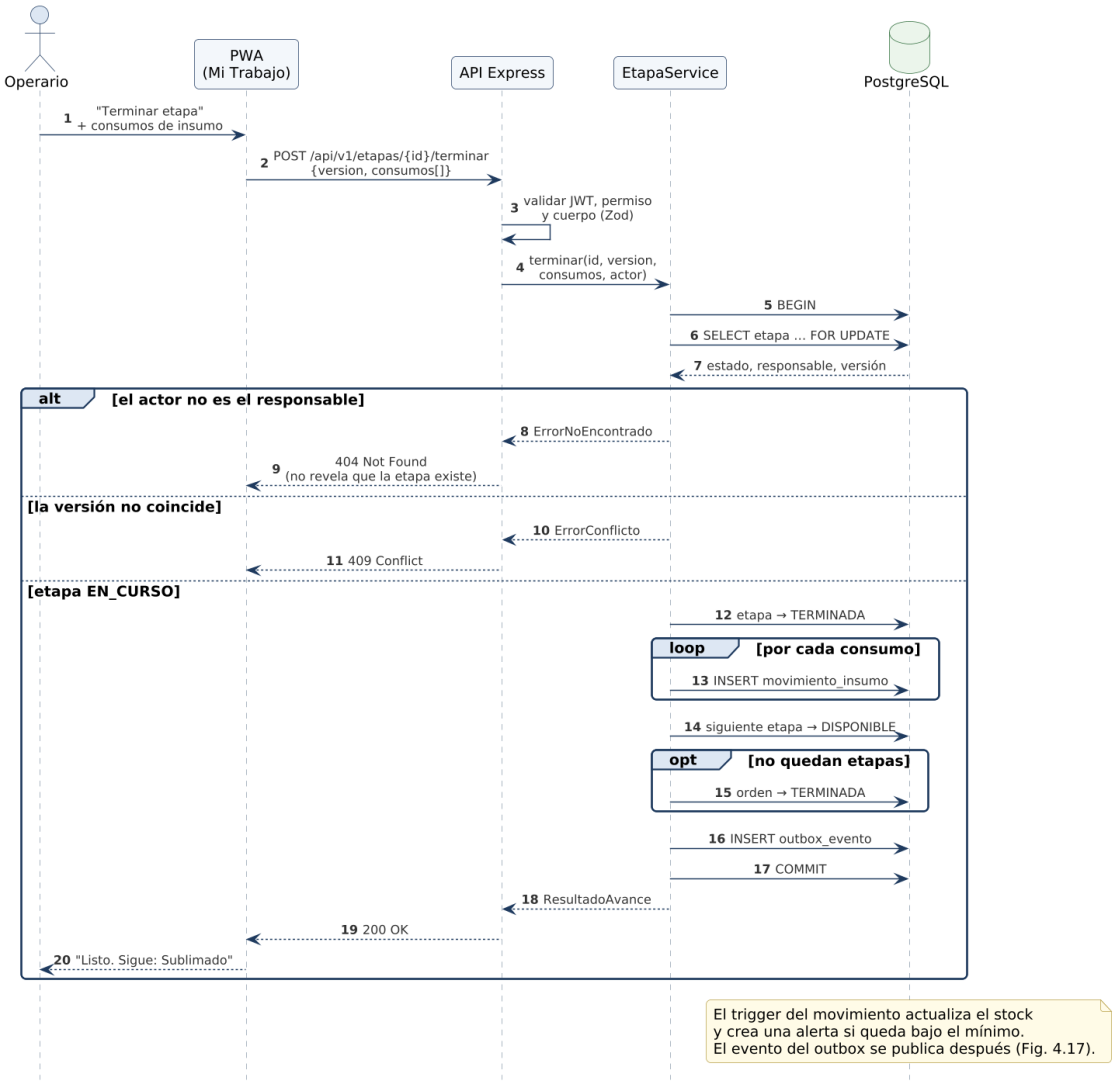
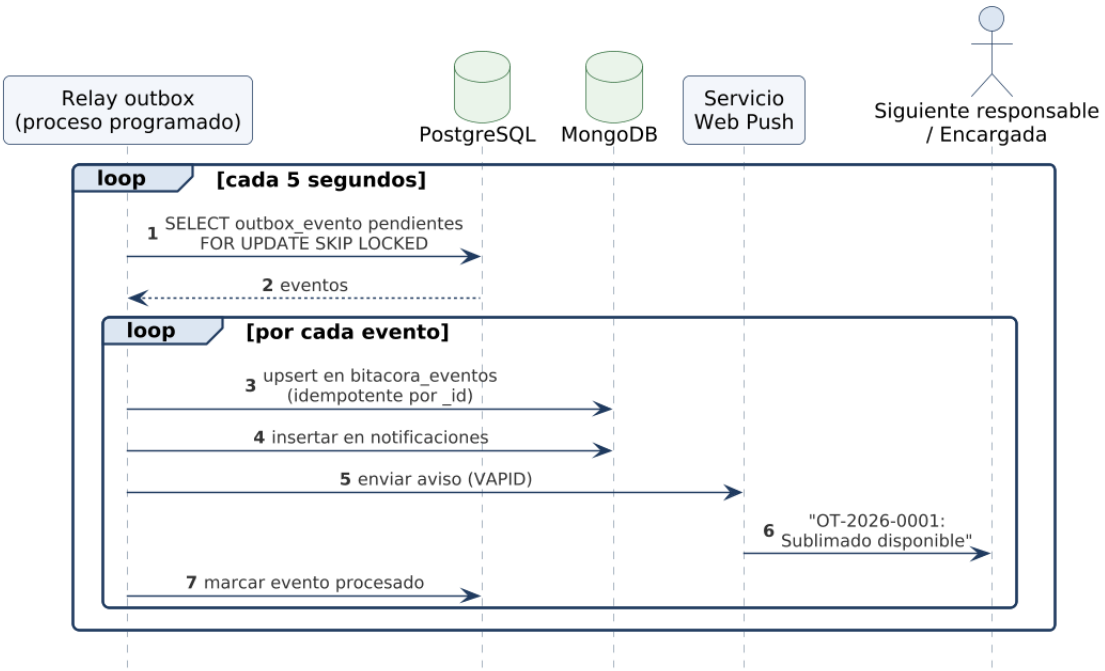


Figura 4.18 — Secuencia del relay outbox: tramo asíncrono. Cada 5 segundos el relay toma los eventos pendientes, los publica en MongoDB y envía el aviso push, sin alargar el tiempo de respuesta del operario. `FOR UPDATE SKIP LOCKED` y la escritura idempotente evitan duplicados si hay varias instancias.



Procesos de ejecución y concurrencia (Tabla 4.1):

Proceso / hilo	Dónde corre	Disparador	Concurrencia y sincronización
Atención de solicitudes HTTP	Contenedor API (event loop de Node.js)	Petición de la PWA	Asíncrono no bloqueante; pool PostgreSQL de 10 conexiones; transacciones con `FOR UPDATE` y versión optimista para evitar dobles avances
Relay outbox	Mismo contenedor, temporizador de 5 s	`outbox_evento` sin procesar	`FOR UPDATE SKIP LOCKED` permite varias instancias sin duplicar; escritura idempotente en Mongo (`_id = evt-n`)
Verificador de atrasos	Mismo contenedor, cada 15 min	Reloj	Recalcula `fecha_estimada_termino` y notifica OT en riesgo; operación idempotente
Triggers de base de datos	PostgreSQL	`UPDATE`/`INSERT`	Se ejecutan dentro de la transacción del llamador (atomicidad)
Service Worker	Dispositivo del usuario	Instalación, `fetch`, `push`	Caché *stale-while-revalidate* de lecturas; las escrituras requieren conexión (sin cola offline en la v1, para evitar conflictos)
Respaldo nocturno	GitHub Actions (cron 03:00 CLT)	Programación	`pg_dump` cifrado hacia R2; no bloquea la operación

4.5 Vista física o de despliegue

Simbología: los nodos (cajas 3D) son dispositivos o entornos de ejecución y las nubes, proveedores. Los artefactos son los entregables desplegados (`dist/`, la imagen del contenedor y los *buckets*). Las líneas son rutas de comunicación etiquetadas con protocolo y puerto.

Figura 4.19 — Despliegue: topología de ejecución. Muestra dónde corre cada pieza en producción y cómo se comunican.

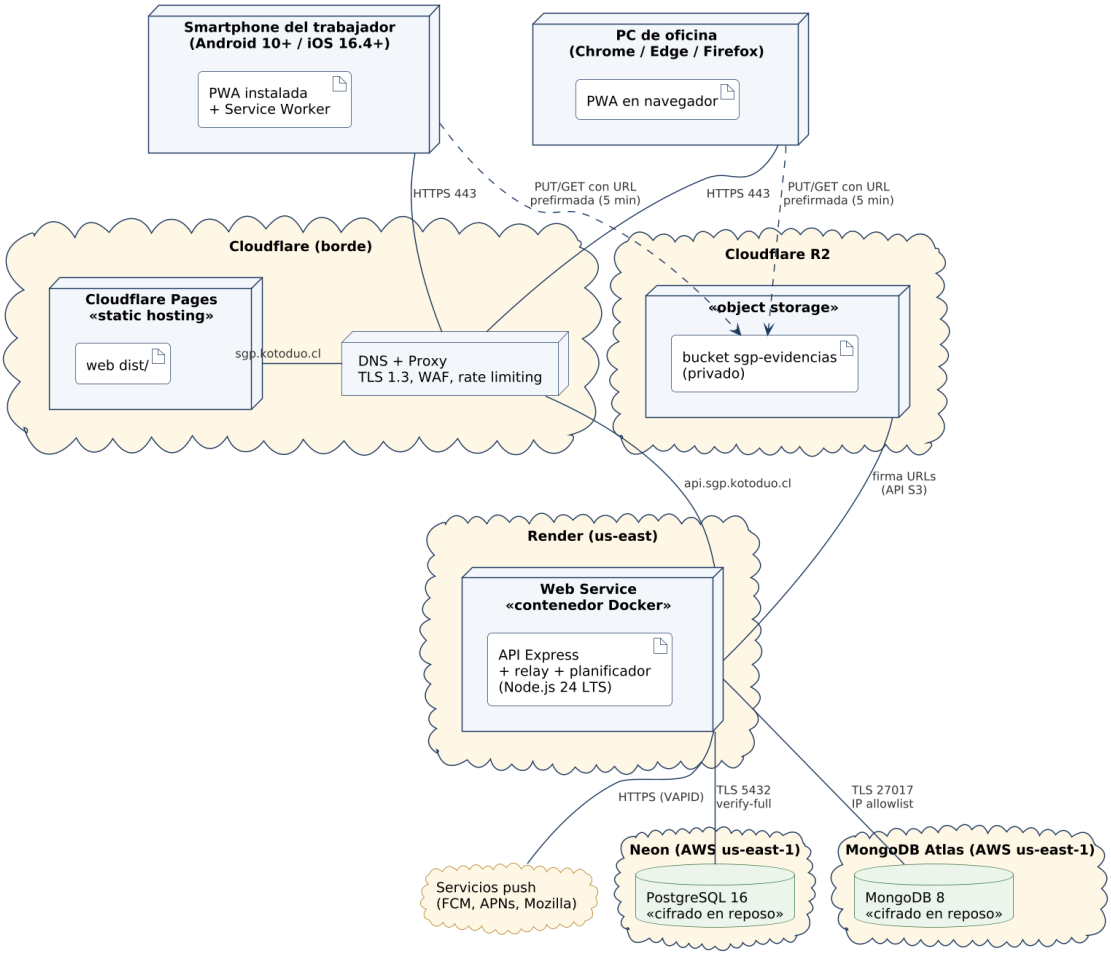
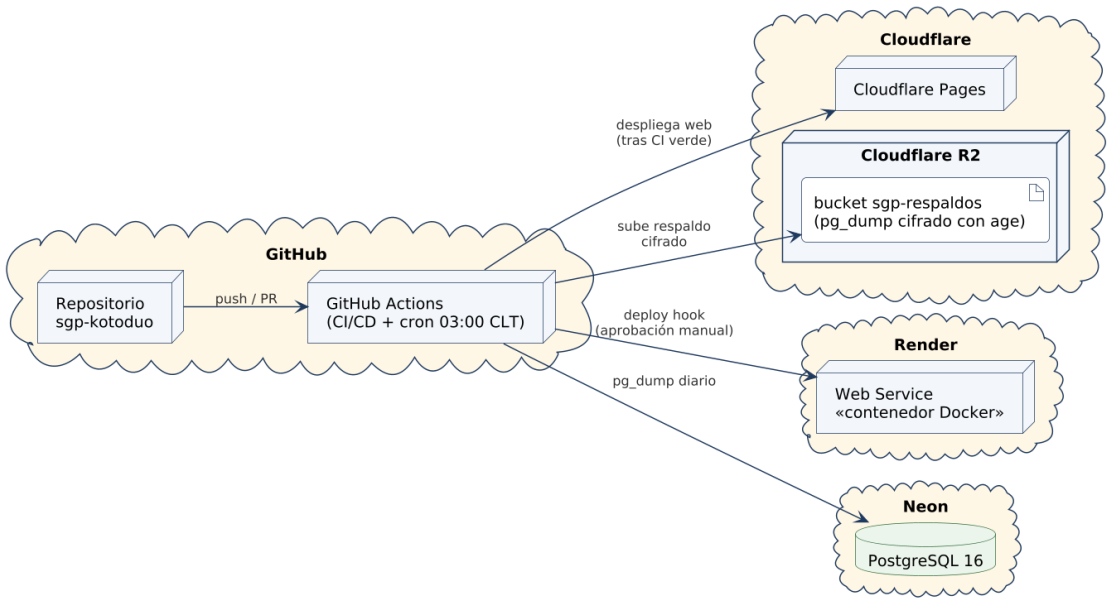


Figura 4.20 — Despliegue: integración continua, entrega y respaldos. Separa del diagrama anterior las rutas que no atiende a usuarios: GitHub Actions despliega la web y la API, y ejecuta el respaldo nocturno cifrado hacia R2.



Topología:

1. Los **smartphones** de los trabajadores ejecutan la PWA instalada y los **PC de oficina** la usan en el navegador.
2. Todo el tráfico entra por **Cloudflare**, que provee DNS, TLS 1.3, WAF y limitación de tasa en el borde. `sgp.kotoduo.cl` sirve la PWA desde **Cloudflare Pages** y `api.sgp.kotoduo.cl` enruta al **Web Service de Render**, un contenedor Docker con Node.js 24 LTS que ejecuta la API y los trabajos programados.
3. La API se conecta por TLS a **PostgreSQL 16 en Neon** (`sslmode=verify-full`) y a **MongoDB 8 en Atlas** (con lista de IP permitidas). Los tres están co-ubicados en la región este de EE. UU. para minimizar la latencia API↔BD.
4. Los archivos se suben y se consultan **directamente** entre el dispositivo (teléfono o PC) y **Cloudflare R2** mediante URL prefirmadas de 5 minutos. La API nunca recibe el binario, lo que reduce la carga y la superficie de ataque.
5. **GitHub Actions** ejecuta CI/CD y el respaldo nocturno.
6. Ambos dominios comparten el sitio `kotoduo.cl`, lo que habilita cookies `SameSite=Strict` (sección 7.2).

Ambientes: *local* (Docker Compose con PostgreSQL y MongoDB), *staging* (rama Neon `staging`, servicio Render de *preview*) y *producción*.

Aprovisionamiento: por el plazo acotado de construcción, los recursos de *staging* y producción se configuran desde los paneles de los proveedores. Cada uno se registra en un inventario del repositorio con su servicio, plan, región y variables de entorno. El ideal, planteado como mejora posterior, es declararlos como **infraestructura como código** para reproducir la topología de la Figura 4.19 con otro proveedor sin rediseñarla (sección 4.7, Tabla 4.3).

4.6 Vista de escenarios o casos de uso

Los catorce casos de uso se dividen en cuatro diagramas, uno por subsistema, para que cada interesado vea solo los que le conciernen.

Simbología: los actores (figuras) son roles externos al sistema y el rectángulo delimita la frontera de SGP-Kotoduo. Las elipses son casos de uso. Las líneas actor–caso indican participación. La flecha con triángulo indica generalización. `<<include>>` indica comportamiento obligatorio reutilizado y `<<extend>>`, comportamiento opcional bajo condición, que se inserta en un punto de extensión declarado en el caso base. Una etiqueta sobre una línea actor–caso restringe la participación de ese actor. Los actores de cada diagrama coinciden con la matriz de permisos (Tabla 7.1).

Figura 4.21 — Casos de uso: actores, acceso y administración. Define la jerarquía de actores: todos los roles heredan de “Usuario” (y por tanto pueden iniciar sesión) y el Administrador hereda las capacidades de la Encargada. La Encargada consulta la bitácora solo por OT; el Administrador la consulta completa.

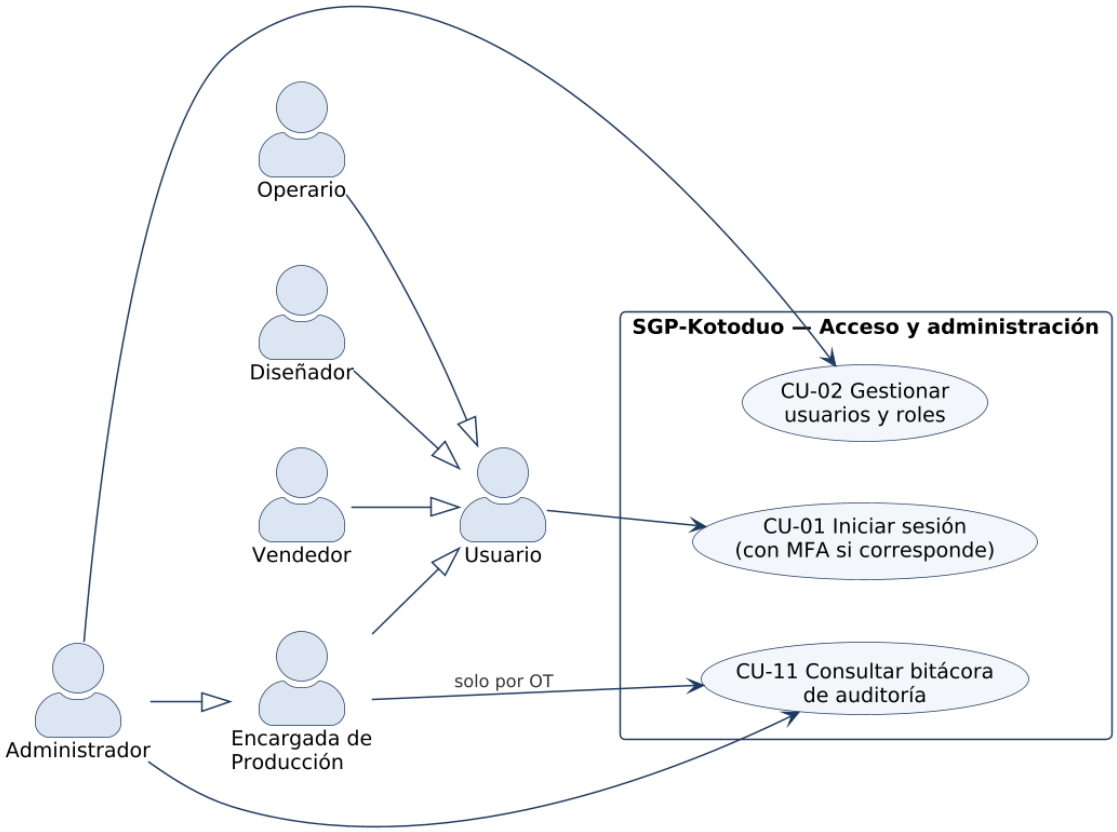


Figura 4.22 — Casos de uso: gestión de órdenes, diseño e insumos. Registrar una OT siempre genera sus etapas (`<<include>>`). En la aprobación de diseño, el Diseñador sube versiones y el Vendedor registra la respuesta del cliente; nunca la misma persona (R6).

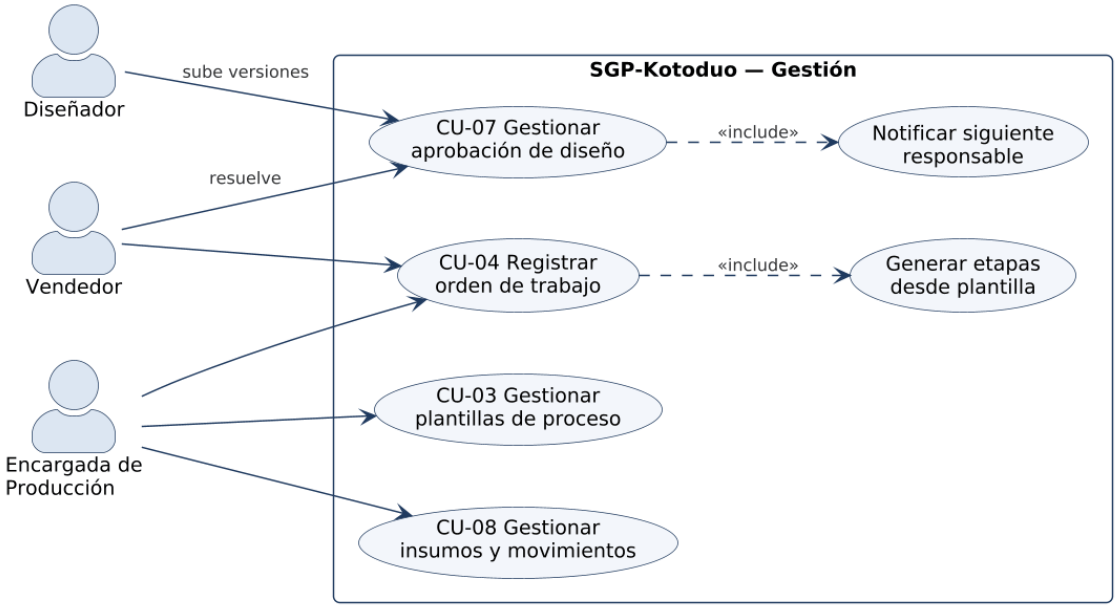


Figura 4.23 — Casos de uso: ejecución en el taller. Contiene el escenario crítico CU-06, que declara dos puntos de extensión: *Etapa en curso* y *Registrar consumos*. Registrar evidencia extiende el primero y reportar un faltante (CU-09) extiende el segundo (`<<extend>>`). Todo avance notifica al siguiente responsable (`<<include>>`).

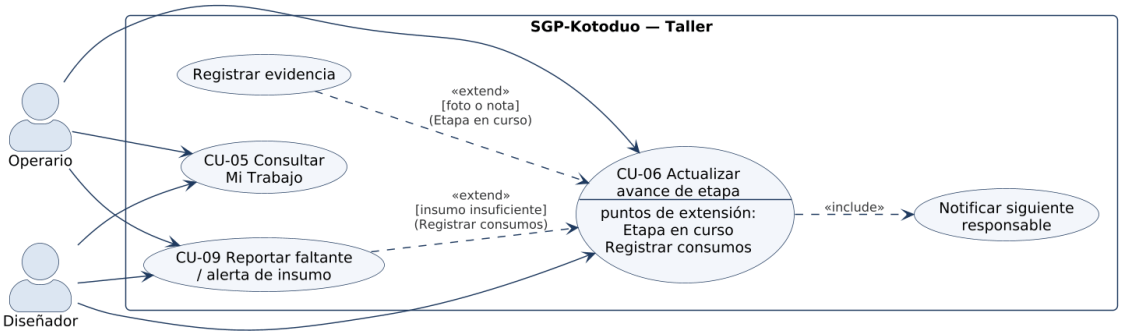
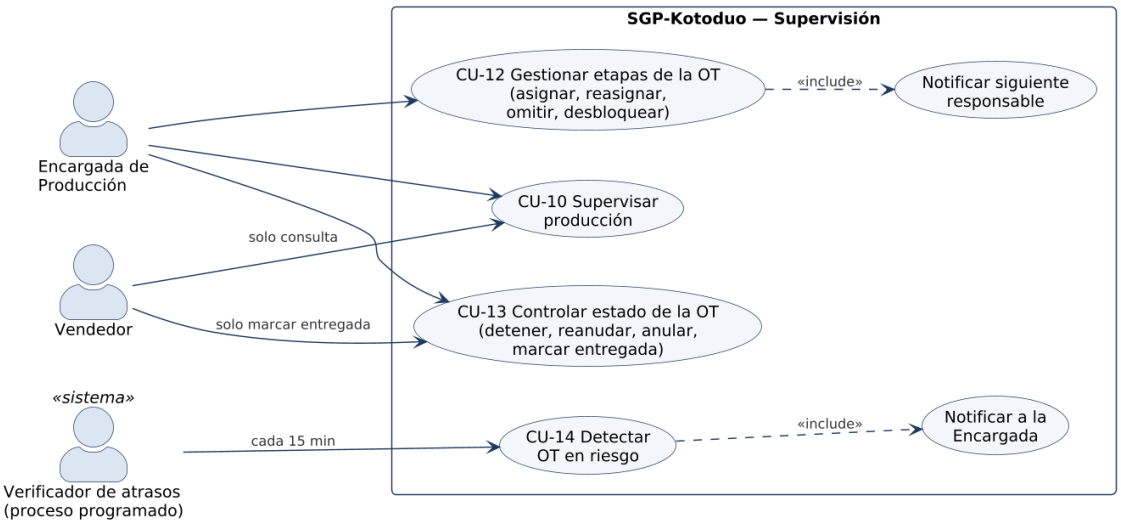


Figura 4.24 — Casos de uso: supervisión y control de la producción. Reúne las acciones de la Encargada que mueven las máquinas de estados de la etapa y de la OT (Fig. 4.7 y 4.8): gestionar etapas (CU-12) y controlar el estado de la OT (CU-13). El Vendedor consulta el panel y marca la OT como entregada, sin otras acciones. El actor `<<sistema>>` "Verificador de atrasos" es el proceso programado de la vista de procesos, que cada 15 minutos detecta las OT en riesgo (CU-14) y avisa a la Encargada.



CU	Nombre	Actores (el Administrador hereda los de la Encargada)	RF	HU	Figura
CU-01	Iniciar sesión (con MFA si corresponde)	Todos	RF-01	HU-11	4.21
CU-02	Gestionar usuarios y roles	Administrador	RF-01	HU-11	4.21
CU-03	Gestionar plantillas de proceso	Encargada de Producción	RF-02	HU-07	4.22
CU-04	Registrar orden de trabajo	Vendedor, Encargada de Producción	RF-03, RF-04	HU-05	4.22
CU-05	Consultar Mi Trabajo	Operario, Diseñador	RF-05	HU-01	4.23

CU	Nombre	Actores (el Administrador hereda los de la Encargada)	RF	HU	Figura
CU-06	Actualizar avance de etapa	Operario, Diseñador	RF-06, RF-07	HU-02, HU-04	4.23
CU-07	Gestionar aprobación de diseño	Diseñador (sube), Vendedor (resuelve)	RF-08	HU-06	4.22
CU-08	Gestionar insumos y movimientos	Encargada de Producción	RF-09	HU-09	4.22
CU-09	Reportar faltante / alerta de insumo	Operario, Diseñador	RF-10	HU-03, HU-09	4.23
CU-10	Supervisar producción	Encargada de Producción; Vendedor (solo consulta)	RF-11	HU-10	4.24
CU-11	Consultar bitácora de auditoría	Administrador; Encargada (solo por OT)	RF-12	HU-12	4.21
CU-12	Gestionar etapas de la OT (asignar, reasignar, omitir, desbloquear)	Encargada de Producción	RF-04, RF-06	HU-08	4.24
CU-13	Controlar estado de la OT (detener, reanudar, anular, marcar entregada)	Encargada de Producción; Vendedor (solo marcar entregada)	RF-06, RF-11	HU-10	4.24
CU-14	Detectar OT en riesgo	Verificador de atrasos (`<<sistema>>`)	RF-11	HU-10	4.24

Especificación del caso de uso crítico CU-06 — Actualizar avance de etapa

Campo	Descripción
Actor	Operario (o Diseñador en etapas de diseño)
Precondiciones	Sesión activa; la etapa está asignada al actor y en estado DISPONIBLE o EN_CURSO; la OT no está DETENIDA
Flujo básico	1. El actor abre *Mi Trabajo* y selecciona la OT. 2. El sistema muestra la etapa, las instrucciones, los insumos requeridos y la siguiente área. 3. El actor pulsa **Iniciar**; el sistema cambia la etapa a EN_CURSO. 4. Al finalizar, el actor pulsa **Terminar** e ingresa los consumos reales. 5. El sistema registra la etapa como TERMINADA, descuenta el stock, habilita la siguiente etapa y notifica a su responsable. 6. El sistema muestra “Sigue: <etapa> en <área>”.
Puntos de extensión	*Etapa en curso* (entre los pasos 3 y 4); *Registrar consumos* (paso 4)
Flujos alternativos	3a. Problema en la ejecución → **Reportar problema** con motivo → BLOQUEADA; se notifica a la Encargada y la OT queda DETENIDA hasta que la Encargada desbloquee la etapa (CU-12) y reanude la OT (CU-13). 4a. Insumo insuficiente → CU-09 extiende este caso en el punto *Registrar consumos*. 4b. Adjuntar foto o nota → “Registrar evidencia” extiende este caso en el punto *Etapa en curso*. 5a. Última etapa → la OT pasa a TERMINADA.

Campo	Descripción
Excepciones	E1. Otro usuario actualizó la etapa → 409 "La etapa cambió; recarga". E2. La etapa no pertenece al actor → 404, sin revelar que existe (AM-05). E3. Sin conexión → la PWA informa y conserva el formulario.
Postcondiciones	El estado de la etapa y de la OT queda consistente; el evento queda en la bitácora; el siguiente responsable está notificado
Reglas	Solo transiciones de la Fig. 4.7; el consumo no puede dejar stock negativo

Especificación del caso de uso crítico CU-04 — Registrar orden de trabajo

Campo	Descripción
Actor	Vendedor (o Encargada de Producción)
Precondiciones	Sesión activa con el permiso `ordenes.crear`; existe un tipo de trabajo activo con plantilla
Flujo básico	1. Selecciona el tipo de trabajo. 2. Ingresa título, referencia del cliente (solo institución), cantidad, especificaciones (tallas, medidas, colores), prioridad y fecha de compromiso. 3. El sistema valida, genera el código `OT-AAAA-NNNN`, crea las etapas desde la plantilla y calcula los requerimientos de insumo y la fecha estimada. 4. Si un insumo proyectado queda bajo el mínimo, crea una alerta. 5. Muestra el resumen con las etapas generadas.
Alternativos	3a. Fecha estimada > compromiso → advertencia "OT en riesgo" antes de confirmar. 3b. La Encargada ajusta etapas y responsables después de registrar la OT (CU-12, HU-08).
Postcondiciones	OT en estado REGISTRADA (o EN_DISENO si requiere diseño); etapas en PENDIENTE y la primera en DISPONIBLE

4.7 Tecnologías y herramientas de infraestructura

La Tabla 4.2 resume la tecnología por capa. La matriz completa, con el uso de cada herramienta en la solución, su justificación técnica y la necesidad futura considerada, está en el **Anexo I (Tabla I.1)**.

Tabla 4.2 — Tecnologías por capa

Capa	Tecnologías	Justificación principal
Front-end y modalidad móvil	React 19 + TypeScript + Vite; **PWA** (Workbox); TanStack Query	Cumple la expectativa de "app para el celular" sin tiendas: una sola base de código para Android, iOS y PC, instalable y con notificaciones
Back-end	Node.js 24 LTS + Express 5 + TypeScript; Zod compartido (`packages/shared`); `pg` + `node-pg-migrate`	Mismo lenguaje en el front-end y la API para un equipo de dos personas; SQL explícito y parametrizado

Capa	Tecnologías	Justificación principal
Datos	**PostgreSQL 16** (Neon); **MongoDB 8** (Atlas); Cloudflare R2	Dominio relacional que exige consistencia ACID; eventos y evidencias de estructura variable; archivos sin cargos por egreso
Seguridad	Argon2id, JWT EdDSA (`jose`) y TOTP (`otplib`); `helmet`, `cors`, `express-rate-limit`	Algoritmos recomendados por OWASP y NIST; mitigación de XSS, CSRF y fuerza bruta (sección 7.4)
Notificaciones	Web Push (VAPID)	Estándar W3C, sin depender de WhatsApp ni de SMS; funciona con la app cerrada
Hosting y red	Cloudflare Pages, DNS, proxy y WAF; Render (contenedor Docker)	TLS, CDN y WAF sin costo; plan básico de Render para evitar la suspensión por inactividad
Entrega y calidad	GitHub Actions + Environments; Docker y Docker Compose; `pino`, Sentry y UptimeRobot	CI con pruebas, SAST y aprobación manual a producción; ambiente local idéntico a producción; observabilidad gratuita
Gestión y documentación	GitHub Projects; PlantUML	Backlog, HU, PR y CI en una sola herramienta; diagramas versionables
Infraestructura como código (ideal)	OpenTofu + Trivy	Fuera del alcance comprometido; primera mejora tras el piloto (ver más abajo)

Costo mensual estimado de operación (valores referenciales a validar al contratar, RNF-08): Cloudflare Pages, R2 y WAF US$ 0; Render Starter ≈ US$ 7; Neon Free US$ 0 (Launch ≈ US$ 19 si se superan 0,5 GB); Atlas M0 US$ 0 (M10 si se superan 512 MB); GitHub, Sentry y UptimeRobot US$ 0. **Total inicial ≈ US$ 7–10/mes**, compatible con la ausencia de presupuesto definido. El crecimiento previsto (~1.500 OT/año, ~12.000 etapas/año) cabe en las capas iniciales por al menos dos años.

Infraestructura como código y portabilidad entre proveedores (práctica ideal)

Kotoduo no tiene un presupuesto definido (RNF-08), y los precios y las capas gratuitas de los servicios cloud cambian con el tiempo. Lo ideal es que la infraestructura no quede amarrada a un proveedor y se declare en archivos versionados con **OpenTofu**, la bifurcación de código abierto de Terraform que mantiene la Linux Foundation (licencia MPL 2.0). Así, si un proveedor sube sus precios, elimina su capa gratuita o Kotoduo obtiene créditos en otra nube, bastaría con cambiar el módulo afectado y volver a desplegar la misma arquitectura.

Alcance dentro del plazo. Como la construcción dura un mes (sección 3.1), la IaC **no forma parte del alcance comprometido**. Se plantea como práctica ideal y como primera mejora después del piloto. Dentro del plazo solo se asegura lo que la hace posible sin costo adicional: el principio 2 (interfaces estándar), que ya forma parte del diseño, y el inventario de recursos de la sección 4.5, que sirve de punto de partida para escribir los módulos.

Principios de diseño de la IaC, para cuando se adopte:

1. **Módulos por rol, no por proveedor.** La configuración se divide en seis módulos:

borde y DNS, front-end, cómputo de la API, base de datos estructurada, base de datos no estructurada y almacenamiento de archivos. Cada módulo entrega las mismas salidas sin importar quién lo implemente: URL de la API, cadenas de conexión y nombre del *bucket*. Cambiar de proveedor significa reemplazar la implementación de un módulo; los demás módulos y la aplicación no cambian.

2. **La aplicación depende de interfaces estándar.** La API solo conoce una imagen Docker, el protocolo de PostgreSQL, el protocolo de MongoDB, la API de S3 y su configuración por variables de entorno (factor III de *The Twelve-Factor App*). Esta es la condición que hace útil la IaC: sin ella, cambiar de proveedor obligaría a reescribir código.
3. **Mismos módulos para cada ambiente.** *Staging* y producción se crean desde los mismos módulos, con distintos tamaños y nombres. *Staging* reproduce producción fielmente, lo que da validez a las pruebas de integración y rendimiento (8.4 y 8.5).
4. **Cambios revisados como el código.** Cada PR que modifica `infra/` ejecuta en CI la validación de sintaxis, el análisis de configuración insegura con Trivy y un *plan*, que muestra qué recursos se crearán, modificarán o destruirán. Aplicar ese plan en producción requiere la misma aprobación manual de GitHub Environments que el despliegue de la API.
5. **Estado remoto y cifrado.** El estado de OpenTofu registra los recursos creados y puede contener secretos. Por eso se guarda en un *bucket* privado compatible con S3 y se cifra del lado del cliente con el cifrado de estado nativo de OpenTofu. Ese *bucket* es el único recurso que se crea una vez de forma manual. Los tokens de los proveedores se guardan en los secretos de GitHub Environments, nunca en el repositorio (sección 7.5).

Tabla 4.3 — Módulos de infraestructura y proveedores intercambiables

Módulo	Proveedor inicial	Alternativa en AWS	Alternativa en Google Cloud	Proveedor de OpenTofu inicial
Borde y DNS	Cloudflare DNS, proxy y WAF	Route 53 + CloudFront + AWS WAF	Cloud DNS + Cloud Load Balancing + Cloud Armor	`cloudflare/cloudflare` (oficial)
Front-end (PWA estática)	Cloudflare Pages	S3 + CloudFront o Amplify Hosting	Firebase Hosting	`cloudflare/cloudflare` (oficial)
Cómputo de la API (contenedor)	Render Web Service	ECS Fargate o Lightsail Containers	Cloud Run	`render-oss/render` (oficial)
BD estructurada (PostgreSQL 16)	Neon	RDS for PostgreSQL	Cloud SQL for PostgreSQL	`kislerdm/neon` (comunitario, patrocinado por Neon)
BD no estructurada (MongoDB 8)	MongoDB Atlas	MongoDB Atlas desplegado en AWS	MongoDB Atlas desplegado en Google Cloud	`mongodb/mongodbatlas` (oficial)
Archivos (API S3)	Cloudflare R2	Amazon S3	Cloud Storage (API XML interoperable con S3)	`cloudflare/cloudflare` (oficial)

La base de datos no estructurada se mantiene en Atlas con cualquier nube, porque Atlas puede desplegarse sobre AWS, Google Cloud o Azure. Amazon DocumentDB se descartó como reemplazo porque es compatible con la API de MongoDB solo en parte, y el diseño de la sección 6.4 depende de los índices TTL y del *aggregation pipeline*.

Límites de la portabilidad. La IaC hace reproducible la infraestructura, pero no traslada los datos ni iguala las diferencias entre servicios. Una migración real requiere además tres pasos. Primero, trasladar los datos con las herramientas nativas de respaldo de cada motor y copiar los objetos del *bucket*. Segundo, actualizar los registros DNS. Tercero, adaptar las funciones sin equivalente directo, como el *branching* de Neon o las reglas del WAF. Estos pasos se documentan como procedimiento de migración en el repositorio y se ensayan junto con la restauración de respaldos (AM-15). Además, el proveedor de Neon es comunitario y el de Render es reciente: si alguno dejara de mantenerse, se reemplaza solo su módulo.

5 Diseño de interfaces de usuario

5.1 Perfiles y flujos principales de usuario

Perfil	Contexto de uso	Necesidad principal	Flujo / interacción clave	Pantallas
Operario	Celular propio en el taller, manos ocupadas, conectividad 4G/Wi-Fi variable, alfabetización digital heterogénea	Saber qué hacer ahora y qué sigue, sin preguntar (HU-01, HU-02)	Login → Mi Trabajo → tarjeta de OT → **Iniciar** → **Terminar** (+ consumo) → mensaje "Sigue: …"; alternativo: **Reportar faltante**	P-01, P-02, P-03
Diseñador	PC en oficina y celular	Subir versiones de diseño y ver sus etapas (HU-06)	Mi Trabajo → OT → Diseño → **Subir versión**	P-02, P-03, P-05
Vendedor	PC de oficina	Registrar la OT completa una sola vez y registrar la respuesta del cliente al diseño (HU-05, HU-06)	Órdenes → **Nueva orden** → tipo de trabajo → datos → confirmar; Diseño → **Aprobar / Rechazar** con motivo	P-04, P-05
Encargada de Producción	PC y celular; supervisa varias áreas	Ver el estado global, actuar sobre atrasos e insumos, estandarizar procesos (HU-07–HU-10)	Panel → filtro "Atrasadas" → OT → reasignar; Insumos → alerta → registrar ingreso → cerrar alerta; Plantillas → editar etapas	P-06, P-07, P-08
Administrador	PC de oficina	Controlar accesos y auditar (HU-11, HU-12)	Admin → Usuarios → crear/asignar rol/restablecer clave; Bitácora → filtrar por OT	P-09, P-10

Principios aplicados a todos los perfiles: **tarea principal en primer plano** (la pantalla de inicio depende del rol), **lenguaje del taller** ("Terminar etapa", no "Actualizar estado") y **acciones de un toque** con confirmación solo para acciones irreversibles.

5.2 Navegación y organización de contenidos

El mapa de navegación es un árbol jerárquico de rutas. Cada nodo es una ruta de la PWA y sus hijos son pantallas o acciones accesibles desde ella. Se presenta en tres figuras: el nivel superior y dos acercamientos por grupo de perfiles.

Figura 5.1 — Mapa de navegación: nivel superior. Tras el inicio de sesión, cada rol entra a su propia pantalla de inicio.

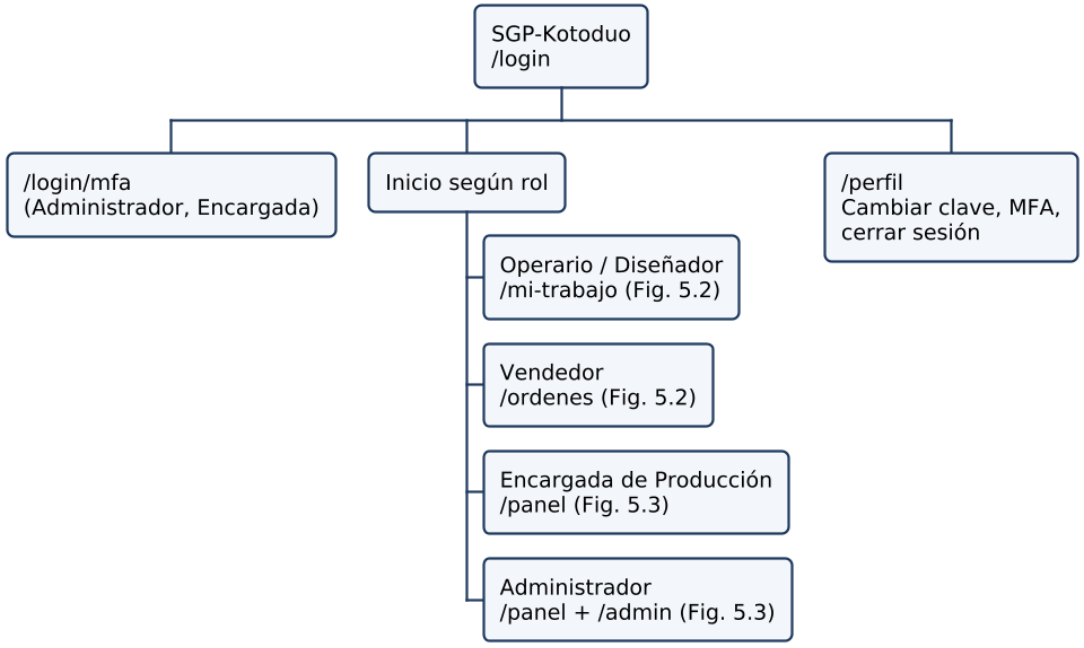


Figura 5.2 — Mapa de navegación: perfiles de taller y ventas

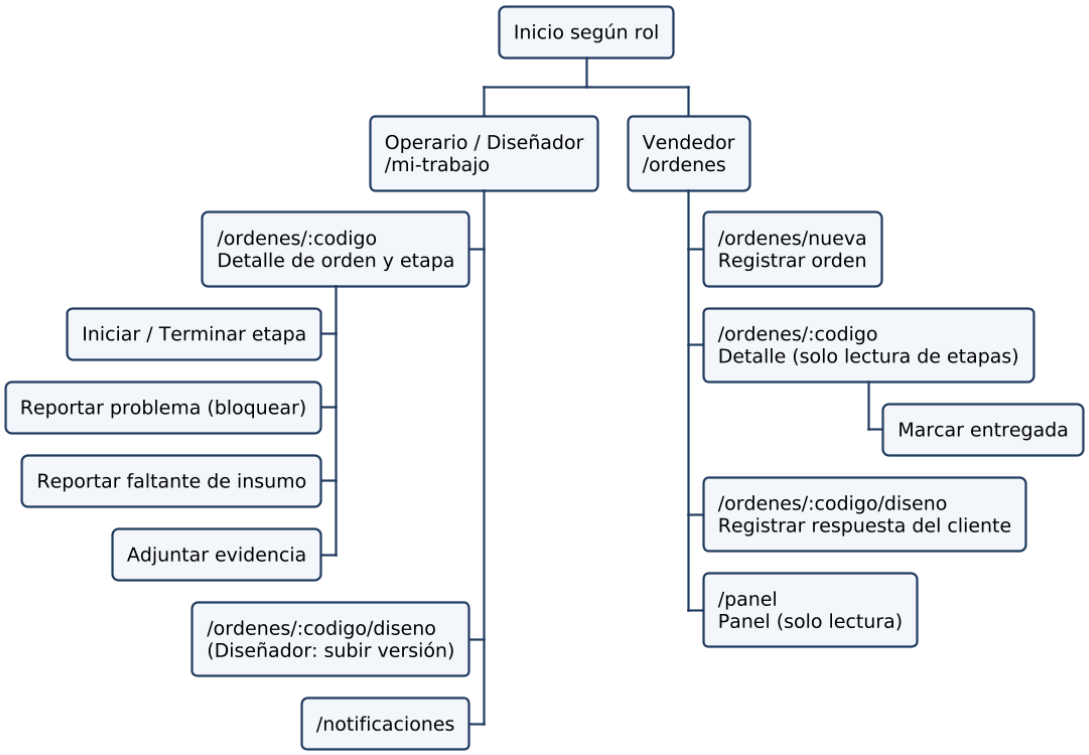
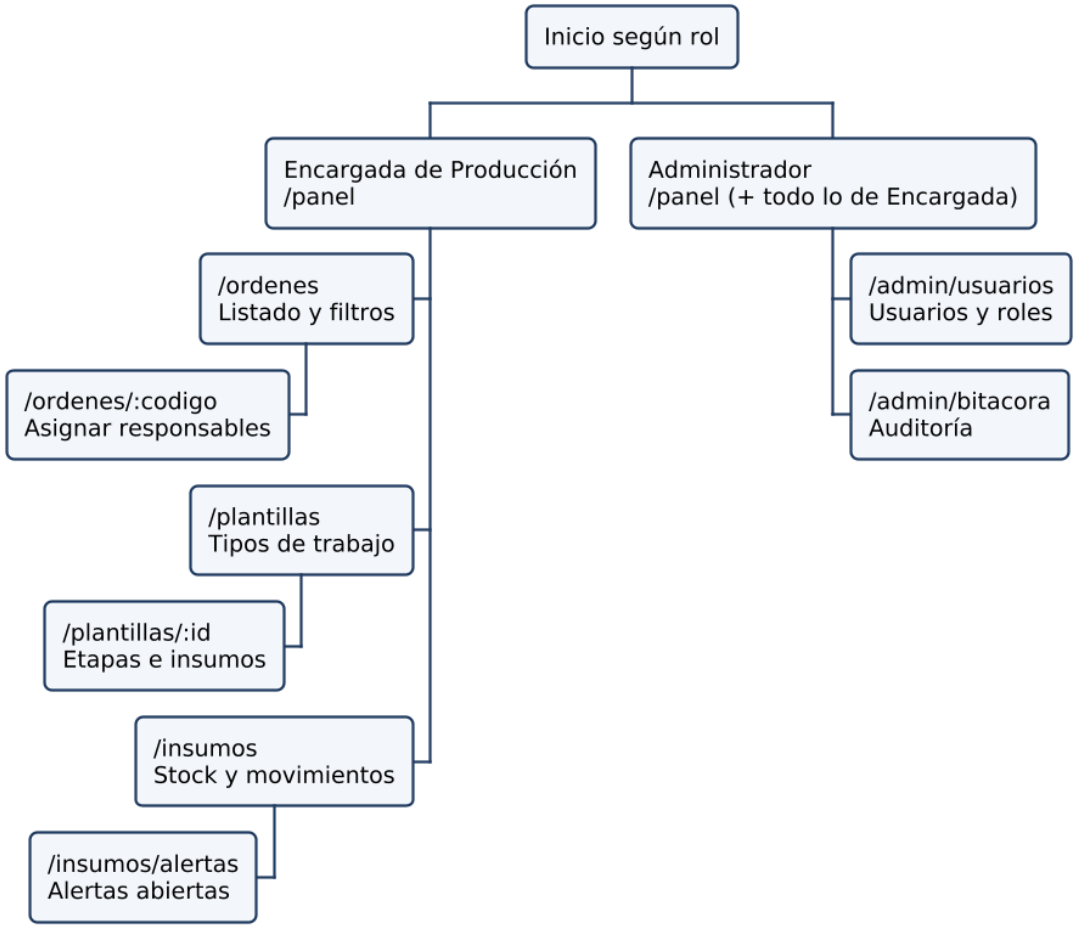


Figura 5.3 — Mapa de navegación: perfiles de gestión



Regla de navegación	**Implementación**
Inicio por rol	Tras el login: Operario/Diseñador → `/mi-trabajo`; Vendedor → `/ordenes`; Encargada/Administrador → `/panel`
Navegación principal	**Móvil:** barra inferior con 3–4 destinos según el rol (íconos + texto). **Escritorio:** barra lateral izquierda
Profundidad máxima	3 niveles (p. ej., Panel → OT → Etapa); botón “Volver” siempre visible
Guardas de ruta	Las rutas no permitidas para el rol no se muestran en el menú y, si se acceden por URL, muestran “No tienes acceso”. La API aplica la autorización real (sección 7.3)
URL compartibles	`/ordenes/ot-2026-0001` permite compartir el enlace por el canal interno sin exponer datos: requiere sesión

5.3 Prototipo interactivo funcional

Elemento	Información
Herramienta utilizada	Maqueta navegable en HTML5 + CSS3 con *design tokens* y JavaScript mínimo (solo navegación por rol, diálogos y cambio de estado visual). Sin back-end; los datos y los nombres de clientes son ficticios, y la maqueta no menciona a la empresa para poder publicarse
Ubicación	Anexo D (maqueta navegable entregada junto con el informe)
Enlace al prototipo	Publicación en GitHub Pages: `https://<usuario>.github.io/sgp-prototipo/` [Completar al publicar]
Credenciales	No aplica: la pantalla de login simula el acceso con un selector de rol.
Interacciones clave demostradas	IK-1 login, MFA para roles privilegiados y redirección por rol · IK-2 Mi Trabajo → detalle de OT · IK-3 Iniciar/Terminar etapa con cambio de estado y mensaje "Sigue: …"; Reportar problema deja la etapa Bloqueada con motivo obligatorio (Fig. 4.7) · IK-4 Reportar faltante (diálogo) · IK-5 Registrar OT con vista previa de etapas generadas · IK-6 Aprobar/Rechazar diseño con motivo obligatorio y segregación de funciones (R6) · IK-7 Panel con indicadores que filtran atrasadas, en riesgo y bloqueadas · IK-8 Plantilla de proceso con etapas ordenadas · IK-9 Insumos con ciclo de alerta (abierta → en gestión → cerrada) y registro de ingreso · IK-10 Navegación por rol (barra inferior en móvil, lateral en escritorio) y guarda "No tienes acceso" · IK-11 Control de la OT en P-03: detener con motivo, reanudar, anular con motivo y marcar entregada (Fig. 4.8), con el historial de la orden para la Encargada

Pantallas del prototipo (Tabla 5.1):

ID	Pantalla	Perfil	HU / CU
P-01	Inicio de sesión y verificación MFA	Todos (MFA: Encargada, Administrador)	HU-11 / CU-01
P-02	Mi Trabajo (incluye estados vacío, cargando y sin conexión)	Operario, Diseñador	HU-01 / CU-05
P-03	Detalle de orden y etapa	Operario y Diseñador (avanzan su etapa), Vendedor (lectura y marcar entregada), Encargada y Administrador (asignan, omiten, detienen, reanudan o anulan y ven el historial de la orden)	HU-02, HU-03, HU-04, HU-08, HU-10, HU-12 / CU-06, CU-09, CU-11, CU-12, CU-13
P-04	Órdenes y nueva orden de trabajo	Vendedor, Encargada, Administrador	HU-05 / CU-04
P-05	Aprobación de diseño	Vendedor (resuelve), Diseñador (sube versiones)	HU-06 / CU-07
P-06	Panel de supervisión	Encargada, Administrador; Vendedor (solo lectura)	HU-10 / CU-10, CU-14
P-07	Plantillas de proceso	Encargada, Administrador	HU-07 / CU-03
P-08	Insumos y alertas	Encargada (gestiona); demás roles (consulta)	HU-09 / CU-08, CU-09
P-09	Usuarios y roles	Administrador	HU-11 / CU-02
P-10	Bitácora	Administrador	HU-12 / CU-11

ID	Pantalla	Perfil	HU / CU
P-11	Avisos	Todos	HU-01 / CU-06 (notificar siguiente responsable)
P-12	Mi perfil	Todos	HU-11 / CU-01

Especificación de la interacción crítica IK-3 (P-03): la tarjeta de la etapa muestra un **chip de estado** (Disponible → En curso → Terminada) y un único **botón primario** contextual ("Iniciar etapa" → "Terminar etapa"). Al terminar, aparece un diálogo para confirmar los consumos, prellenados con la cantidad requerida y editables. Al confirmar, se muestra una franja de éxito con el texto **"Listo. Sigue: Sublimado — área Sublimación (responsable: ...)"**, que es la respuesta directa al problema "no sé qué sigue". Las acciones secundarias ("Reportar faltante", "Reportar problema", "Adjuntar foto") están agrupadas bajo el botón primario y separadas visualmente para evitar toques accidentales.

5.4 Consistencia visual y adaptabilidad

La guía de estilo se implementa como *design tokens* (variables CSS del prototipo, Anexo D) que se reutilizarán en el front-end real. La identidad corporativa de Kotoduo no está documentada en el levantamiento: la paleta propuesta usa un azul institucional con un acento naranja (energía, publicidad) y se ajustará a su logotipo en la validación A01.

Paleta cromática y contraste (WCAG 2.2, criterio 1.4.3 AA $\geq$ 4,5:1 para texto normal; Tabla 5.2):

Token	Hex	Uso	Contraste sobre blanco	Contraste sobre fondo `#F8FAFC`
`--color-primario`	`#1E40AF`	Botones primarios, enlaces, barra de navegación activa	8,72:1 ✓	8,34:1 ✓
`--color-acento`	`#C2410C`	Prioridad URGENTE, destacados	5,18:1 ✓	4,95:1 ✓
`--color-exito`	`#15803D`	Estado Terminada, confirmaciones	5,02:1 ✓	4,79:1 ✓
`--color-advertencia`	`#A16207`	Estado Bloqueada, OT en riesgo, stock bajo	4,92:1 ✓	4,71:1 ✓
`--color-peligro`	`#B91C1C`	Atrasada, alerta de insumo, errores	6,47:1 ✓	6,18:1 ✓
`--color-texto`	`#111827`	Texto principal	17,74:1 ✓	16,96:1 ✓
`--color-texto-2`	`#4B5563`	Texto secundario, metadatos	7,56:1 ✓	7,22:1 ✓
`--color-fondo`	`#F8FAFC`	Fondo de la aplicación	—	—

El color **nunca es el único portador de significado** (WCAG 1.4.1): cada estado combina color, ícono y texto (p. ej., "⚠ Bloqueada").

Aspecto	Regla / decisión	Evidencia
Tipografía y jerarquía	Fuente del sistema (`system-ui`: Roboto en Android, San Francisco en iOS, Segoe UI en Windows); no requiere descarga, es legible y familiar. Escala: H1 1,5 rem/700, H2 1,25 rem/600, cuerpo 1 rem (16 px mínimo, evita el zoom automático en iOS), metadatos 0,875 rem. Interlineado 1,5	Todas las pantallas del prototipo (tokens `--fs-*`)
Colores y contraste	Tabla 5.2; foco visible de 3 px con `--color-primario` (WCAG 2.4.7 / 2.4.11)	P-01 a P-08
Componentes y consistencia	Biblioteca única: botón (primario, secundario, peligro), chip de estado, tarjeta de OT, tabla, diálogo, campo con etiqueta visible y mensaje de error asociado (`aria-describedby`), franja de aviso. Mismo verbo, misma acción en todas las pantallas ("Terminar etapa")	Clases `.btn`, `.chip`, `.card` y `dialog` del prototipo
Estados del sistema	Vacío ("No tienes etapas pendientes"), cargando (esqueleto), error con acción ("Sin conexión" + "Reintentar"), éxito (franja verde)	P-02, P-03
Áreas táctiles	Objetivos de al menos 48×48 px (supera el mínimo de 24×24 px de WCAG 2.5.8); separación ≥ 8 px entre acciones	Botones de P-03
Diseño adaptable / responsive	*Mobile-first* con CSS Grid/Flexbox y tres *breakpoints*: < **600 px** (1 columna, barra inferior, tablas como tarjetas), **600–1023 px** (2 columnas, tablet), ≥ **1024 px** (barra lateral + contenido de hasta 1200 px, tablas completas). Imágenes `max-width: 100%`; sin desplazamiento horizontal a 320 px (WCAG 1.4.10 *Reflow*)	Pruebas en 360×800 (Android típico), 390×844 (iPhone), 768×1024 y 1366×768; CP-34
Idioma y redacción	`lang="es-CL"`, tuteo, frases cortas, fechas `dd-mm-aaaa`, sin anglicismos técnicos	Todas

5.5 Retroalimentación de usuarios y mejoras aplicadas

La retroalimentación se gestiona en dos rondas:

- **Ronda 0** (antecedentes del cliente): observaciones reales entregadas por Kotoduo en el documento de antecedentes complementarios, que modificaron el diseño inicial.
- **Ronda 1** (actividad A06): prueba de usabilidad moderada del prototipo v1 con al menos 5 participantes (2 operarios, 1 vendedor, 1 diseñador y la encargada de producción). Cada participante realiza las tareas T1 "¿Qué te toca hacer ahora y qué sigue?", T2 "Termina la etapa de impresión", T3 "Avisa que se acabó el papel" y T4 "Registra una orden de 30 camisetas", con protocolo de pensamiento en voz alta, tiempo por tarea, tasa de éxito y cuestionario SUS (Brooke, 1996).

Tabla 5.3 — Registro de retroalimentación

ID	Ronda / fecha	Usuario / perfil	Observación	Decisión	Cambio realizado / justificación
FB-01	R0 / antecedentes	Contraparte Kotoduo (gerencia)	"Sería crear una app para el celular, así es más práctica para los trabajadores"	Incorporar	Diseño *mobile-first* como PWA instalable; navegación inferior en móvil (sección 4.7, 5.4)
FB-02	R0 / antecedentes	Contraparte Kotoduo	"Que sea un sistema fácil, práctico y amigable"	Incorporar	Inicio por rol con una sola tarea principal; botón primario único y contextual en P-03 (IK-3); lenguaje del taller
FB-03	R0 / antecedentes	Contraparte Kotoduo	Excluir "los contactos de clientes" y "temas de dinero"	Incorporar	P-04 sin teléfono, correo, precio ni pago; campo `referencia_cliente` limitado al nombre de la institución (sección 6.2)
FB-04	R0 / antecedentes	Contraparte Kotoduo	La aceptación del diseño es información relevante del proceso	Incorporar	Se agrega el módulo M6 y la pantalla P-05; la producción no se habilita sin diseño aprobado (Fig. 4.15)
FB-05	R0 / antecedentes	Contraparte Kotoduo	Trabajadores "sin quedar parados cuando no estén las personas a cargo"	Incorporar	El mensaje "Sigue: etapa — área — responsable" al terminar, y las instrucciones visibles en cada etapa, se convierten en el elemento central de P-03
FB-06	R0 / antecedentes	Contraparte Kotoduo	Perfiles "administrador" y "de respuesta"	Incorporar con ajuste	Se refinan en 5 roles para respetar el mínimo privilegio; la equivalencia se valida en A01 (sección 7.1)
FB-07	R1 / [dd-mm]	[Operario 1]	[Completar tras la sesión A06]	[Incorporar / No incorporar]	[Cambio v1 → v2 y pantalla afectada]
FB-08	R1 / [dd-mm]	[Operario 2]	[Completar]	☐	☐
FB-09	R1 / [dd-mm]	[Vendedor]	[Completar]	☐	☐
FB-10	R1 / [dd-mm]	[Diseñador]	[Completar]	☐	☐
FB-11	R1 / [dd-mm]	[Encargada de Producción]	[Completar]	☐	☐

Criterios de decisión: se incorpora la observación si (a) afecta a una tarea crítica (T1–T3), (b) se repite en 2 o más participantes, o (c) impide completar una tarea. Se posterga al backlog si es una mejora estética o de bajo impacto. Se descarta si contradice el alcance acordado, y en ese caso se documenta la razón. La comparación v1 → v2 se adjunta en el Anexo D (capturas antes/después).

6 Diseño e implementación del modelo de datos

6.1 Modelo de datos: entidades y relaciones

Simbología: el modelo usa la notación de Ingeniería de la Información ("pata de gallo"). `||` significa exactamente uno, `|o` cero o uno, `|{` uno o muchos y `o{` cero o muchos. Los atributos marcados con `*` son obligatorios (NOT NULL). `<<PK>>`, `<<FK>>` y `<<UK>>` indican clave primaria, foránea y única. La línea punteada `etapa_orden` → `plantilla_etapa` señala una relación no identificadora y opcional. Las tablas en gris (`«ref»`) se detallan en otra figura.

Figura 6.1 — Modelo entidad-relación: visión general. Muestra las 16 tablas y todas sus relaciones, sin atributos. El color identifica el grupo y la figura donde se detalla.

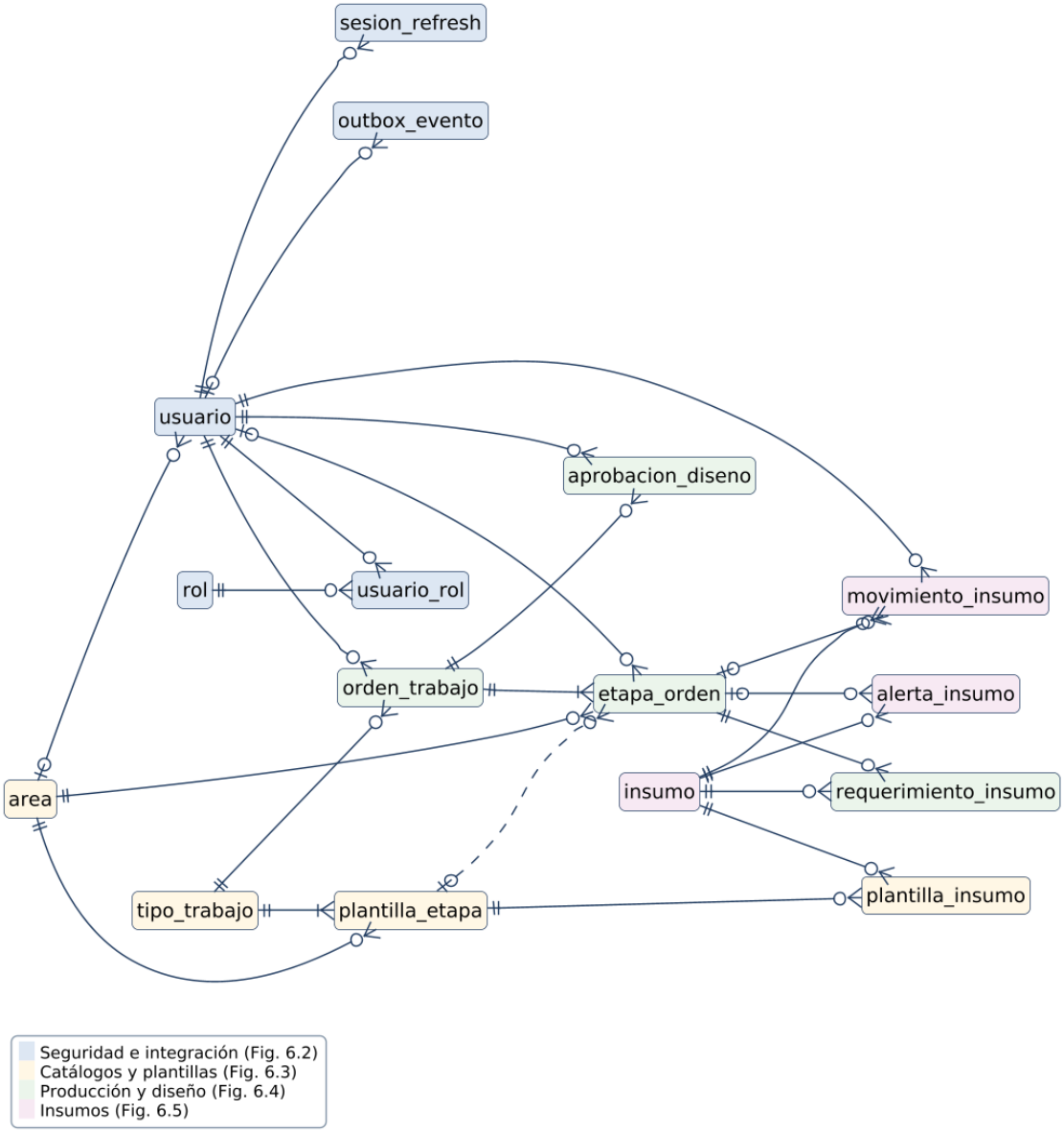


Figura 6.2 — ER: seguridad e integración

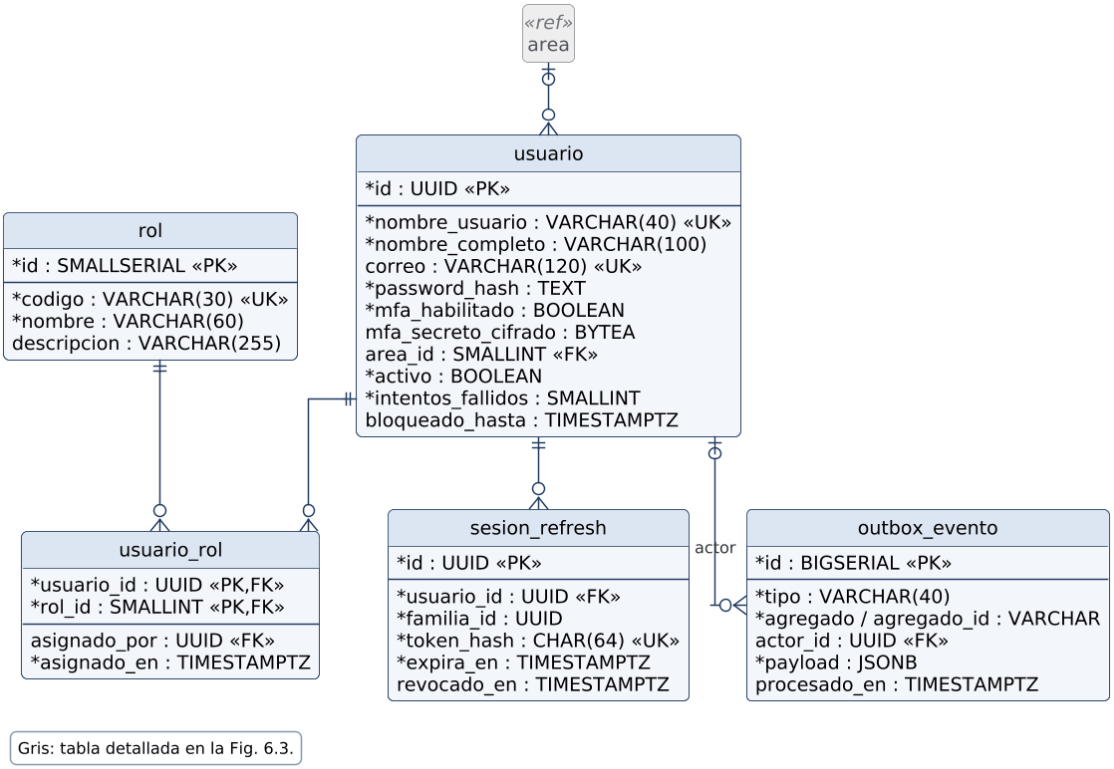


Figura 6.3 — ER: catálogos y plantillas de proceso

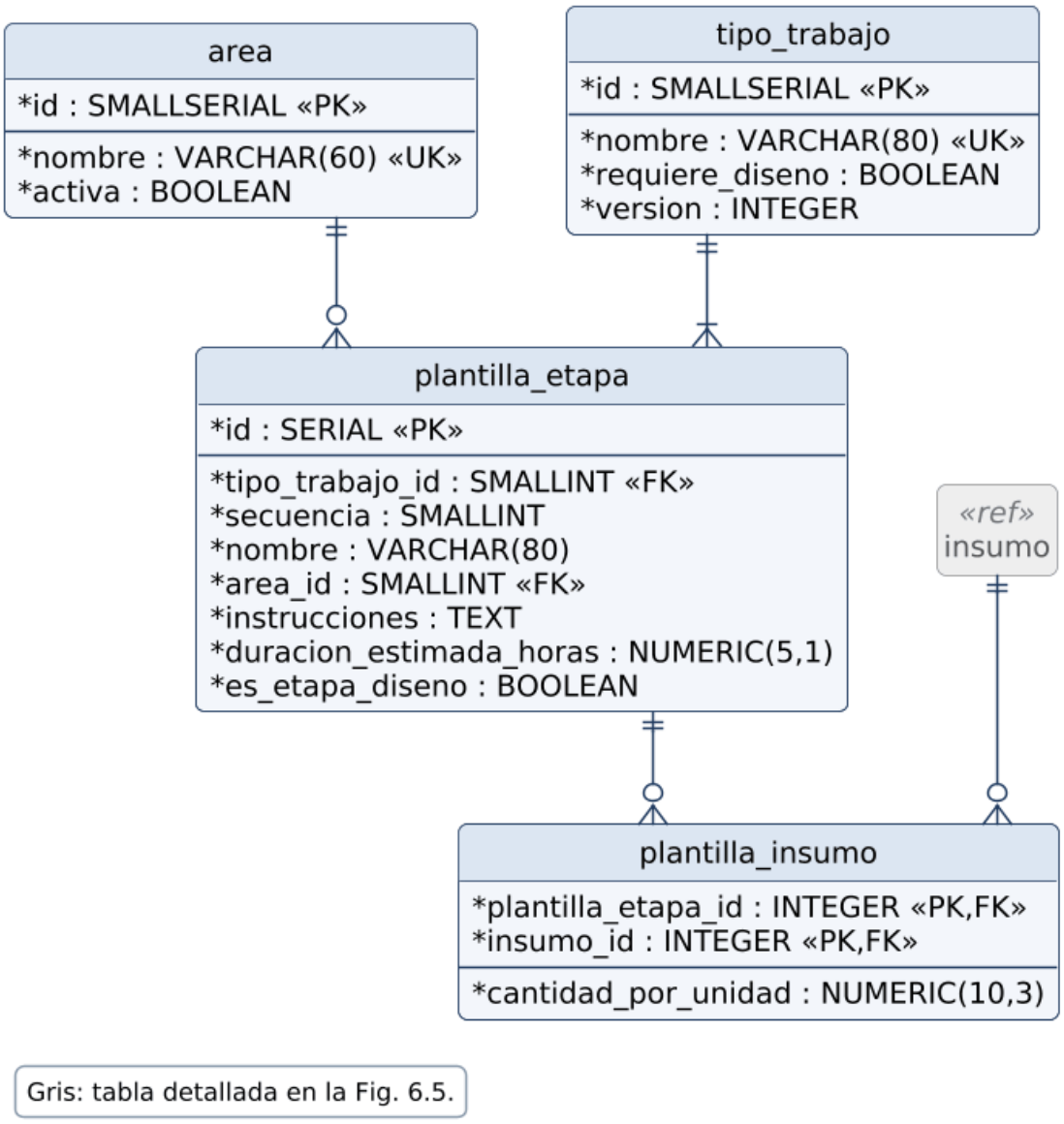


Figura 6.4 — ER: producción y diseño

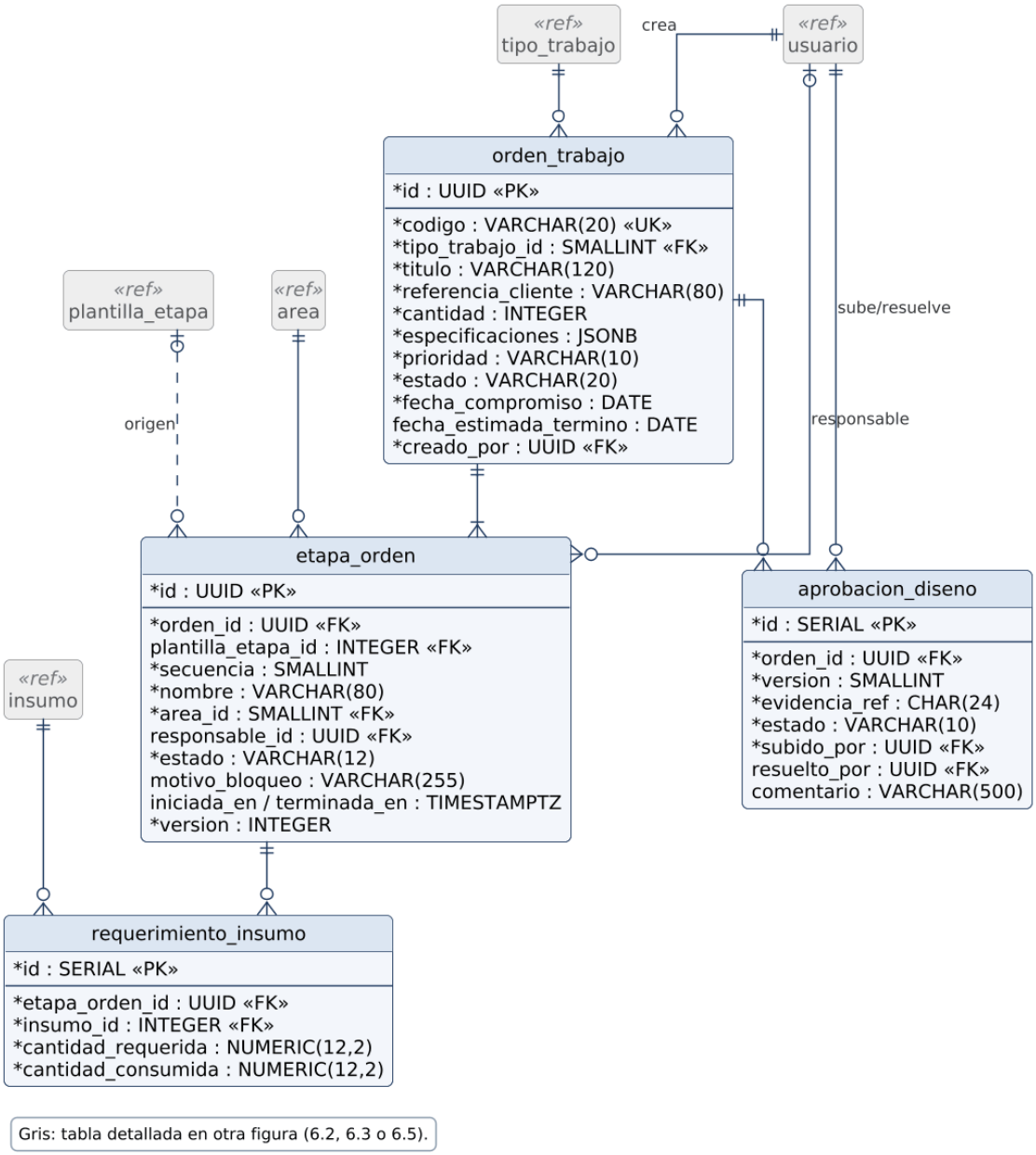
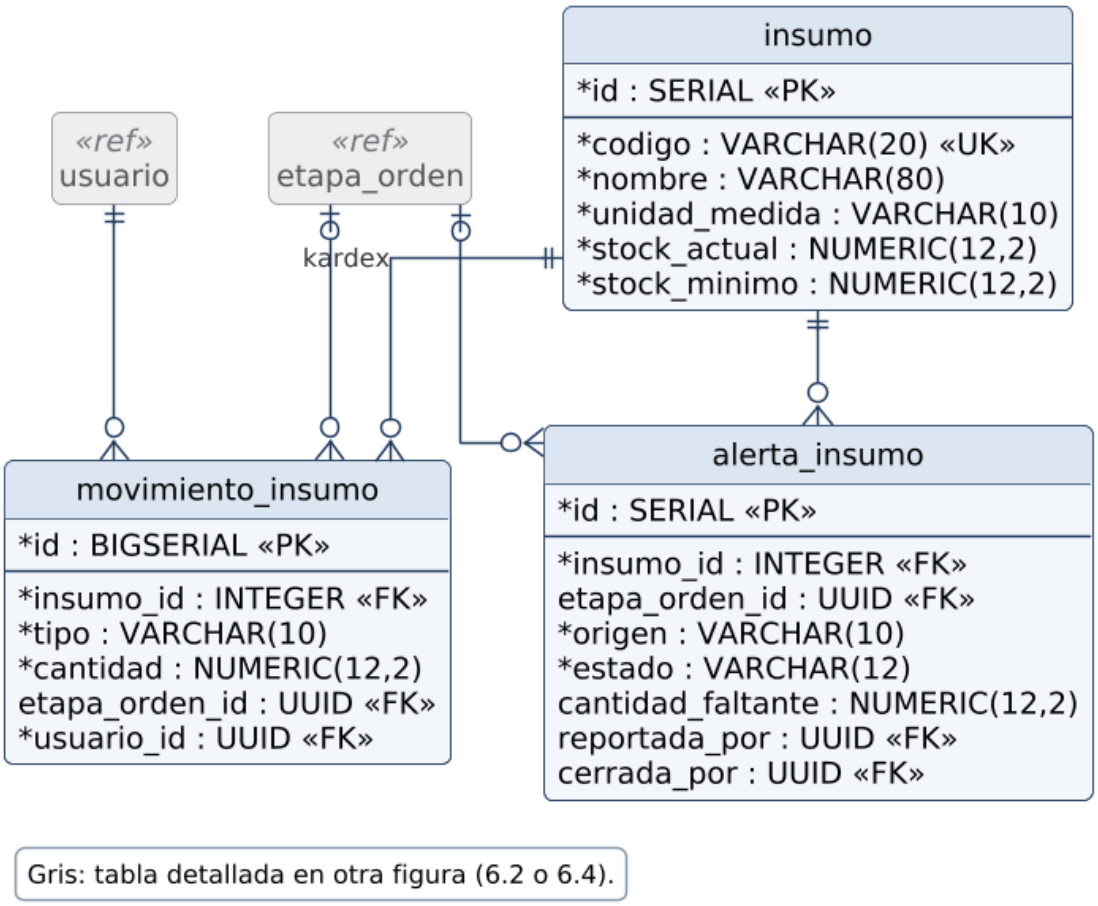


Figura 6.5 — ER: insumos, kardex y alertas



El modelo tiene **16 tablas** en el esquema `sgp`, agrupadas por módulo:

Grupo	Tablas	Módulo
Seguridad	`rol`, `usuario`, `usuario_rol`, `sesion_refresh`	M1
Catálogos	`area`, `tipo_trabajo`, `plantilla_etapa`, `insumo`, `plantilla_insumo`	M2, M5
Producción	`orden_trabajo`, `etapa_orden`, `requerimiento_insumo`	M3, M4
Insumos	`movimiento_insumo`, `alerta_insumo`	M5
Diseño	`aprobacion_diseno`	M6
Integración	`outbox_evento`	M8

Relaciones y cardinalidades principales:

Relación	Cardinalidad	Regla de negocio que expresa
`usuario` — `rol` (vía `usuario_rol`)	N:M	Un usuario puede tener varios roles (p. ej., Encargada y Vendedor en temporada alta)
`tipo_trabajo` — `plantilla_etapa`	1:N (1..*)	Un tipo de trabajo define al menos una etapa ordenada

Relación	Cardinalidad	Regla de negocio que expresa
`plantilla_etapa` — `insumo` (vía `plantilla_insumo`)	N:M con atributo	Cada etapa estándar consume insumos por unidad producida
`orden_trabajo` — `etapa_orden`	1:N (1..*) identificadora	Una OT se compone de sus etapas; se eliminan en cascada
`etapa_orden` — `usuario` (responsable)	N:0..1	Una etapa puede estar aún sin asignar
`etapa_orden` — `insumo` (vía `requerimiento_insumo`)	N:M con atributos	Cantidad requerida y consumida por etapa concreta
`insumo` — `movimiento_insumo`	1:N	Kardex: el stock es la suma de movimientos
`orden_trabajo` — `aprobacion_diseno`	1:N	Versiones sucesivas del diseño

Decisiones de optimización:

1. **Tercera forma normal.** `requerimiento_insumo` no almacena `orden_id`, porque se deriva de `etapa_orden` y evita la anomalía de actualización. `rol` y `area` son catálogos referenciados en lugar de textos repetidos.
2. **Desnormalización controlada y justificada.**
   - `etapa_orden` copia nombre, área, instrucciones y duración desde la plantilla: es una *instantánea* histórica, no una redundancia, porque la plantilla puede cambiar.
   - `insumo.stock_actual` es un valor derivado del kardex, mantenido por el trigger R3 para evitar sumar miles de movimientos en cada consulta.
3. **JSONB acotado.** `orden_trabajo.especificaciones` guarda tallas, medidas y colores, que varían por tipo de trabajo, con `CHECK jsonb_typeof = 'object'`. Los atributos por los que se filtra (estado, fechas, tipo) son columnas relacionales indexadas.
4. **Claves.** Se usa UUID en las entidades expuestas en URL o eventos (`usuario`, `orden_trabajo`, `etapa_orden`): no son enumerables, lo que dificulta el IDOR, y se pueden generar sin coordinación. Se usan `SERIAL` en catálogos internos. El código de negocio `OT-AAAA-NNNN` es una clave alternativa (`UNIQUE`).
5. **Índices diseñados por consulta.**
   - `ix_etapa_responsable_estado` para *Mi Trabajo*.
   - `ix_ot_estado_compromiso` para el panel.
   - `ix_mov_insumo_fecha` para el kardex.
   - Índices **parciales**: `ux_alerta_auto_abierta` (a lo más una alerta automática abierta por insumo), `ux_aprobacion_pendiente` e `ix_outbox_pendiente`.

6.2 Diccionario y reglas de integridad

El diccionario completo documenta cada campo de las 16 tablas con su tipo, clave o restricción y descripción, y está en el **Anexo J (Tabla J.1)**. La Tabla 6.1 resume, por tabla, la clave primaria y las restricciones que sostienen las reglas de negocio.

Tabla 6.1 — Resumen del diccionario de datos (PK = clave primaria, FK = clave foránea, UQ = única, CK = restricción CHECK)

Tabla	PK	Restricciones destacadas
`rol`	SMALLSERIAL	Código UQ, limitado a los cinco roles del RBAC
`area`	SMALLSERIAL	Nombre UQ; baja lógica (`activa`)
`usuario`	UUID	`nombre_usuario` UQ con formato validado; hash Argon2id; secreto TOTP cifrado; control de intentos fallidos; se desactiva en vez de borrarse
`usuario_rol`	(usuario_id, rol_id)	FK a `usuario` en cascada y a `rol` con RESTRICT; registra quién asignó el rol
`sesion_refresh`	UUID	Solo guarda el hash SHA-256 del token (UQ); `familia_id` para detectar reutilización; CK la expiración es posterior a la emisión
`tipo_trabajo`	SMALLSERIAL	Nombre UQ; indica si requiere diseño
`plantilla_etapa`	SERIAL	UQ (tipo_trabajo_id, secuencia); duración CK $>$ 0; marca la etapa de diseño
`insumo`	SERIAL	Código UQ; unidad de medida controlada; stock y mínimo CK ≥ 0
`plantilla_insumo`	(plantilla_etapa_id, insumo_id)	Consumo por unidad CK > 0
`orden_trabajo`	UUID	Código UQ `OT-AAAA-NNNN`; estado y prioridad controlados (trigger R2); sin datos de contacto del cliente
`etapa_orden`	UUID	UQ (orden_id, secuencia); estado controlado (trigger R2); motivo obligatorio si está bloqueada; `version` para concurrencia optimista
`requerimiento_insumo`	SERIAL	UQ (etapa, insumo); cantidad requerida > 0 y consumida ≥ 0
`movimiento_insumo`	BIGSERIAL	Signo según el tipo (ingreso, consumo, ajuste); etapa obligatoria si es consumo; inmutable
`alerta_insumo`	SERIAL	Origen automático o manual; CERRADA $\Leftrightarrow$ `cerrada_en`; faltante > 0
`aprobacion_diseno`	SERIAL	UQ (orden_id, version); una sola versión PENDIENTE por OT; quien sube el diseño no lo resuelve (segregación)
`outbox_evento`	BIGSERIAL	`payload` JSONB obligatorio; índice parcial sobre los eventos pendientes

Reglas de integridad referencial (política ON DELETE):

Política	Dónde	Justificación
`CASCADE`	`orden_trabajo` → `etapa_orden` → `requerimiento_insumo`; `tipo_trabajo` → `plantilla_etapa` → `plantilla_insumo`; `usuario` → `sesion_refresh`, `usuario_rol`	Composición: las partes no tienen sentido sin el todo. En la práctica las OT no se borran, se anulan
`RESTRICT`	Referencias a `insumo`, `area`, `tipo_trabajo`, `rol`; `orden_trabajo.creado_por`; `movimiento_insumo.usuario_id`; `aprobacion_diseno.subido_por`	Impide perder historia: no se elimina un insumo con movimientos ni un usuario autor (se desactiva con `activo = FALSE`)
`SET NULL`	`etapa_orden.responsable_id`, `etapa_orden.plantilla_etapa_id`, `usuario.area_id`, referencias opcionales de alertas y outbox	La entidad sobrevive aunque desaparezca la referencia opcional

Reglas de negocio implementadas en la base de datos:

Regla	Mecanismo	Verificación
R1. `actualizado_en` siempre vigente	Trigger `fn_set_actualizado_en`	—
R2. Solo transiciones de estado válidas en `etapa_orden` (Fig. 4.7) y en `orden_trabajo` (Fig. 4.8), con sus condiciones: la OT termina solo con todas sus etapas cerradas, se reanuda solo sin etapas bloqueadas y, mientras está DETENIDA, ninguna etapa se inicia ni se termina; en la etapa, con incremento de `version`	Triggers `trg_etapa_transicion` y `trg_orden_transicion` (lanzan `check_violation`)	CP-02
R3. Cada movimiento actualiza el stock; si queda ≤ mínimo, se crea **una** alerta automática	Trigger `trg_movimiento_insumo` + índice único parcial + `ON CONFLICT DO NOTHING`	CP-14, consulta 6
R4. Stock nunca negativo	`CHECK (stock_actual >= 0)` aborta la transacción completa	CP-14
R5. Bloqueo con motivo; terminada con fecha	`ck_etapa_bloqueo`, `ck_etapa_terminada`	CP-01
R6. Quien sube un diseño no puede aprobarlo	`ck_aprobacion_segregacion`	CP-16
R7. Registros inmutables (kardex, outbox)	`REVOKE UPDATE, DELETE` al rol `sgp_app`	CP-32 (revisión de privilegios)
R8. Mínimo privilegio en la conexión	Roles `sgp_app` (DML) y `sgp_relay` (solo `procesado_en` del outbox); la aplicación no usa el superusuario	Checklist ASVS (CP-32)

6.3 Bases de implementación de la base de datos estructurada

En esta etapa se fijan los **principios y buenas prácticas** con que se construirá el esquema `sgp` en PostgreSQL 16; no se incluye código. El script se escribe como migraciones versionadas en A11 (Sprint 1) y se completa en cada Sprint junto al módulo que lo usa. Para la sintaxis y los ejemplos concretos, el equipo seguirá la documentación oficial citada en cada fila. El **Anexo B** contiene un borrador referencial del esquema (DDL y consultas de ejemplo), que sirve como punto de partida para A11 y se ajustará en las migraciones.

Tabla 6.2 — Principios de implementación en PostgreSQL

Principio	Qué se establece	Reglas del modelo que lo aplican	Documentación de referencia
Integridad declarativa primero	Las reglas que se pueden expresar como restricción (`PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `CHECK`, `NOT NULL`) se declaran en el esquema y no solo en la aplicación. La base de datos rechaza un dato inválido venga de donde venga	R4, R5, R6 y las políticas `ON DELETE` (6.2)	PostgreSQL 16, *Constraints*: `https://www.postgresql.org/docs/16/ddl-constraints.html`
Transacciones ACID por caso de uso	Cada caso de uso que modifica varias tablas (p. ej., terminar una etapa) se ejecuta en **una** transacción: o se aplican todos los cambios o ninguno. Se usa el nivel de aislamiento por defecto (*Read Committed*) más bloqueo explícito donde hay competencia	CU-06, outbox (6.5)	*Transaction Isolation*: `https://www.postgresql.org/docs/16/transaction-iso.html`
Control de concurrencia	Bloqueo pesimista de fila (`SELECT … FOR UPDATE`) sobre la etapa que se modifica, combinado con concurrencia optimista mediante la columna `version`. Así, dos operarios que actúan a la vez producen una sola transición	R2, CP-19	*Explicit Locking*: `https://www.postgresql.org/docs/16/explicit-locking.html`
Triggers solo para invariantes críticas	Se reservan para reglas que deben cumplirse ante cualquier cliente y que una restricción no puede expresar: transiciones de estado, mantenimiento del stock derivado y fecha de actualización. La lógica de negocio general permanece en la capa de servicio	R1, R2, R3	*Trigger Functions* (PL/pgSQL): `https://www.postgresql.org/docs/16/plpgsql-trigger.html`
Operaciones idempotentes	Las inserciones que pueden repetirse (alerta automática de insumo) usan una restricción única y la cláusula `ON CONFLICT`, de modo que un reintento o una carrera no duplican el registro	R3	*INSERT — ON CONFLICT*: `https://www.postgresql.org/docs/16/sql-insert.html`

Principio	Qué se establece	Reglas del modelo que lo aplican	Documentación de referencia
Índices diseñados por consulta	Cada índice se justifica por una consulta frecuente (Mi Trabajo, panel, kardex). Los índices **parciales** cubren solo el subconjunto relevante (alertas abiertas, outbox pendiente) y además garantizan unicidad condicional. Se verifican con `EXPLAIN` en A26	Índices de 6.1	*Partial Indexes*: `https://www.postgresql.org/docs/16/indexes-partial.html`; *Using EXPLAIN*: `https://www.postgresql.org/docs/16/using-explain.html`
JSONB acotado	Solo los atributos variables por tipo de trabajo van en `JSONB`; todo atributo por el que se filtra o se relaciona es una columna tipada	`orden_trabajo.especificaciones`	*JSON Types*: `https://www.postgresql.org/docs/16/datatype-json.html`
Mínimo privilegio	La aplicación se conecta con un rol sin privilegios de administración ni de DDL. Los registros históricos (kardex, outbox) no admiten `UPDATE` ni `DELETE` para ese rol	R7, R8	*Privileges*: `https://www.postgresql.org/docs/16/ddl-priv.html`; *Database Roles*: `https://www.postgresql.org/docs/16/user-manag.html`
Esquema versionado	Todo cambio de estructura es una migración revisada en PR y aplicada por el pipeline, nunca un cambio manual en producción. Cada migración se prueba primero en staging	A10, A11	Documentación de `node-pg-migrate`: `https://salsita.github.io/node-pg-migrate/`
Respaldo y recuperación	Se usa la recuperación a un punto en el tiempo (PITR) del proveedor gestionado y se prueba una restauración antes del piloto	RNF-05, A29	*Continuous Archiving and PITR*: `https://www.postgresql.org/docs/16/continuous-archiving.html`

Criterios de verificación. Cada criterio se verifica dos veces: primero sobre el borrador del Anexo B en un ambiente local (Tabla 6.4, ya ejecutada) y luego, durante la construcción, sobre la base de datos de staging, con captura (Figura 6.6) y reporte de pruebas (Anexo F):

Comportamiento esperado	Regla	Caso de prueba
Una transición inválida (p. ej., PENDIENTE → TERMINADA en una etapa o TERMINADA → EN_PRODUCCION en una OT) es rechazada por la base de datos	R2	CP-02
Una OT con etapas abiertas no pasa a TERMINADA, no se reanuda con una etapa bloqueada y, mientras está DETENIDA, no se inicia ninguna de sus etapas	R2	CP-02

Comportamiento esperado	Regla	Caso de prueba
Terminar una etapa actualiza la etapa, habilita la siguiente y registra el evento outbox en una sola transacción	Transacción por caso de uso	CP-01, CP-11
Un consumo que dejaría stock negativo aborta la transacción completa	R4	CP-14
Varios consumos bajo el mínimo generan **una sola** alerta automática abierta	R3	CP-14
No se puede eliminar un insumo con movimientos	`RESTRICT`	CP-14
Quien sube un diseño no puede aprobarlo	R6	CP-16
Dos terminaciones simultáneas de la misma etapa producen una sola transición	Concurrencia	CP-19
El rol de la aplicación no puede modificar el kardex ni el outbox	R7, R8	CP-32

Verificación anticipada de los borradores. Para no esperar a la construcción, los borradores de los Anexos B y C se ejecutaron sobre los motores reales con contenedores Docker locales (PostgreSQL 16.13 y MongoDB 9.0.4) el 22-10-2026. Se usó MongoDB 9.0 en local porque la versión 8.0 no arranca en kernels Linux 6.19 o superiores (SERVER-121912); la 9.0 expone la misma API y el diseño sigue apuntando a MongoDB 8 en Atlas. Cada fila resume una prueba sobre una base recién creada; las pruebas que modifican datos se revierten al terminar.

Tabla 6.4 — Resultados de la verificación local de los borradores

Verificación	Regla	Resultado obtenido
Creación del esquema `sgp` deteniéndose ante el primer error	—	16 tablas y 6 triggers creados sin errores
Consultas de ejemplo del Anexo B: registro de OT, Mi Trabajo, avance de etapa, panel y faltantes proyectados	—	Ejecutadas sin errores
Etapa PENDIENTE → TERMINADA	R2	Rechazada: “Transición de etapa no permitida: PENDIENTE -> TERMINADA”
OT a TERMINADA con etapas abiertas y luego con todas cerradas	R2	Rechazada: “La orden no puede terminar: tiene etapas abiertas”; aceptada con todas las etapas cerradas
OT TERMINADA → EN_ PRODUCCION	R2	Rechazada: “Transición de orden no permitida”
Iniciar una etapa con la OT DETENIDA	R2	Rechazada: “La orden está DETENIDA”
Reanudar una OT con una etapa BLOQUEADA; luego desbloquear y reanudar	R2	Rechazada: “tiene etapas bloqueadas”; aceptada después de desbloquear la etapa

Verificación	Regla	Resultado obtenido
Dos consumos que dejan el stock bajo el mínimo	R3	Una sola alerta automática abierta
Consumo que dejaría el stock negativo	R4	Rechazado; el kardex conserva sus 2 movimientos (transacción revertida)
Eliminar un insumo con movimientos	`RESTRICT`	Bloqueado por integridad referencial
Quien sube un diseño lo aprueba	R6	Rechazado por `ck_aprobacion_segregacion`
El rol `sgp_app` modifica el kardex, borra el outbox o crea una tabla	R7, R8	"permission denied" en los tres casos
Colecciones de MongoDB con validador e índices	6.4	3 colecciones con validador en modo de rechazo; índices TTL de 24 meses (bitácora) y 90 días (notificaciones)
Documento con un tipo de evento inexistente	6.4	Rechazado (código 121, *Document failed validation*)
Reintento de la publicación de un evento ya procesado	6.4, 6.5	0 documentos nuevos (*upsert* idempotente)
Consulta de la bandeja de notificaciones	6.4	Resuelta con el índice `ix_bandeja`

[Insertar captura de la ejecución en staging como Figura 6.6, una vez completado A11.]

6.4 Bases de implementación de la base de datos no estructurada

MongoDB (base `sgp_docs`) almacena la información **de esquema variable, alto volumen de escritura y sin necesidad de *joins***. Igual que en 6.3, aquí se definen las decisiones de modelado y las buenas prácticas; la sintaxis concreta se toma de la documentación oficial de MongoDB. El **Anexo C** contiene un borrador referencial de los validadores, los índices y el script de inicialización.

Colección	Propósito	Clave / identificador	Índices	Retención
`bitacora_eventos`	Auditoría inmutable (RF-12)	Identificador determinista derivado del `id` de `outbox_evento` (idempotente)	Por orden y fecha; por actor y fecha; por tipo y fecha	TTL 24 meses (RNF-05)
`evidencias_etapa`	Fotos, notas y versiones de diseño con metadatos libres por tipo de trabajo (RF-07, RF-08)	`ObjectId` (referenciado por `aprobacion_diseno.evidencia_ref`)	Por orden y fecha; por etapa; por hash del archivo	Indefinida
`notificaciones`	Bandeja in-app y estado del push	Identificador determinista por evento y destinatario	Por destinatario, estado de lectura y fecha	TTL 90 días

Tabla 6.3 — Principios de implementación en MongoDB

Principio	Qué se establece	Documentación de referencia
Modelar según los patrones de acceso	Se **incrusta** lo que se lee siempre junto (actor, estado anterior y nuevo dentro del evento) y se **referencia** lo que tiene ciclo de vida propio (la OT y la etapa, que viven en PostgreSQL). Cada documento se diseña para responder a una pantalla concreta sin *joins*	MongoDB Manual, *Data Modeling*: `https://www.mongodb.com/docs/manual/data-modeling/`
Esquema flexible, pero validado	Cada colección tiene un validador `$jsonSchema` en modo de rechazo: los campos obligatorios, los tipos y los valores permitidos se controlan en el servidor, mientras los metadatos variables quedan abiertos	*Schema Validation*: `https://www.mongodb.com/docs/manual/core/schema-validation/`
Índices por consulta	Cada índice compuesto responde a una consulta de la aplicación y sigue la guía *Equality-Sort-Range* (primero los campos de igualdad, luego el de orden y al final los de rango)	*Indexes*: `https://www.mongodb.com/docs/manual/indexes/`
Retención automática	La caducidad de la bitácora y de las notificaciones se implementa con índices TTL, sin procesos de limpieza manuales	*TTL Indexes*: `https://www.mongodb.com/docs/manual/core/index-ttl/`
Escrituras idempotentes	El relay escribe con *upsert* sobre un identificador determinista: reintentar el mismo evento no crea un duplicado	*updateOne* (opción `upsert`): `https://www.mongodb.com/docs/manual/reference/method/db.collection.updateOne/`
Durabilidad confirmada	Las escrituras de auditoría usan *write concern* `majority`, para que el relay marque un evento como procesado solo cuando está replicado	*Write Concern*: `https://www.mongodb.com/docs/manual/reference/write-concern/`
Inmutabilidad de la auditoría	Solo el usuario de base de datos del relay tiene permiso de escritura sobre `bitacora_eventos`; el usuario de la API solo puede leerla. Ninguno tiene permiso de borrado	*Security Checklist*: `https://www.mongodb.com/docs/manual/administration/security-checklist/`

Criterios de verificación planificados (se ejecutan en A18 y A21, con captura como Figura 6.7):

- Las tres colecciones existen con su validador e índices.
- Un documento que no cumple el validador (p. ej., un tipo de evento inexistente) es rechazado.
- Reintentar la publicación de un evento ya procesado no crea un segundo documento (prueba de integración del relay, A23).

- Las consultas de la bandeja y del historial de una orden usan su índice, verificado con `explain`.

Los cuatro criterios ya se cumplieron sobre el borrador del Anexo C en el ambiente local (Tabla 6.4); en A18 y A21 se repiten sobre staging.

[Insertar captura de la ejecución en staging como Figura 6.7, una vez completados A18 y A21.]

6.5 Integración entre datos estructurados y no estructurados

Estrategia: outbox transaccional (Richardson, 2018). Una transacción no puede abarcar PostgreSQL y MongoDB a la vez. Escribir en ambas bases desde la API ("doble escritura") arriesgaría perder eventos si el proceso falla entre una escritura y otra. Por eso:

1. El servicio escribe el cambio de negocio **y** su evento en `outbox_evento` en la **misma transacción PostgreSQL**: o se guardan ambos o ninguno.
2. El **relay** (cada 5 s) lee los eventos pendientes con `FOR UPDATE SKIP LOCKED` y los publica en `bitacora_eventos` y `notificaciones` con `_id` determinista (`evt-<id>`), mediante un *upsert* idempotente. Luego marca `procesado_en`.
3. Si MongoDB no está disponible, el evento queda pendiente y se reintenta (`intentos` + 1). Si un evento acumula 10 intentos fallidos, se genera una alerta en Sentry. La consistencia es **eventual con garantía de entrega al menos una vez**, y la idempotencia la convierte en *exactamente una vez* en efecto.

Dato / proceso	**Origen estructurado (PostgreSQL)**	**Origen no estructurado (MongoDB / R2)**	**Mecanismo de relación / integración**
Auditoría de cambios de estado	`outbox_evento` (escrito en la transacción de negocio)	`bitacora_eventos`	Relay outbox; `_id = evt-<outbox_evento.id>`; `orden_id` y `agregado.id` = UUID de PostgreSQL
Versiones de diseño	`aprobacion_diseno` (estado, versión, quién aprueba)	`evidencias_etapa` (tipo DISENO) + archivo en R2	`aprobacion_diseno.evidencia_ref` = `ObjectId` del documento; el documento guarda `object_key` y `sha256` del archivo
Evidencias de etapa	`etapa_orden.id`	`evidencias_etapa.etapa_orden_id`	Referencia lógica por UUID; la API valida que la etapa exista y pertenezca al actor antes de emitir la URL prefirmada
Notificaciones	Evento outbox (ETAPA_TERMINADA, ALERTA_INSUMO_CREADA)	`notificaciones`	El relay resuelve los destinatarios (responsable de la siguiente etapa o usuarios con rol Encargada) y crea un documento por destinatario

Dato / proceso	**Origen estructurado (PostgreSQL)**	**Origen no estructurado (MongoDB / R2)**	**Mecanismo de relación / integración**
Vista "Detalle de orden"	OT, etapas y requerimientos (consulta SQL)	Historial y evidencias (consulta Mongo por `orden_id`)	Composición en la capa de servicio (*API composition*): dos consultas ejecutadas en paralelo
Analítica futura de tiempos	`etapa_orden. iniciada_en / terminada_en`	`bitacora_ eventos` (eventos ETAPA_*)	Agregaciones en Mongo sin cargar la BD transaccional

Regla de propiedad del dato: PostgreSQL es la **fuente de verdad** de todo estado de negocio. MongoDB nunca se usa para decidir una transición; solo registra lo ocurrido y almacena contenido variable. Por eso la eventual demora del relay ($\leq$ 5 s) no afecta la consistencia operativa.

7 Seguridad y estándares de programación segura

El sistema gestiona principalmente información operativa, pero su indisponibilidad o manipulación detendría la producción, el mismo problema que se busca resolver. Además, guarda datos personales de trabajadores (nombre, usuario, actividad), sujetos a la Ley N.º 19.628 modificada por la Ley N.º 21.719 y al Código del Trabajo (sección 7.7). Los controles técnicos se alinean con OWASP ASVS 4.0.3 nivel 2 en autenticación (V2), sesión (V3) y control de acceso (V4), y con OWASP Top 10:2021. En el plano de gestión, se alinean con ISO/IEC 27001/27002, CIS Controls v8.1 y NIST CSF 2.0.

7.1 Roles y perfiles de acceso

Rol / perfil	Funciones permitidas	Información accesible	Restricciones
Administrador	Todo lo de la Encargada; crear, desactivar y restablecer usuarios; asignar roles; revocar sesiones; consultar la bitácora completa	Toda la información del sistema	MFA obligatorio; no puede eliminar registros de bitácora ni kardex; no se puede quitar a sí mismo el último rol Administrador (siempre existe al menos uno)
Encargada de Producción	Gestionar plantillas, tipos de trabajo e insumos; ver todas las OT; asignar/reasignar responsables; omitir o desbloquear etapas; detener, reanudar o anular OT; gestionar alertas; ver el panel; consultar la bitácora por OT; restablecer la clave de un operario	Todas las OT, etapas, insumos, alertas y panel; usuarios (solo nombre y área)	MFA obligatorio; sin gestión de usuarios ni roles (solo restablece claves de operarios)
Vendedor	Crear OT; editar sus OT en estado REGISTRADA; registrar la respuesta del cliente al diseño; marcar una OT como ENTREGADA	Todas las OT (lectura) y su avance; versiones de diseño	No avanza etapas de producción; no gestiona insumos ni plantillas
Diseñador	Ver y avanzar **sus** etapas de diseño; subir versiones de diseño; reportar faltantes	Sus etapas asignadas y las OT asociadas; stock (lectura)	No puede aprobar sus propios diseños (R6)
Operario	Ver y avanzar **sus** etapas; registrar consumo; reportar faltante o problema; adjuntar evidencia	Solo las etapas asignadas a él y los datos de producción de esas OT; stock (lectura)	No ve el panel global, la bitácora ni etapas de otros; no edita OT

Correspondencia con el levantamiento: el perfil “administrador” mencionado por el cliente equivale a Administrador más Encargada de Producción. El perfil “de respuesta” equivale a Operario y Diseñador. Vendedor surge del proceso confirmado *vendedor → encargada de producción → operarios*.

7.2 Política de autenticación

Control	Política	Fundamento
Identificador	`nombre_usuario` único (los operarios pueden no tener correo)	Usabilidad del taller
Contraseña	Mínimo **12 caracteres**, máximo 128; se permiten espacios y frases; **sin reglas de composición** ni expiración periódica; se rechazan claves de una lista de 100.000 contraseñas comprometidas y las que contienen el nombre de usuario	ASVS V2.1.1, V2.1.7; NIST SP 800-63B
Almacenamiento	**Argon2id** (m = 19 MiB, t = 2, p = 1) con sal aleatoria por usuario; parámetros en el hash (formato PHC) para recalibrar sin migración	OWASP Password Storage Cheat Sheet
MFA	**TOTP obligatorio** para Administrador y Encargada de Producción (cuentas privilegiadas), opcional para el resto. Secreto cifrado con AES-256-GCM; 10 códigos de recuperación de un solo uso, guardados con hash	ASVS V2.8; privilegio elevado
Fuerza bruta	5 intentos fallidos → bloqueo de cuenta por 15 min (progresivo: 30 y 60 min); *rate limit* de 10 solicitudes/min por IP en `/auth/*` (API) más regla WAF en Cloudflare	ASVS V2.2.1
Mensajes	Error genérico: “Usuario o contraseña incorrectos”. Se verifica contra un hash ficticio cuando el usuario no existe, para igualar los tiempos y evitar la enumeración de cuentas	OWASP Authentication Cheat Sheet
Token de acceso	JWT firmado con **EdDSA (Ed25519)**, vigencia **10 min**, con `sub`, `roles` y `jti`; se guarda **solo en memoria** de la PWA (nunca en `localStorage`) y se envía en `Authorization: Bearer`	Limita el impacto de un XSS
Token de refresco	Opaco de 256 bits, guardado con hash SHA-256 en `sesion_refresh`. Cookie `__Host-`, `HttpOnly; Secure; SameSite=Strict; Path=/`. **Rotación en cada uso**; si se reutiliza un token ya rotado, se revoca toda la `familia_id` (posible robo)	OWASP Session Management; RFC 9700
Duración de sesión	Inactividad: 2 h (operarios) / 30 min (Administrador, Encargada). Absoluta: 12 h, igual a una jornada	ASVS V3.3
Cierre de sesión	Revoca el refresh en el servidor; el Administrador puede revocar todas las sesiones de un usuario (p. ej., por pérdida del teléfono)	ASVS V3.3.1
Recuperación	Operarios: la Encargada o el Administrador generan una **clave temporal de un solo uso** (24 h) con cambio obligatorio (`debe_cambiar_clave`). Usuarios con correo: enlace de un solo uso, 30 min, con token guardado como hash	ASVS V2.5
Primer ingreso	Cambio de clave obligatorio y registro de MFA si el rol lo exige	—

Control	Política	Fundamento
Transporte	Solo HTTPS (TLS 1.2+, HSTS de 1 año con `preload`)	ASVS V9
Registro	Todo inicio de sesión exitoso o fallido, bloqueo y revocación genera un evento en la bitácora (sin la contraseña)	ASVS V7

7.3 Política de autorización

Modelo: RBAC con **permisos atómicos** definidos como código en `packages/shared/permisos.ts`, versionados y revisados en PR. Se aplica en tres capas:

1. **Denegación por defecto:** toda ruta de la API declara su permiso. Una ruta sin `requierePermiso()` falla una prueba automatizada de inventario de rutas.
2. **Autorización por propiedad (nivel de objeto):** el servicio verifica que el recurso pertenezca al actor, por ejemplo `etapa.responsable_id = actor.id`. Además, las consultas del Operario filtran en SQL por `responsable_id = $1`, de modo que nunca se cargan datos ajenos (prevención de IDOR, OWASP A01).
3. **Autorización por estado:** una acción solo procede si el estado lo permite, por ejemplo no se termina una etapa PENDIENTE ni se inicia una etapa de una OT DETENIDA (Fig. 4.7 y 4.8).

La PWA oculta los botones no permitidos usando el **mismo mapa**, pero esa ocultación es solo usabilidad: la autoridad es siempre la API.

Tabla 7.1 — Matriz de permisos (RBAC). ✓ = permitido · ◐ = permitido con restricción de propiedad o estado · — = denegado

Permiso	Administrador	Encargada de Producción	Vendedor	Diseñador	Operario
`usuarios.gestionar` (crear, desactivar, roles, restablecer)	✓	—	—	—	—
`usuarios.restablecer_operario`	✓	✓	—	—	—
`sesiones.revocar`	✓	—	—	—	—
`plantillas.gestionar`	✓	✓	—	—	—
`plantillas.ver`	✓	✓	✓	✓	—
`ordenes.crear`	✓	✓	✓	—	—
`ordenes.editar`	✓	✓	◐ propias en REGISTRADA	—	—
`ordenes.ver_todas`	✓	✓	✓	—	—
`ordenes.ver_asignadas`	✓	✓	✓	◐	◐
`ordenes.anular`	✓	✓	—	—	—

Permiso	Administrador	Encargada de Producción	Vendedor	Diseñador	Operario
`ordenes.detener` (detener y reanudar)	✓	✓	—	—	—
`ordenes.marcar_entregada`	✓	✓	✓	—	—
`etapas.asignar` / `etapas.omitir` / `etapas.desbloquear`	✓	✓	—	—	—
`etapas.avanzar_propias` (iniciar, terminar, bloquear)	◐	◐	—	◐	◐
`evidencias.adjuntar`	✓	✓	—	◐ propias	◐ propias
`diseno.subir`	✓	✓	—	✓	—
`diseno.resolver` (registrar respuesta del cliente)	✓	✓	✓	— (R6)	—
`insumos.ver`	✓	✓	✓	✓	✓
`insumos.gestionar` (catálogo, ingresos, ajustes)	✓	✓	—	—	—
`insumos.reportar_faltante`	✓	✓	—	✓	✓
`alertas.gestionar`	✓	✓	—	—	—
`panel.ver`	✓	✓	✓ (solo lectura)	—	—
`bitacora.ver`	✓	◐ por OT	—	—	—

Reglas complementarias:

- **Segregación de funciones:** quien sube un diseño no lo resuelve (R6, validado en la API y en la BD).
- **Cambios de rol:** solo un Administrador asigna roles; el cambio invalida el JWT vigente porque los roles se revalidan al refrescar ($\leq$ 10 min) y la acción queda en la bitácora.
- **Cuentas desactivadas:** `activo = FALSE` revoca de inmediato las sesiones de refresh.
- **Mínimo privilegio en infraestructura:** el usuario de BD de la aplicación (`sgp_app`) no puede alterar el esquema ni borrar el kardex o el outbox; el usuario de Mongo de la aplicación solo puede hacer `insert`/`find` en `bitacora_eventos`.

7.4 Amenazas y medidas de mitigación

La metodología combina **STRIDE** (Shostack, 2014) sobre los flujos de la Fig. 4.19 con las categorías de **OWASP Top 10:2021**. El riesgo se calcula como probabilidad $\times$ impacto (Alto / Medio / Bajo).

La Tabla 7.2 resume las 16 amenazas identificadas con su nivel de riesgo, el control principal y el caso de prueba que lo verifica. Ocho son de riesgo alto (AM-01, AM-03, AM-05 a AM-08, AM-10 y AM-12). La matriz completa, con el activo afectado, el escenario de riesgo, la probabilidad, el impacto y todas las medidas de mitigación, está en el **Anexo K (Tabla K.1)**.

Tabla 7.2 — Resumen del modelo de amenazas

ID	Amenaza (STRIDE / OWASP)	Riesgo	Control principal	Verificación
AM-01	Inyección SQL (T / A03)	**Alto**	Consultas 100 % parametrizadas en la capa *repository*	CP-24, SAST en CI
AM-02	Inyección NoSQL (T / A03)	Medio	Zod exige tipos primitivos; `sanitizeFilter` del driver	CP-24
AM-03	XSS almacenado (T / A03)	**Alto**	Escape por defecto de React y CSP estricta `script-src 'self'`	CP-25
AM-04	CSRF (S / A01)	Medio	Acciones de negocio con *Bearer*; cookie `SameSite=Strict` y verificación de `Origin`	CP-26
AM-05	Acceso no autorizado horizontal / IDOR (E / A01)	**Alto**	Verificación de propiedad en el servicio; respuesta 404 para recursos ajenos	CP-27
AM-06	Escalada vertical de privilegios (E / A01)	**Alto**	`requierePermiso()` en cada ruta con denegación por defecto	CP-28
AM-07	Fuerza bruta / relleno de credenciales (S / A07)	**Alto**	Bloqueo progresivo, *rate limit* y MFA en cuentas privilegiadas	CP-29
AM-08	Robo o reutilización de sesión (S / A07)	**Alto**	Refresh `HttpOnly` con rotación y detección de reutilización	CP-30
AM-09	Subida de archivo malicioso (T / A04)	Medio	URL prefirmada con tipo y tamaño firmados; *bucket* privado	CP-31
AM-10	Exposición de secretos (I / A05)	**Alto**	Secretos solo en el gestor del proveedor; *secret scanning* y *push protection*	CP-32, revisión de PR
AM-11	Configuración insegura / cabeceras (A05)	Medio	`helmet` con CSP, HSTS y `frame-ancestors 'none'`	CP-32
AM-12	Componentes vulnerables (A06)	**Alto**	`npm audit` y Dependabot en CI	CI (cada PR)
AM-13	Repudio de acciones (R / A09)	Medio	Outbox en la misma transacción; bitácora inmutable	CP-11
AM-14	Denegación de servicio (D)	Medio	Protección DDoS de Cloudflare; paginación obligatoria y `statement_timeout`	CP-20, CP-23
AM-15	Pérdida de datos (D / integridad)	Medio	PITR de Neon y respaldo diario cifrado; restauración ensayada	Ensayo de restauración en A29
AM-16	Exposición de datos personales (I / Ley 19.628/21.719)	Bajo	Minimización y logs sin datos sensibles	Revisión de logs (CP-32)

7.5 Estándares de codificación segura

Los estándares se derivan de OWASP Top 10:2021, ASVS 4.0.3 y la OWASP Cheat Sheet Series, y son **exigibles por la Definition of Done**. Cada PR incluye la lista de cotejo de seguridad (Anexo E) y no se fusiona si el pipeline detecta un hallazgo High.

Área	Estándar / regla definida	Aplicación en el proyecto	Control automático
Validación de entradas	Toda entrada (body, params, query, headers) se valida con un esquema Zod `.strict()` con listas blancas, longitudes máximas y tipos; se rechaza lo desconocido (A03)	Middleware de validación en cada ruta; los mismos esquemas en el formulario de la PWA	Prueba que verifica que cada ruta declare un esquema
Codificación de salida	Prohibido `innerHTML`/`dangerouslySet InnerHTML`; el texto de usuario se renderiza como texto; CSP sin `unsafe-inline`	React JSX; `helmet.content SecurityPolicy`	ESLint `react/no-danger` = error
Autenticación y autorización	Ver 7.2–7.3; denegación por defecto; verificación de propiedad en el servicio	Middleware de permisos y capa de servicio (patrón 3 de 7.6)	Prueba de inventario de rutas; CP-27, CP-28
Gestión de errores	Manejador centralizado; respuesta genérica con `request_id`; el detalle va solo al log; nunca se exponen *stack traces*, SQL ni versiones	Manejador de errores único de la API	CP-24 (sin fugas)
Logging y auditoría	Logs JSON estructurados; sin contraseñas, tokens ni cuerpos completos (`redact`); eventos de seguridad en la bitácora	`pino` + outbox	Revisión de PR
Protección / cifrado de información	TLS 1.2+ extremo a extremo (`sslmode=verify-full`); Argon2id para claves; AES-256-GCM para el secreto TOTP; SHA-256 para tokens de refresco y recuperación	`crypto` nativo de Node.js	Pruebas unitarias del módulo de cifrado
Gestión de secretos y configuración	Secretos solo en variables de entorno del proveedor; configuración validada con Zod al arrancar (la app falla si falta un secreto); rotación semestral de las claves JWT (`kid`)	Módulo de configuración validado al arranque	GitHub *secret scanning* + *push protection*
Acceso a datos	Solo consultas parametrizadas; la capa *repository* es la única con SQL; transacciones explícitas; usuario de BD con mínimo privilegio; `statement_timeout`	Driver `pg` con parámetros posicionales	Regla Semgrep `no-sql-string-concat`
Dependencias	Versiones fijadas con lockfile; auditoría en cada PR; Dependabot semanal; imágenes base oficiales *slim*	`package-lock.json`, `Dockerfile`	`npm audit`, Dependabot
Archivos	Lista blanca de tipos y tamaños; *bucket* privado; URL prefirmadas de corta vida	Servicio de evidencias (patrón 8 de 7.6)	CP-31
Revisión de código	Todo cambio pasa por PR revisado por el otro integrante con la lista de cotejo de seguridad	Protección de rama `main`	Regla de GitHub

7.6 Evidencia de aplicación de estándares seguros

En esta etapa la evidencia es de **diseño**: para cada estándar de 7.5 se define el patrón obligatorio, la amenaza que mitiga, la documentación que el equipo seguirá para implementarlo y la prueba que demostrará que se aplicó. No se incluye código. Los patrones se implementan primero en los Sprints 1–2 (A12, A16, A18) y desde ahí se exigen en el resto de los módulos mediante la Definition of Done y la lista de cotejo del Anexo E. El **Anexo G** contiene un borrador referencial en TypeScript de cada patrón, que sirve como punto de partida para la construcción.

La Tabla 7.3 resume los ocho patrones obligatorios. El **Anexo K (Tabla K.2)** describe cada patrón en detalle y enlaza la documentación oficial que el equipo seguirá para implementarlo.

Tabla 7.3 — Patrones de codificación segura

#	Patrón obligatorio	Amenaza que mitiga	Verificación
1	Consultas parametrizadas	Inyección SQL (A03); IDOR (A01)	Regla Semgrep contra la concatenación; CP-24
2	Validación estricta de entradas	Inyección (A03); asignación masiva	Prueba de inventario: toda ruta declara un esquema; CP-06, CP-24
3	Autorización en tres niveles: rol, propiedad y estado	Control de acceso roto (A01); IDOR; escalada de privilegios	CP-27, CP-28
4	Almacenamiento de contraseñas y login uniforme	Robo de credenciales; enumeración de usuarios; fuerza bruta (A07)	CP-07, CP-29; Anexo E (ítems 1–3)
5	Sesión con dos tokens	Secuestro de sesión; XSS que roba tokens; reutilización de tokens	CP-09, CP-30
6	Cabeceras, CORS, CSRF y límites de tasa	XSS (A03); CSRF; abuso y fuerza bruta; configuración insegura (A05)	CP-25, CP-26, CP-29, CP-32
7	Manejo centralizado de errores	Fuga de información (A05, A09)	CP-24 (sin fugas); revisión de logs
8	Subida segura de archivos	Archivos maliciosos; IDOR; agotamiento de almacenamiento	CP-31

Cómo se demostrará la aplicación. Durante la construcción, la evidencia de cada patrón será: (1) el PR que lo implementa, revisado por el otro integrante con la lista de cotejo del Anexo E; (2) el resultado del control automático del pipeline (Semgrep, ESLint, `npm audit`, *secret scanning*); y (3) el reporte del caso de prueba asociado, que se adjunta en el Anexo F.

7.7 Marco normativo chileno y estándares de referencia

El SGP trata datos personales de los trabajadores de Kotoduo: identificación (nombre, usuario, correo), credenciales, área, asignaciones y registro de actividad (inicio y término de etapas, bitácora y fotografías de evidencia). No trata datos sensibles (salud, biometría, afiliación sindical) ni datos de contacto de clientes (RNF-09). **Kotoduo es el responsable del tratamiento.** El equipo de desarrollo, durante el proyecto, y los proveedores cloud actúan como **encargados** que tratan los datos por cuenta de Kotoduo.

Las modificaciones de la Ley N.º 21.719 a la Ley N.º 19.628 entran en vigencia el 1 de diciembre de 2026. El piloto (H2) operará bajo la ley modificada, por lo que el diseño se ajusta desde ya a ella y no solo al texto anterior. El contenido legal de esta sección es una interpretación técnica para orientar el diseño; Kotoduo debe validarla con asesoría jurídica antes del paso a producción.

Tabla 7.4 — Normativa chilena aplicable.

Norma	Alcance en el SGP	Obligación relevante	Medida en el diseño	Evidencia
Constitución Política, art. 19 N.º 4 (reformado por la Ley N.º 21.096, 2018)	Todo dato personal tratado	Consagra el derecho a la protección de los datos personales	Fundamento del resto de las medidas	—
Ley N.º 19.628 sobre protección de datos personales, modificada por la Ley N.º 21.719 (2024)	Datos de los trabajadores usuarios	Principios de licitud, finalidad, proporcionalidad, calidad, responsabilidad, seguridad, transparencia y confidencialidad; deber de información; derechos de acceso, rectificación, supresión, oposición, portabilidad y bloqueo; medidas de seguridad apropiadas al riesgo; notificación de vulneraciones a la Agencia de Protección de Datos Personales; reglas para encargados y transferencias internacionales. Las infracciones gravísimas se sancionan con multas de hasta 20.000 UTM	Minimización (RNF-09); finalidad declarada; controles de 7.1–7.5; aviso de privacidad; atención de derechos; retención definida; procedimiento de incidentes (decisiones D1–D7)	AM-16, CP-32, Tabla 7.6
Código del Trabajo, arts. 5, 153, 154 y 154 bis	Registro de actividad de los trabajadores (bitácora, tiempos de etapa, evidencias)	Las facultades del empleador tienen como límite los derechos fundamentales del trabajador, en especial su intimidad, vida privada y honra. Las medidas de control deben constar en el Reglamento Interno, ser generales, impersonales y respetuosas de la dignidad. El empleador debe mantener reserva de los datos privados del trabajador. Una vulneración puede reclamarse por el procedimiento de tutela laboral (art. 485)	El registro tiene por finalidad la trazabilidad de la OT, no la vigilancia; sin geolocalización, cámara permanente ni ranking de productividad individual; cada operario ve solo lo propio	D1, D8; guía rápida (A30)

Norma	Alcance en el SGP	Obligación relevante	Medida en el diseño	Evidencia
Ley N.º 21.459 (2022), delitos informáticos	Pruebas de seguridad y pentest (A27)	Sanciona el acceso a un sistema sin autorización o excediendo la autorización otorgada	Pentest solo con autorización escrita de Kotoduo, alcance definido y contra staging; se respetan las políticas de pruebas de los proveedores	D9; acta de autorización (A27)
Ley N.º 21.663 (2024), Ley Marco de Ciberseguridad	Kotoduo no es prestador de servicios esenciales ni operador de importancia vital	No impone obligaciones directas al proyecto	Se adoptan voluntariamente sus principios de gestión de riesgos y reporte de incidentes, y las recomendaciones de la Agencia Nacional de Ciberseguridad (ANCI)	D6

Decisiones de diseño derivadas de la normativa:

1. **D1. Finalidad y proporcionalidad del registro de actividad.** La bitácora y los tiempos de etapa existen para la trazabilidad de la OT (RF-12) y el análisis agregado por tipo de trabajo. El panel agrega por OT, no por trabajador, y el sistema no genera rankings ni reportes de productividad individual. Si Kotoduo decide usar estos registros como medida de control laboral, debe incorporarlos a su Reglamento Interno e informarlo a los trabajadores (art. 154).
2. **D2. Deber de información.** Un aviso de privacidad, accesible desde el login y el perfil, indica qué datos se tratan, con qué finalidad, por cuánto tiempo, quién es el responsable y cómo ejercer los derechos. Se construye junto con HU-11.
3. **D3. Derechos de los titulares.** Se atienden con funciones existentes: el perfil permite la rectificación; la consulta de bitácora filtrada por usuario (HU-12) permite entregar al titular el registro de su actividad (acceso y portabilidad); la baja lógica de la cuenta implementa la supresión o el bloqueo. Los registros de auditoría se conservan durante su periodo de retención porque su integridad es necesaria para la trazabilidad.
4. **D4. Retención.** Bitácora: 24 meses (RNF-05, TTL). Notificaciones: 90 días (TTL). Al vencer la retención, los datos identificatorios de usuarios desactivados se seudonimizan; durante el piloto es un procedimiento manual del Administrador y su automatización queda para una versión posterior.
5. **D5. Encargados y transferencia internacional.** Render, Neon, MongoDB Atlas, Cloudflare y Sentry procesan datos fuera de Chile. Antes de cargar datos reales se aceptan sus acuerdos de tratamiento de datos (DPA) con cláusulas contractuales tipo, se elige la región más cercana disponible y la transferencia se documenta en el registro de actividades de tratamiento. A Sentry no se envían datos personales (depuración de eventos).

6. **D6. Vulneraciones de seguridad.** Procedimiento en cinco pasos: detectar (Sentry, UptimeRobot, alerta de reutilización de refresh), contener (revocar sesiones, rotar claves `kid`), evaluar el alcance, notificar a la Agencia sin dilación indebida y a los titulares cuando la ley lo exija, y registrar el incidente con sus lecciones aprendidas. Kotoduo lo ejecuta con apoyo del equipo.
7. **D7. Datos de prueba.** Desarrollo y staging usan solo datos semilla ficticios. La carga inicial desde el Excel actual (A29) se hace solo en producción y únicamente con las columnas necesarias.
8. **D8. Fotografías de evidencia.** Registran el trabajo, no a las personas. La guía rápida lo indica y el formulario de evidencia lo recuerda.
9. **D9. Pentest autorizado.** Antes de A27 se firma un acta con Kotoduo que define el alcance, el ambiente (staging), las fechas y las técnicas permitidas.

Pertinencia de los estándares. La Ley N.º 21.719 exige medidas de seguridad apropiadas según el riesgo, el estado de la técnica y el costo de aplicación, pero no prescribe un estándar. Los estándares internacionales permiten demostrar esa diligencia (principio de responsabilidad) y sirven de base para el modelo de prevención de infracciones que la ley contempla como voluntario. El INN adoptó la norma ISO/IEC 27001 en Chile como NCh-ISO 27001. No se busca una certificación, cuyo costo no se justifica para una pyme, sino usar estos estándares como catálogo de controles verificables.

Tabla 7.5 — Estándares de referencia y su relación con la normativa chilena.

Estándar	Uso en el proyecto	Relación con la normativa chilena	Aplicación en el SGP
ISO/IEC 27001:2022 (NCh-ISO 27001)	Marco de gestión de la seguridad de la información y del riesgo	Medidas de seguridad apropiadas al riesgo; principio de responsabilidad	Modelo de amenazas con probabilidad e impacto (7.4); controles exigibles por la DoD
ISO/IEC 27002:2022	Catálogo de controles	Seguridad y confidencialidad de los datos personales	5.15 control de acceso (7.1–7.3); 5.34 privacidad y protección de datos personales (D1–D5); 8.5 autenticación segura (7.2); 8.13 respaldo (AM-15); 8.15 registro de eventos (bitácora); 8.25 y 8.28 desarrollo y codificación seguros (7.5); 8.29 pruebas de seguridad (A27); 8.33 información de prueba (D7)
ISO/IEC 27701	Extensión de privacidad de ISO/IEC 27001 para responsables y encargados	Deber de información, derechos de los titulares y obligaciones del encargado	D2, D3 y D5
ISO/IEC 29100:2011	Marco de principios de privacidad	Equivale a los principios de finalidad, proporcionalidad y transparencia de la ley	RNF-09 y D1
ISO/IEC 27018:2019	Protección de datos personales en nubes públicas que actúan como encargadas	Selección de encargados y transferencia internacional	Criterio para evaluar a los proveedores (D5)

Estándar	Uso en el proyecto	Relación con la normativa chilena	Aplicación en el SGP
ISO/IEC 27035-1:2023	Gestión de incidentes de seguridad	Notificación de vulneraciones a la Agencia y a los titulares	Procedimiento D6
CIS Critical Security Controls v8.1 (IG1 + control 16)	Salvaguardas priorizadas para organizaciones pequeñas sin equipo de TI	Traduce las “medidas apropiadas” en acciones verificables para una pyme	3 protección de datos (inventario, retención y eliminación segura: D4); 5 y 6 gestión de cuentas y accesos con MFA (7.2); 8 registros de auditoría (bitácora); 11 recuperación de datos (AM-15); 14 concientización (A30); 16 seguridad del software (7.5, 8.6); 17 respuesta a incidentes (D6)
NIST Cybersecurity Framework 2.0	Estructura de cumplimiento en seis funciones	La función Gobernar (GV.OC-03) exige conocer y gestionar los requisitos legales, incluidos los de privacidad	Gobernar: esta sección; Proteger: 7.1–7.5; Detectar: Sentry y UptimeRobot; Responder: D6; Recuperar: PITR y respaldos
NIST Privacy Framework 1.0	Gestión del riesgo de privacidad	Derechos de los titulares, transparencia y minimización	D1–D4
NIST SP 800-218 (SSDF) 1.1	Prácticas de desarrollo seguro	Medidas de seguridad desde el diseño	DoD, SAST, revisión de PR (incluido el código generado con IA)
NIST SP 800-115	Metodología de pruebas técnicas y pentest	Pruebas autorizadas (Ley N.º 21.459)	Planificación, ejecución e informe de A27
NIST SP 800-61 Rev. 3	Respuesta a incidentes alineada con CSF 2.0	Notificación de vulneraciones	Procedimiento D6

Tabla 7.6 — Cumplimiento normativo en la planificación. Las tareas normativas se integran en actividades existentes del plan (Anexo H, Tabla H.1), sin agregar días al cronograma.

Actividad	Tarea normativa	Entregable
A01 (D1)	Validar con Kotoduo su rol de responsable, la finalidad del registro de actividad (D1) y si incluirá las medidas de control en su Reglamento Interno	Acta de validación
A07 (D8–D9)	Elaborar la matriz normativa y el registro de actividades de tratamiento	Sección 7.7
A11 (D12)	Cargar solo datos semilla ficticios en desarrollo y staging (D7)	Script de semillas
A12, A15 (Sprint 1)	Construir el aviso de privacidad en el login y el perfil (D2)	Pantallas P-01 y P-12
A27 (D28–D29)	Firmar el acta de autorización del pentest (D9); aplicar NIST SP 800-115; verificar que los logs no contengan datos personales	Acta + informe CP-24 a CP-32

Actividad	**Tarea normativa**	**Entregable**
A29 (D30)	Aceptar los DPA de los proveedores antes de cargar datos reales; cargar desde el Excel solo las columnas necesarias (D5, D7)	Registro de transferencias
A30 (D30)	Informar a los trabajadores sobre el tratamiento de sus datos y las reglas para las fotografías de evidencia (D2, D8)	Guía rápida + registro de asistencia
A31 (D30)	Entregar a Kotoduo los procedimientos de incidentes (D6), de atención de derechos (D3) y de retención (D4)	Anexo de procedimientos

8 Plan de pruebas

8.1 Objetivo y alcance del plan de pruebas

Objetivo: demostrar con evidencia reproducible que SGP-Kotoduo cumple RF-01–RF-12 y RNF-01–RNF-09. En particular, que un trabajador puede conocer y ejecutar su siguiente actividad sin intervención de una persona encargada (criterio de éxito del cliente), y que los controles de la sección 7 mitigan las amenazas AM-01–AM-16.

Aspecto	Definición
Incluido	Módulos M1–M8 de la API, PWA (P-01 a P-12), triggers y vista de PostgreSQL, validadores de MongoDB, relay outbox, URL prefirmadas, cabeceras y configuración de despliegue
Excluido	Infraestructura interna de los proveedores (Cloudflare, Render, Neon, Atlas), pruebas de carga de tipo estrés destructivo en producción, compatibilidad con navegadores sin soporte de Service Worker, funciones fuera de alcance (precios, contactos, Softland)
Niveles	Unitarias → integración → sistema/E2E → aceptación (usabilidad con usuarios reales); no funcionales: rendimiento y seguridad
Ambientes	Unitarias: local y CI. Integración: CI con Testcontainers (PostgreSQL 16 + MongoDB). E2E, rendimiento y seguridad: **staging**, con datos sintéticos (nunca en producción)
Datos de prueba	Semillas del script DDL (Anexo B) + generador de 5.000 OT sintéticas para volumen; usuarios ficticios por rol
Criterios de inicio	HU en estado “Hecho” según la DoD del Sprint; build en verde; staging desplegado con la migración vigente
Criterios de término	100 % de los CP ejecutados; 0 defectos críticos o altos abiertos; cobertura de líneas $\geq$ 70 % en `services`; umbrales de rendimiento cumplidos; ZAP sin alertas High/Medium; SUS $\geq$ 70
Gestión de defectos	Issue en GitHub con severidad (Crítica / Alta / Media / Baja), CP asociado y evidencia; un defecto Crítico o Alto bloquea el *merge* y el despliegue
Responsables	Diseño y E2E/usabilidad: Integrante 2. Rendimiento y seguridad: Integrante 1. Unitarias: autor del código, revisadas por el otro integrante

8.2 Trazabilidad entre requerimientos, casos de uso y casos de prueba

HU / Req.	Caso de uso / escenario	Caso(s) de prueba	Tipo de prueba	Cobertura esperada
HU-11 / RF-01	CU-01, CU-02	CP-07, CP-08, CP-09, CP-10, CP-28, CP-29, CP-30	Unitaria, integración, seguridad	Política de claves, login, MFA, bloqueo, sesión, permisos
HU-07 / RF-02	CU-03	CP-03, CP-28	Unitaria, seguridad	Plantilla ordenada; solo Encargada/Admin la modifican
HU-05 / RF-03, RF-04	CU-04	CP-04, CP-06, CP-11	Unitaria, integración	Validación, generación de etapas, fecha estimada, evento de bitácora

HU / Req.	Caso de uso / escenario	Caso(s) de prueba	Tipo de prueba	Cobertura esperada
HU-08 / RF-04	CU-12 (y CU-04, alternativo 3b)	CP-01, CP-11, CP-12, CP-27	Unitaria, integración, seguridad	Asignación, omisión y desbloqueo de etapas; aislamiento por responsable
HU-01 / RF-05	CU-05	CP-20, CP-33, CP-34	Rendimiento, usabilidad, E2E	Mi Trabajo rápido, comprensible y adaptable
HU-02 / RF-06	CU-06	CP-01, CP-02, CP-12, CP-13, CP-19, CP-21, CP-27	Unitaria, integración, rendimiento, seguridad	Máquina de estados, avance, cierre de OT, concurrencia, IDOR
HU-04 / RF-07	CU-06 (extensión)	CP-17, CP-25, CP-31	Integración, seguridad	Evidencia almacenada, XSS y archivos maliciosos
HU-06 / RF-08	CU-07	CP-16	Integración	Aprobación, rechazo y segregación de funciones
HU-09 / RF-09, RF-10	CU-08, CU-09	CP-05, CP-14, CP-15	Unitaria, integración	Umbral, alerta automática única, alerta manual
HU-03 / RF-10	CU-09	CP-15, CP-33	Integración, usabilidad	Reporte desde la etapa
HU-10 / RF-11	CU-10, CU-13, CU-14	CP-01, CP-02, CP-18, CP-23	Unitaria, integración, rendimiento	Atrasadas y en riesgo correctas; detener, reanudar y anular según la Fig. 4.8; panel con volumen
HU-12 / RF-12	CU-11	CP-11, CP-13	Integración	100 % de cambios en la bitácora
RNF-01	—	CP-33, CP-35, CP-36	Usabilidad, accesibilidad	≤ 3 toques, SUS ≥ 70, WCAG AA
RNF-02	—	CP-22, CP-34	E2E	Instalable y responsive
RNF-03	—	CP-20, CP-21, CP-22, CP-23	Rendimiento	p95 y LCP dentro de los umbrales
RNF-04	—	CP-24 a CP-32	Seguridad	AM-01 a AM-12 verificadas

8.3 Pruebas unitarias

Las pruebas unitarias aíslan la lógica de dominio y los servicios usando repositorios simulados. Se ejecutan con **Vitest** en cada *commit* y en CI, con una meta de cobertura $\geq$ 70 % en `services` y del 100 % en las funciones de transición y permisos. Unidades: `puede Transicionar` (máquina de estados), `generarEtapas`, `calcularFechaEstimada` (jornada de 8 h, días hábiles), `Insumo.estaBajoMinimo`, esquemas Zod, `validarPoliticaClave` y `PERMISOS_POR_ROL`. Casos: CP-01 a CP-08 (Anexo L, Tabla L.1).

8.4 Pruebas de integración

Las pruebas de integración verifican la API completa (rutas $\rightarrow$ servicios $\rightarrow$ BD real) con **Supertest** contra contenedores efímeros de PostgreSQL 16 y MongoDB levantados por **Testcontainers**. Cada suite aplica las migraciones y semillas y se limpia al terminar. También validan el contrato real con la base de datos (triggers, CHECK, FK), el relay outbox $\rightarrow$ MongoDB y las URL prefirmadas (contra un emulador S3, MinIO). Casos: CP-09 a CP-19 (Anexo L, Tabla L.2). Se complementan con colecciones **Postman** para

exploración manual de la API en staging.

8.5 Pruebas de rendimiento

Las pruebas de rendimiento usan **k6** en staging, con una carga derivada del contexto: ~25 trabajadores, picos al inicio de la jornada y un escenario de 40 usuarios virtuales (VU) concurrentes (1,6× la dotación) como margen. Métricas: latencia p95/p99, tasa de error y *throughput*. En el front-end se mide **Lighthouse** (LCP, TBT) con la limitación "4G lenta". Para volumen, se usa una BD con 5.000 OT y 30.000 etapas sintéticas, y `EXPLAIN ANALYZE` para confirmar el uso de índices. Casos: CP-20 a CP-23 (Anexo L, Tabla L.3). Los umbrales son *thresholds* de k6: la ejecución falla automáticamente si no se cumplen.

8.6 Pruebas de seguridad

Las pruebas de seguridad derivan directamente de la Tabla 7.2:

- **DAST** con OWASP ZAP (*baseline* en cada despliegue a staging y escaneo activo antes de producción).
- **SAST** con Semgrep y ESLint security en cada PR.
- Auditoría de dependencias (`npm audit`, Dependabot).
- **Pruebas manuales dirigidas** con Postman: IDOR, escalada, CSRF y reutilización de tokens.
- **Lista de cotejo ASVS nivel 2** (Anexo E).

Casos: CP-24 a CP-32 (Anexo L, Tabla L.4).

8.7 Pruebas de usabilidad

Las pruebas de usabilidad se hacen con usuarios reales de Kotoduo (mínimo 5, uno o más por rol) sobre staging, en sus propios teléfonos, con el protocolo de la sección 5.5 (tareas T1–T4, pensamiento en voz alta y cuestionario SUS). Se complementan con pruebas automatizadas de accesibilidad (axe-core en Playwright) y capturas *responsive* en 4 *viewports*. Casos: CP-33 a CP-36 (Anexo L, Tabla L.5). El indicador clave de éxito del cliente es el **porcentaje de operarios que identifican su siguiente actividad sin ayuda** (meta $\geq 80\,\%$).

8.8 Diseño, ejecución y resultados de casos de prueba

Los 36 casos de prueba comparten un mismo formato: identificador, historia de usuario o amenaza que verifican, descripción, precondición, pasos y datos, resultado esperado, resultado obtenido y estado. La Tabla 8.1 los resume por tipo; las especificaciones completas están en el **Anexo L (Tablas L.1 a L.5)**.

Tabla 8.1 — Casos de prueba por tipo

Tipo	Casos	Qué verifican	Herramientas	Detalle
Unitarias	CP-01 a CP-08 (8)	Máquinas de estados, generación de etapas, fecha estimada, umbral de stock, validación de la OT, política de claves y mapa de permisos	Vitest	Tabla L.1
Integración	CP-09 a CP-19 (11)	Login y MFA, registro de OT con su evento de bitácora, avance y cierre de etapas, alertas de insumo, aprobación de diseño, evidencias, panel y concurrencia	Supertest + Testcontainers	Tabla L.2
Rendimiento	CP-20 a CP-23 (4)	Latencia de Mi Trabajo y del panel con volumen, escrituras concurrentes y carga inicial de la PWA	k6, Lighthouse	Tabla L.3
Seguridad	CP-24 a CP-32 (9)	Inyección, XSS, CSRF, IDOR, escalada, fuerza bruta, reutilización de sesión, archivos y configuración (AM-01 a AM-12)	ZAP, Postman, Semgrep, lista de cotejo (Anexo E)	Tabla L.4
Usabilidad y adaptabilidad	CP-33 a CP-36 (4)	Autonomía del operario, diseño responsive, satisfacción (SUS) y accesibilidad	Sesiones moderadas, Playwright, axe-core	Tabla L.5

Registro de resultados. La columna "Resultado obtenido" de las tablas del Anexo L se completa al ejecutar cada caso en su Sprint (actividades A23, A26, A27 y A28). Los reportes completos (Vitest, k6, ZAP y Lighthouse) se adjuntan en el Anexo F. Si el resultado obtenido difiere del esperado, se registra la incidencia (ID de issue, severidad y acción tomada) y el estado pasa a "Rechazado" hasta su corrección.

Tabla 8.2 — Resumen de ejecución (a completar)

Tipo	Casos	Aprobados	Rechazados	Pendientes
Unitarias	8	☐	☐	8
Integración	11	☐	☐	11
Rendimiento	4	☐	☐	4
Seguridad	9	☐	☐	9
Usabilidad / adaptabilidad	4	☐	☐	4
Total	**36**			**36**

8.9 Herramientas manuales y automatizadas

La estrategia equilibra **automatización** para todo lo repetible y regresivo (se ejecuta en cada PR, sin costo marginal) con **pruebas manuales** donde el juicio humano aporta lo que una herramienta no puede: comprensión del usuario real, exploración creativa de la seguridad y validación del negocio. De los 36 casos, 27 son automatizables en CI y 9 requieren ejecución manual o mixta (CP-26 a CP-30 en su fase exploratoria, CP-32 con su lista de cotejo, CP-33 y CP-35).

Herramienta / técnica	**Manual / automatizada**	**Pruebas que apoya**	**Propósito**	**Evidencia**
Vitest (+ cobertura v8)	Automatizada	Unitarias CP-01 a CP-08	Verificar la lógica de dominio y los permisos en cada *commit*	Reporte JUnit y de cobertura en CI (Anexo F)
Supertest + Testcontainers	Automatizada	Integración CP-09 a CP-19; parte de CP-24, CP-27, CP-28	API contra PostgreSQL y MongoDB reales y efímeros	Log de GitHub Actions
Playwright (+ axe-core)	Automatizada	E2E, CP-25, CP-34, CP-36	Flujos en navegador real (Chromium, WebKit = Safari iOS), capturas responsive, accesibilidad	Traza HTML de Playwright y capturas
k6	Automatizada	Rendimiento CP-20, CP-21, CP-23	Carga con umbrales que hacen fallar la ejecución si no se cumplen	Resumen JSON/HTML de k6
Lighthouse CI	Automatizada	CP-22	Métricas web y PWA en cada despliegue a staging	Reporte Lighthouse
OWASP ZAP (*baseline* y escaneo activo)	Automatizada (DAST)	CP-24, CP-32	Detectar inyecciones, cabeceras faltantes y configuración insegura	Reporte HTML de ZAP
Semgrep + ESLint (`eslint-plugin-security`, `react/no-danger`)	Automatizada (SAST)	AM-01, AM-03, AM-10	Bloquear patrones inseguros antes del *merge*	Comentarios en el PR
`npm audit` + Dependabot + GitHub *secret scanning*	Automatizada	CP-32, AM-12, AM-10	Dependencias vulnerables y secretos filtrados	Alertas de GitHub
Postman (colecciones por rol)	Manual (exploratoria)	CP-26 a CP-30; exploración de la API	Pruebas dirigidas de autorización, CSRF y sesión que requieren manipular tokens y orígenes	Colección exportada + capturas
Lista de cotejo OWASP ASVS L2 (Anexo E)	Manual	CP-32; revisión de PR	Verificar controles no detectables por herramientas (política de sesión, recuperación, logs)	Lista de cotejo firmada por el revisor
Sesiones de usabilidad moderadas (pensamiento en voz alta)	Manual	CP-33, rondas de feedback 5.5	Validar autonomía y comprensión con trabajadores reales	Registro de tareas, tiempos y observaciones
Cuestionario SUS	Manual	CP-35	Medir la satisfacción con un instrumento estandarizado	Planilla de puntajes
Revisión de código por pares	Manual	Todas (preventiva)	Detectar defectos lógicos y de seguridad antes de la integración	Aprobación del PR en GitHub

Herramienta / técnica	**Manual / automatizada**	**Pruebas que apoya**	**Propósito**	**Evidencia**
Ensayo de restauración de respaldo	Manual	AM-15	Verificar RPO/RTO restaurando el `pg_dump` en una rama de Neon	Acta con tiempos medidos

9 Conclusiones

1. **Coherencia diagnóstico–diseño.** El problema central confirmado por Kotoduo, que los trabajadores no saben qué sigue cuando faltan las personas encargadas, se resuelve con tres decisiones trazables:
   - plantillas de proceso que externalizan el conocimiento experto (M2);
   - la máquina de estados de la etapa con habilitación y notificación automática de la siguiente (M4);
   - una vista personal *mobile-first* que responde "qué hago ahora y qué sigue".

   La gestión reactiva de insumos se ataca en la causa, porque las alertas se generan al planificar y al consumir, y no al descubrir el faltante.
2. **Arquitectura proporcional.** Un monolito modular con PWA y persistencia políglota equilibra simplicidad operativa para un equipo de dos personas, bajo costo (≈ US$ 7–10/mes) y capacidad de evolución. Las cinco vistas 4+1 describen la misma solución con nombres consistentes y se validan con los casos de uso CU-01 a CU-14.
3. **Integridad en varias capas.** Las reglas críticas (transiciones, stock no negativo, alertas únicas, segregación en la aprobación de diseños) se aplican en el dominio **y** en PostgreSQL. La integración con MongoDB mediante outbox transaccional evita la pérdida de eventos de auditoría sin introducir transacciones distribuidas. Los borradores del DDL y de los validadores documentales se ejecutaron sobre los motores reales y cumplieron los 16 criterios verificables en la base de datos (Tabla 6.4).
4. **Seguridad basada en riesgo.** Se identificaron 16 amenazas, de las cuales 8 son de riesgo alto (inyección, XSS, IDOR, escalada, fuerza bruta, robo de sesión, secretos, dependencias). Cada una tiene una mitigación concreta, un estándar de codificación exigible por la DoD y un caso de prueba que la verifica.
5. **Riesgos y aspectos pendientes para la implementación:**
   - validar con la contraparte los supuestos marcados como "S" (A01), en especial el nivel de detalle de insumos y quién registra la aceptación del diseño;
   - confirmar la disponibilidad de teléfonos y conectividad en el taller y en terreno (instalaciones);
   - ejecutar la ronda de usabilidad con trabajadores reales (FB-07 a FB-11);
   - ejecutar y documentar los 36 casos de prueba en los Sprints 1–4;
   - definir con Kotoduo quién financia el costo mensual mínimo del hosting.

10 Referencias

Aqua Security. (s.f.). *Trivy documentation*. `https://trivy.dev/`

Brooke, J. (1996). SUS: A "quick and dirty" usability scale. En P. W. Jordan, B. Thomas, B. A. Weerdmeester & I. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor & Francis.

Biblioteca del Congreso Nacional de Chile. (1999). *Ley N.º 19.628 sobre protección de la vida privada*. `https://www.bcn.cl/leychile/navegar?idNorma=141599`

Biblioteca del Congreso Nacional de Chile. (2002). *Código del Trabajo (DFL N.º 1, de 2002, del Ministerio del Trabajo y Previsión Social, texto refundido)*. `https://www.bcn.cl/leychile`

Biblioteca del Congreso Nacional de Chile. (2018). *Ley N.º 21.096 que consagra el derecho a protección de los datos personales*. https://www.bcn.cl/leychile

Biblioteca del Congreso Nacional de Chile. (2022). *Ley N.º 21.459 que establece normas sobre delitos informáticos*. https://www.bcn.cl/leychile

Biblioteca del Congreso Nacional de Chile. (2024). *Ley N.º 21.663, Ley Marco de Ciberseguridad*. https://www.bcn.cl/leychile

Biblioteca del Congreso Nacional de Chile. (2024). *Ley N.º 21.719 que regula la protección y el tratamiento de los datos personales y crea la Agencia de Protección de Datos Personales*. https://www.bcn.cl/leychile

Center for Internet Security. (2024). *CIS Critical Security Controls* (versión 8.1). https://www.cisecurity.org/controls

International Organization for Standardization & International Electrotechnical Commission. (2011). *Information technology — Security techniques — Privacy framework* (ISO/IEC 29100:2011).

International Organization for Standardization & International Electrotechnical Commission. (2019). *Code of practice for protection of personally identifiable information (PII) in public clouds acting as PII processors* (ISO/IEC 27018:2019).

International Organization for Standardization & International Electrotechnical Commission. (2019). *Extension to ISO/IEC 27001 and ISO/IEC 27002 for privacy information management* (ISO/IEC 27701:2019).

International Organization for Standardization & International Electrotechnical Commission. (2022). *Information security management systems — Requirements* (ISO/IEC 27001:2022).

International Organization for Standardization & International Electrotechnical Commission. (2022). *Information security controls* (ISO/IEC 27002:2022).

International Organization for Standardization & International Electrotechnical Commission. (2023). *Information security incident management — Part 1: Principles and process* (ISO/IEC 27035-1:2023).

Kotoduo. (2026). *Antecedentes complementarios para definición del proyecto Kotoduo* [Documento no publicado]. Proyecto Integrado, INACAP.

Kruchten, P. B. (1995). The 4+1 view model of architecture. *IEEE Software, 12*(6), 42–50. https://doi.org/10.1109/52.469759

Lodderstedt, T., Bradley, J., Labunets, A., & Fett, D. (2025). *Best current practice for OAuth 2.0 security* (RFC 9700). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc9700

MongoDB, Inc. (2025). *Schema validation*. MongoDB Manual. https://www.mongodb.com/docs/manual/core/schema-validation/

Morris, K. (2020). *Infrastructure as code: Dynamic systems for the cloud age* (2.ª ed.). O'Reilly Media.

National Institute of Standards and Technology. (2008). *Technical guide to information security testing and assessment* (NIST SP 800-115). https://doi.org/10.6028/NIST.SP.800-115

National Institute of Standards and Technology. (2020). *NIST privacy framework: A tool for improving privacy through enterprise risk management* (versión 1.0). https://doi.org/10.6028/NIST.CSWP.01162020

National Institute of Standards and Technology. (2022). *Secure software development framework (SSDF) version 1.1* (NIST SP 800-218). https://doi.org/10.6028/NIST.SP.800-218

National Institute of Standards and Technology. (2024). *The NIST cybersecurity framework (CSF) 2.0* (NIST CSWP 29). https://doi.org/10.6028/NIST.CSWP.29

National Institute of Standards and Technology. (2025). *Digital identity guidelines: Authentication and authenticator management* (NIST SP 800-63B-4). https://doi.org/10.6028/NIST.SP.800-63B-4

National Institute of Standards and Technology. (2025). *Incident response recommendations and considerations for cybersecurity risk management* (NIST SP 800-61 Rev. 3). https://doi.org/10.6028/NIST.SP.800-61r3

Neon. (s.f.). *Manage Neon with Terraform*. Neon documentation. https://neon.com/docs/reference/terraform

Nielsen, J. (1994, actualizado 2024). *10 usability heuristics for user interface design*. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

OpenTofu. (s.f.). *OpenTofu documentation*. The Linux Foundation. https://opentofu.org/docs/

OpenTofu. (s.f.). *State and plan encryption*. OpenTofu documentation. https://opentofu.org/docs/language/state/encryption/

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

OWASP Foundation. (2021). *Application Security Verification Standard 4.0.3*. https://owasp.org/www-project-application-security-verification-standard/

OWASP Foundation. (s.f.). *OWASP Cheat Sheet Series: Password storage, session management, cross-site request forgery prevention, file upload*. https://cheatsheetseries.owasp.org/

PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/

Render. (s.f.). *Render Terraform provider*. Render documentation. https://render.com/docs/terraform-provider

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning.

Sánchez Cárcamo, M. (2026). *Planteamiento del problema: Caso Kotoduo* [Material de curso]. Proyecto Integrado, INACAP.

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. https://scrumguides.org/scrum-guide.html

Shostack, A. (2014). *Threat modeling: Designing for security*. Wiley.

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson.

Wiggins, A. (2011). *The twelve-factor app*. https://12factor.net/

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/

11 Preparación para la defensa oral

Pregunta 1 (Arquitectura). ¿Por qué un monolito modular y no microservicios? ¿Cómo demuestran que las cinco vistas 4+1 describen la misma solución?

Respuesta: La elección responde a los requerimientos y restricciones:

- **Escala:** unos 25 usuarios y cerca de 30 OT por semana. Un solo proceso Node.js con un pool de 10 conexiones cubre el umbral de RNF-03 (p95 $<$ 500 ms con 40 VU, CP-20).
- **Equipo y costo:** son dos desarrolladores y no hay presupuesto definido (RNF-08). Los microservicios agregarían despliegues independientes, observabilidad distribuida y consistencia eventual en todas partes sin aportar valor al cliente.
- **Evolución:** el monolito es **modular**. M1–M8 tienen límites explícitos (Fig. 4.11), con dependencias en una sola dirección (Fig. 4.13), y la integración asíncrona ya pasa por el outbox. Si en el futuro M5 se integrara con proveedores, se podría extraer sin reescribir el resto.

La coherencia se demuestra recorriendo el escenario CU-06 por todas las vistas:

- **Lógica:** `EtapaOrden` y `EtapaService.terminar` (Fig. 4.3, 4.6 y 4.7).
- **Desarrollo:** se implementa en el componente M4, paquete `modules/etapas` (Fig. 4.11–4.13).
- **Procesos:** está en el bucle de producción, con concurrencia en el *fork* y trabajo del relay (Fig. 4.14, 4.16–4.18 y Tabla 4.1).
- **Física:** se ejecuta en el contenedor de Render y persiste en Neon y Atlas (Fig. 4.19).
- **Verificación:** CP-01, CP-12, CP-19 y CP-27.

Los nombres de módulos, entidades y roles son idénticos en todas las vistas.

Pregunta 2 (Base de datos). Usan PostgreSQL y MongoDB. ¿Cómo garantizan la consistencia si no pueden hacer una transacción que abarque ambas? ¿Qué pasa si MongoDB está caído cuando un operario termina una etapa?

Respuesta: Se aplica el patrón **outbox transaccional**. El cambio de negocio (etapa terminada, consumo, siguiente etapa disponible) y el evento de auditoría se escriben en la **misma transacción ACID de PostgreSQL**, en la tabla `outbox_evento`: o se guardan ambos o ninguno. El relay publica luego en MongoDB con un `_id` determinista (`evt-<id>`) mediante un *upsert*. Esto da entrega **al menos una vez** y efecto **exactamente una vez**. Esto se verificará con la prueba de reintento definida en 6.4: publicar dos veces el mismo evento debe dejar un solo documento.

Si MongoDB está caído, la operación del operario **no falla**: el evento queda pendiente (`procesado_en IS NULL`) y se reintenta. Tras 10 intentos fallidos se alerta en Sentry. Esto es aceptable porque PostgreSQL es la única fuente de verdad del estado de negocio y MongoDB nunca decide transiciones; solo registra lo ocurrido.

Se descartó la doble escritura desde la API porque una caída entre ambas escrituras dejaría una acción sin auditar, lo que rompería RNF-05. También se descartó el *two-phase commit* porque ambos motores gestionados no lo soportan entre sí y bloquearía la operación.

Pregunta 3 (Base de datos / integridad). ¿Por qué validan las transiciones de estado y el stock en la base de datos si ya lo hace la aplicación? ¿No es duplicar

lógica?

Respuesta: Es **defensa en profundidad** sobre las reglas cuya violación tendría más impacto en el negocio. La aplicación valida primero (`puedeTransicionar`) para dar mensajes claros al usuario. La base de datos garantiza la regla **ante cualquier cliente**: un error en una versión futura, un script de migración, una carga desde el Excel o una consulta manual de soporte.

- Los triggers `trg_etapa_transicion` y `trg_orden_transicion` rechazan, por ejemplo, PENDIENTE→TERMINADA en una etapa, TERMINADA→EN_PRODUCCION en una OT o cerrar una OT con etapas abiertas (CP-02, Tabla 6.4).
- `CHECK (stock_actual >= 0)` aborta la transacción **completa** si un consumo deja stock negativo, de modo que no queda una etapa terminada con el kardex inconsistente (CP-14).
- El índice único parcial `ux_alerta_auto_abierta` con `ON CONFLICT DO NOTHING` garantiza **una** alerta automática abierta por insumo aun con consumos concurrentes, algo que la aplicación no puede asegurar sin bloqueos adicionales.

La duplicación está controlada: un único grafo de estados (Fig. 4.7) es la especificación de ambas implementaciones, y CP-01 y CP-02 prueban las dos.

Pregunta 4 (Seguridad / autenticación). ¿Por qué no guardan el JWT en `localStorage`, como muchos tutoriales? ¿Cómo se protegen de XSS y CSRF a la vez?

Respuesta: `localStorage` es legible por cualquier script de la página, así que un solo XSS (AM-03) permitiría robar un token de larga duración. El diseño separa dos tokens:

- **Token de acceso:** JWT EdDSA de **10 minutos**, guardado **solo en memoria** y enviado como `Bearer`. Un XSS no puede persistirlo más allá de su breve vida y, como no es una cookie, las peticiones de negocio **no son vulnerables a CSRF**: un sitio externo no puede agregar el encabezado `Authorization`.
- **Token de refresco:** opaco y guardado con hash en la BD. Viaja en una cookie `__Host-` con `HttpOnly` (inaccesible para JavaScript, mitiga XSS) y `SameSite=Strict` (no se envía desde otros sitios, mitiga CSRF). El único endpoint que usa esa cookie verifica además `Origin` y un encabezado personalizado (CP-26). El refresco **rota** el token en cada uso y, si detecta la reutilización de uno ya rotado, revoca toda la familia (CP-30), lo que detecta robos de sesión.

Contra XSS se suman: React escapa por defecto, `dangerouslySetInnerHTML` está prohibido por lint y la CSP `script-src 'self'` bloquea scripts inyectados (CP-25).

Pregunta 5 (Seguridad / autorización y concurrencia). Un operario cambia el ID en la URL para terminar la etapa de un compañero, o dos operarios presionan "Terminar" al mismo tiempo. ¿Qué ocurre exactamente?

Respuesta: Son dos riesgos distintos con controles distintos:

- **IDOR (AM-05, OWASP A01).** El permiso por rol (`etapas.avanzar_propias`) no basta, porque ambos operarios lo tienen. Por eso `EtapaService.terminar` bloquea la fila con `SELECT … FOR UPDATE` y compara `etapa.responsable_id` con `actor.id`, que proviene del JWT verificado y nunca del cuerpo de la solicitud. Si no coinciden, responde **404**, para no revelar que la etapa existe. Además, las consultas de lectura del operario filtran en SQL por `responsable_id = $1`, y los identificadores son UUID no enumerables. CP-27 lo verifica.

- **Concurrencia (CP-19).** Ambas solicitudes llegan con `version = 3`. La primera obtiene el bloqueo de fila, termina la etapa y el trigger incrementa `version` a 4. La segunda espera el bloqueo y, al obtenerlo, encuentra `version = 4` $\neq$ `3` y estado TERMINADA. Responde **409 Conflict** ("la etapa fue actualizada por otro usuario") y no vuelve a habilitar la siguiente etapa ni a descontar insumos dos veces.

El resultado es una sola transición, un solo consumo y un solo evento en la bitácora: la integridad se mantiene sin depender de que la UI deshabilite el botón.

12 Manifiesto de uso de inteligencia artificial

Este manifiesto declara cómo se usó la inteligencia artificial generativa en la elaboración de este informe y de sus anexos, qué partes son responsabilidad directa del estudiante y cómo se verificó lo producido con asistencia.

Herramienta utilizada. Claude Code (Anthropic), con el modelo Claude Opus, usado como asistente dentro del editor de código.

Usos declarados.

Entregable	Uso de la IA	Intervención del estudiante
Texto del informe	Redacción de borradores, corrección de estilo y revisión de coherencia entre secciones	Definición del contenido, revisión y aprobación de cada cambio
Diagramas (Anexo A)	Escritura de las fuentes PlantUML a partir de las decisiones de arquitectura	Decisiones de arquitectura, revisión de las figuras renderizadas
Modelo de datos (Anexos B y C)	Borradores del DDL de PostgreSQL, de los validadores de MongoDB y de las consultas de ejemplo	Reglas de negocio e integridad, ejecución de la verificación (Tabla 6.4)
Prototipo (Anexo D)	Maquetación HTML/CSS de las pantallas	Flujos, perfiles y contenido de cada pantalla
Patrones seguros (Anexo G)	Fragmentos de código de referencia	Selección de controles y estándares (capítulo 7)
Presentación de disertación	Armado de las diapositivas a partir del informe	Revisión del contenido y preparación de la exposición
Versión LaTeX	Conversión del documento a la plantilla institucional	Revisión del documento compilado

Lo que no se delegó. El diagnóstico de la Unidad 1 y la información entregada por la contraparte de Kotoduo, el alcance del proyecto, las decisiones de arquitectura, tecnología y priorización, y la aprobación final de cada cambio incorporado al informe. La IA no tuvo contacto con la contraparte ni accedió a sus sistemas.

Verificación aplicada. Ningún resultado se incorporó sin revisión:

- los borradores de base de datos se ejecutaron sobre los motores reales y se contrastaron con los criterios verificables de la Tabla 6.4;
- los diagramas se renderizaron y se revisaron contra el texto y la matriz de permisos;
- las afirmaciones técnicas y normativas se contrastaron con la documentación oficial citada en la sección 10;
- el texto final se revisó para comprobar la coherencia entre el informe, los anexos y la presentación.

Límites y resguardos. Los modelos de lenguaje pueden producir errores o información inexacta con apariencia correcta; por eso toda salida se trató como borrador sujeto a verificación. La información de la contraparte entregada al asistente se limitó a la ya descrita en el diagnóstico de la Unidad 1, y el prototipo usa datos ficticios y anonimizados. La responsabilidad por el contenido de este informe es exclusivamente del estudiante.

Declaración. Declaro que el uso de inteligencia artificial descrito en este manifiesto es completo y veraz, y que el contenido de este informe refleja mi comprensión y mis decisiones sobre el proyecto.

Carlos David Mac-Iver Reydet — 23 de octubre de 2026

13 Anexos

Los anexos E y H a L forman parte de este documento y reúnen las tablas extensas de consulta. Los demás son archivos del repositorio entregado junto con el informe.

Anexo	Contenido	Formato	Referenciado en
A	Fuentes PlantUML de todas las figuras y sus imágenes renderizadas	Archivos PlantUML y PNG	Caps. 4, 5 y 6
B	Borrador referencial: script DDL de PostgreSQL y consultas de ejemplo	Scripts SQL	6.1–6.3
C	Borrador referencial: validadores, índices y documentos de MongoDB + script de inicialización	Esquemas JSON y script de MongoDB	6.4–6.5
D	Prototipo navegable y capturas v1 → v2	Maqueta HTML/CSS	5.3–5.5
E	Lista de cotejo de seguridad para PR y pruebas (ASVS L2, extracto)	Este documento	7.5, 8.6, 8.9
F	Reportes de ejecución de pruebas (Vitest, k6, ZAP, Lighthouse, Playwright)	Reportes HTML/JSON [Adjuntar tras la ejecución]	8.8
G	Borrador referencial: fragmentos de código de los patrones de codificación segura	Documento con código TypeScript	7.6
H	Plan detallado de actividades (A00–A31) y matriz de dependencias	Este documento	3.1, 3.2
I	Matriz de tecnologías con justificación y necesidad futura	Este documento	4.7
J	Diccionario de datos completo	Este documento	6.2
K	Matriz de modelado de amenazas y patrones de codificación segura en detalle	Este documento	7.4, 7.6
L	Especificación de los casos de prueba CP-01 a CP-36	Este documento	8.3 a 8.8

Anexo E — Lista de cotejo de seguridad (extracto ASVS 4.0.3 L2)

#	Control	Ref. ASVS	Cumple (S/N)	Evidencia
1	Las contraseñas tienen ≥ 12 caracteres, sin reglas de composición, y se comparan contra una lista de claves comprometidas	V2.1.1, V2.1.7	☐	CP-07
2	Las contraseñas se almacenan con Argon2id con parámetros iguales o superiores a los de OWASP	V2.4.1	☐	Revisión del servicio de autenticación
3	El bloqueo y la limitación de tasa protegen el login	V2.2.1	☐	CP-29
4	Las cuentas privilegiadas exigen MFA	V2.8 / V4.3.1	☐	CP-10
5	Los tokens de sesión se generan con ≥ 128 bits de entropía y se guardan con hash	V3.2.2	☐	Revisión del servicio de sesiones
6	Las cookies usan `__Host-`, `Secure`, `HttpOnly` y `SameSite`	V3.4.1–V3.4.5	☐	CP-09
7	Las sesiones expiran por inactividad y tiempo absoluto, y se revocan al cerrar sesión	V3.3.1–V3.3.4	☐	CP-30
8	El control de acceso deniega por defecto y se aplica en el servidor	V4.1.1, V4.1.5	☐	CP-28
9	Se verifica la autorización a nivel de objeto (IDOR)	V4.2.1	☐	CP-27
10	Toda entrada se valida con listas blancas	V5.1.3	☐	CP-06, CP-24
11	Las consultas a BD están parametrizadas	V5.3.4	☐	Semgrep + CP-24
12	Los errores no exponen información sensible	V7.4.1	☐	CP-24
13	Los logs no contienen credenciales ni tokens	V7.1.1	☐	Revisión de logs
14	Las cabeceras CSP, HSTS y `nosniff` están presentes	V14.4	☐	CP-32
15	Los archivos subidos se validan por tipo y tamaño y se sirven sin ejecución	V12.1, V12.5	☐	CP-31
16	No hay secretos en el repositorio y se rotan	V2.10, V14.3	☐	*Secret scanning*
17	Las dependencias no tienen vulnerabilidades altas conocidas	V14.2.1	☐	`npm audit`

Anexo H — Plan detallado de actividades y matriz de dependencias

Detalla el resumen por Sprint de la sección 3.1 (Tabla 3.2) y las dependencias clave de la sección 3.2 (Tabla 3.5).

Tabla H.1 — Plan detallado de actividades. Duración en días hábiles; "D" indica el día relativo del proyecto.

ID	Fase / Sprint	Actividad	Entregable	Responsable	Inicio	Término	Duración
A00	S0 – Diseño	*Spike* técnico: PWA en iOS/Android, límites de capas gratuitas	Nota técnica de factibilidad	Integrante 1	D1	D2	2 d
A01	S0 – Diseño	Validar con Kotoduo la línea base RF/RNF/HU y los supuestos	Acta de validación + backlog v1	Integrante 2	D1	D1	1 d
A02	S0 – Diseño	Priorizar backlog, estimar SP, definir DoR/DoD	Product Backlog priorizado (Tabla 3.1)	Integrante 2	D2	D2	1 d
A03	S0 – Diseño	Arquitectura 4+1, contrato OpenAPI y selección tecnológica	Cap. 4 + `openapi.yaml` v0	Integrante 1	D3	D5	3 d
A04	S0 – Diseño	Prototipo navegable v1 y guía de estilo	Prototipo v1 (Anexo D) + sección 5.4	Integrante 2	D3	D5	3 d
A05	S0 – Diseño	Modelo de datos: ER, diccionario, DDL y esquemas MongoDB	Cap. 6 + Anexos B y C	Integrante 1	D6	D7	2 d
A06	S0 – Diseño	Prueba de usabilidad del prototipo con trabajadores y rediseño v2	Registro de feedback (5.5) + prototipo v2	Integrante 2	D6	D7	2 d
A07	S0 – Diseño	Diseño de seguridad: RBAC, autenticación, amenazas, estándares y marco normativo chileno	Cap. 7 (incluye matriz normativa 7.7)	Integrante 1	D8	D9	2 d
A08	S0 – Diseño	Plan de pruebas y casos de prueba	Cap. 8	Integrante 2	D8	D9	2 d
A09	S0 – Diseño	Consolidación del Informe N.º 2, Sprint Review y Retro	**Hito H1: Informe N.º 2**	Integrante 1	D10	D10	1 d
A10	S1 – Fundaciones	Monorepo, CI en GitHub Actions, Docker Compose, staging	Pipeline CI en verde	Integrante 1	D11	D11	1 d
A11	S1 – Fundaciones	Migraciones y datos semilla (roles, áreas, tipos de trabajo)	Esquema `sgp` en staging	Integrante 1	D12	D12	0,5 d
A12	S1 – Fundaciones	API M1: login, Argon2id, JWT/refresh, MFA, bloqueo, usuarios	Endpoints `/auth`, `/usuarios` + pruebas	Integrante 1	D12	D13	1,5 d
A13	S1 – Fundaciones	Shell PWA, componentes de UI, pantalla de login	PWA instalable en staging	Integrante 2	D11	D12	2 d

ID	Fase / Sprint	Actividad	Entregable	Responsable	Inicio	Término	Duración
A14	S1 – Fundaciones	API M2: plantillas de proceso	Endpoints `/plantillas` + pruebas	Integrante 1	D14	D14	1 d
A15	S1 – Fundaciones	UI de plantillas, administración de usuarios y perfil	Pantallas P-07, P-09 y P-12	Integrante 2	D13	D14	2 d
A16	S2 – Flujo central	API M3: registro de OT, generación de etapas, asignación	Endpoints `/ordenes` + pruebas	Integrante 1	D16	D17	2 d
A17	S2 – Flujo central	UI de registro de OT y asignación	Pantalla P-04	Integrante 2	D16	D17	2 d
A18	S2 – Flujo central	API M4: avance de etapa transaccional, outbox, notificaciones	Endpoints `/etapas` + relay outbox	Integrante 1	D18	D19	2 d
A19	S2 – Flujo central	UI Mi Trabajo, Detalle de orden y Avisos (*mobile-first*)	Pantallas P-02, P-03 y P-11	Integrante 2	D18	D19	2 d
A20	S3 – Insumos y diseño	API M5: insumos, movimientos, alertas, faltantes proyectados	Endpoints `/insumos`, `/alertas`	Integrante 1	D21	D22	2 d
A21	S3 – Insumos y diseño	API M6 y evidencias: URL prefirmadas R2 + `evidencias_etapa`	Endpoints `/diseno`, `/evidencias`	Integrante 1	D23	D24	2 d
A22	S3 – Insumos y diseño	UI de insumos, faltante, aprobación de diseño y evidencia	Pantallas P-05 y P-08 + diálogos	Integrante 2	D21	D22	2 d
A23	S3 – Insumos y diseño	Suite de pruebas de integración (Supertest + Testcontainers)	CP-09 a CP-19 automatizados	Integrante 2	D23	D24	2 d
A24	S4 – Supervisión y piloto	API M7 (vista panel) y M8 (consulta de bitácora)	Endpoints `/panel`, `/bitacora`	Integrante 1	D26	D26	1 d
A25	S4 – Supervisión y piloto	UI de panel y bitácora	Pantallas P-06 y P-10	Integrante 2	D26	D26	1 d
A26	S4 – Supervisión y piloto	Pruebas de rendimiento (k6) y ajuste de índices	Informe CP-20 a CP-23	Integrante 1	D27	D27	1 d
A27	S4 – Supervisión y piloto	Pruebas de seguridad (ZAP, checklist ASVS) y correcciones	Informe CP-24 a CP-32	Integrante 1	D28	D29	2 d
A28	S4 – Supervisión y piloto	Pruebas E2E (Playwright) y usabilidad con usuarios reales	Informe CP-33 a CP-36	Integrante 2	D27	D29	3 d

ID	Fase / Sprint	Actividad	Entregable	Responsable	Inicio	Término	Duración
A29	S4 – Supervisión y piloto	Despliegue en producción y carga inicial desde el Excel actual	Sistema en producción (piloto)	Integrante 1	D30	D30	0,5 d
A30	S4 – Supervisión y piloto	Capacitación a trabajadores y guía rápida de uso	Guía rápida + registro de asistencia	Integrante 2	D30	D30	0,5 d
A31	S4 – Supervisión y piloto	Sprint Review final con el cliente, retrospectiva y cierre	**Hito H2: piloto aceptado**	Integrante 2	D30	D30	0,5 d

Tabla H.2 — Matriz de dependencias y prioridades. FC = fin-comienzo; CC = comienzo-comienzo (definiciones en la sección 3.2).

Tarea	Depende de	Tipo	Prioridad	Justificación técnica del orden	Efecto si se retrasa
A00	—	—	Media	Despeja el riesgo de soporte PWA/push en iOS antes de fijar la arquitectura	+1 d retrasa A03; riesgo de rediseño si iOS no soporta push
A01	—	—	**Alta**	Todo el diseño descansa en supuestos que el cliente debe validar	Cada día retrasa A02–A04; riesgo de diseñar flujos incorrectos
A02	A01	FC	Alta	No se puede priorizar sin la línea base validada	Retrasa A04 y el compromiso del Sprint 1
A03	A00, A01	FC	**Alta**	Define los módulos, el contrato API y la tecnología de los que dependen datos, seguridad y pruebas	Ruta crítica: +n d desplaza H1 en n d
A04	A02	FC	Alta	El prototipo debe reflejar las HU priorizadas	Retrasa A06; sin prototipo no hay feedback
A05	A03	FC	**Alta**	Las entidades derivan del modelo de clases (vista lógica)	Ruta crítica; bloquea A11
A06	A04	FC	Alta	Se prueba un prototipo existente	Se pierde el ciclo de mejora de la UI
A07	A03, A05	FC	**Alta**	RBAC y amenazas requieren conocer componentes y datos sensibles	Ruta crítica; bloquea A12
A08	A03, A06	FC	Media	Los casos de prueba trazan CU (A03) y usabilidad (A06)	Retrasa H1; no bloquea la construcción
A09	A02–A08	FC	Alta	Consolida todos los artefactos	Retrasa la entrega del Informe N.º 2
A10	A03	FC	**Alta**	Sin CI ni ambientes no se cumple la DoD	Ruta crítica; bloquea A11 y A13
A11	A05, A10	FC	**Alta**	Las migraciones implementan el DDL sobre el ambiente creado	Ruta crítica; bloquea A12
A12	A07, A11	FC	**Alta**	Toda la API exige identidad y permisos	Ruta crítica: +1 d consume el buffer D15 y desplaza S2 en medio día
A13	A06; A10 (CC)	FC/CC	Alta	Implementa el diseño v2; puede iniciar en paralelo con la creación del repositorio	Retrasa A15 y la demo del Sprint 1
A14	A12	FC	**Alta**	Los endpoints de plantillas ya requieren autorización por rol	Ruta crítica; bloquea A16
A15	A13; A12 (CC)	FC/CC	Media	Usa el contrato OpenAPI, no la API terminada	Plantillas se cargan por script temporalmente
A16	A14	FC	**Alta**	La OT se genera a partir de la plantilla	Ruta crítica; bloquea A18 y A20
A17	A15; A16 (CC)	FC/CC	Alta	Consume el contrato de `/ordenes`	Retrasa A19
A18	A16	FC	**Alta**	El avance de etapa requiere etapas generadas; es el corazón de RF-05/RF-06	Ruta crítica; el problema principal del cliente queda sin resolver

Tarea	Depende de	Tipo	Priori-dad	Justificación técnica del orden	Efecto si se retrasa
A19	A17; A18 (CC)	FC/CC	**Alta**	Pantalla más usada por operarios	Bloquea las pruebas de usabilidad de A28
A20	A16, A18	FC	Media	Los requerimientos de insumo nacen de la OT y el consumo se registra al terminar la etapa	Ruta crítica (por recurso); +n d desplaza A21
A21	A18	FC	Media	Evidencia y diseño se adjuntan a etapas existentes	Ruta crítica (por recurso)
A22	A19; A20 (CC)	FC/CC	Media	Reutiliza el diálogo de etapa de A19	Retrasa A23
A23	A20, A21 (CC), A22	FC	Alta	Verifica la integración de M3–M6 antes del endurecimiento	Defectos de integración llegan a S4
A24	A18, A20	FC	Media	El panel agrega estados de etapas e insumos	Ruta crítica; bloquea A26 y A27
A25	A22; A24 (CC)	FC/CC	Media	Consume la vista del panel	Retrasa A28
A26	A24	FC	Alta	El rendimiento se mide con todas las consultas pesadas implementadas	Riesgo de no cumplir RNF-03 en el piloto
A27	A24, A23	FC	**Alta**	Las pruebas de seguridad deben cubrir la superficie completa	No se autoriza pasar a producción (control de salida)
A28	A25	FC	Alta	Las pruebas E2E y de usabilidad requieren la UI completa	Sin evidencia del criterio de éxito de autonomía
A29	A26, A27, A28	FC	**Alta**	Solo se despliega lo que pasó las pruebas (DoD)	Desplaza H2
A30	A28; A29 (CC)	FC/CC	Alta	Se capacita sobre la versión aprobada por las pruebas, el mismo día de su paso a producción	Baja adopción del sistema
A31	A29, A30	FC	Alta	Cierre formal con el cliente	Desplaza H2

Anexo I — Matriz de tecnologías

Detalla la Tabla 4.2 de la sección 4.7.

Tabla I.1 — Matriz de tecnologías

Componente	Tecnología / herramienta	Uso en la solución	Justificación técnica	Necesidad futura considerada
Front-end	React 19 + TypeScript + Vite	SPA de las pantallas P-01 a P-12	Ecosistema maduro, tipado estático compartido con la API, *build* rápido; amplia oferta de desarrolladores en Chile para mantenimiento	Componentes reutilizables si se agrega un portal para clientes
Modalidad móvil	**PWA** (Web App Manifest + Service Worker con `vite-plugin-pwa`/Workbox)	Instalación en el teléfono, ícono propio, caché y notificaciones push	Cumple la expectativa de "app para el celular" sin publicar en tiendas (sin costo de cuentas de desarrollador ni revisiones), una sola base de código para Android, iOS y PC, y actualización inmediata. Push en iOS desde 16.4 con la PWA instalada (validado en A00)	Si se requiere hardware nativo, como un lector de códigos avanzado, la misma UI puede empaquetarse con Capacitor
Estado / datos en cliente	TanStack Query	Caché y revalidación de llamadas API	Reduce peticiones y da sensación de inmediatez en redes 4G inestables del taller	Base para el modo offline de lectura
Validación compartida	Zod (`packages/shared`)	Esquemas de DTO en el formulario y en la API	Una única fuente de verdad para las reglas de entrada (OWASP A03)	Generación de documentación OpenAPI desde los esquemas
Back-end	Node.js 24 LTS + Express 5 + TypeScript	API REST `/api/v1` (M1–M8), relay y planificador	Modelo de E/S asíncrono adecuado para muchas operaciones cortas; mismo lenguaje que el front-end (equipo de dos personas); soporte LTS hasta 2028	Separable en servicios por módulo; *escalado horizontal* (el relay ya soporta varias instancias)
Acceso a datos SQL	`pg` (node-postgres) + `node-pg-migrate`	Repositorios con consultas parametrizadas; migraciones versionadas	Control explícito del SQL (bloqueos, vistas, CTE) y parametrización nativa contra SQLi; sin la "magia" de un ORM que oculte consultas N+1	Réplicas de lectura para el panel si crece el volumen

Componente	Tecnología / herramienta	Uso en la solución	Justificación técnica	Necesidad futura considerada
BD estructurada	**PostgreSQL 16** (Neon, gestionado)	OT, etapas, plantillas, insumos, usuarios, outbox	ACID, integridad referencial, CHECK, triggers, índices parciales, JSONB y vistas: el dominio es relacional y exige consistencia (stock, estados). Neon ofrece *branching* para staging, PITR y capa gratuita	Particionar `movimiento_insumo` por año; Row-Level Security como defensa adicional
BD no estructurada	**MongoDB 8** (Atlas)	`bitacora_eventos`, `evidencias_etapa`, `notificaciones`	Eventos y evidencias con estructura variable por tipo de trabajo; escritura masiva *append-only*; índices TTL para retención automática (RNF-05) sin sobrecargar la BD transaccional	Analítica de tiempos de proceso con *aggregation pipeline* (cuellos de botella)
Almacenamiento de archivos	Cloudflare R2 (API S3)	Fotos de evidencia, diseños, respaldos	10 GB sin costo y **sin cargos por egreso**; compatible con S3 (portabilidad); URL prefirmadas sin pasar binarios por la API	Ciclo de vida para archivar diseños antiguos
Autenticación	`argon2` (Argon2id), `jose` (JWT EdDSA), `otplib` (TOTP)	Hash de claves, tokens y MFA	Algoritmos recomendados por OWASP y NIST SP 800-63B; librerías mantenidas	Inicio de sesión con Google Workspace (OIDC) si Kotoduo lo adopta
Seguridad HTTP	`helmet`, `cors`, `express-rate-limit`	Cabeceras CSP/HSTS, origen permitido, límites de tasa	Mitigan XSS, *clickjacking*, CSRF entre orígenes y fuerza bruta (sección 7.4)	Limitación distribuida con Redis si hay varias instancias
Notificaciones	Web Push (VAPID) con la librería `web-push`	Aviso de etapa disponible, alertas de insumo	Estándar W3C, sin depender de WhatsApp ni de costos de SMS; funciona con la app cerrada	Canal adicional por correo para administración
Hosting front-end	Cloudflare Pages	PWA estática en `sgp.kotoduo.cl`	CDN global, TLS automático, costo cero, *deploy* desde GitHub	Previsualizaciones por PR
Hosting back-end	Render — Web Service (plan Starter)	Contenedor Docker de la API	Despliegue por imagen Docker (portable), TLS y *health checks*; plan pagado básico (≈ US$ 7/mes) para evitar la suspensión por inactividad de la capa gratuita, que dejaría a los operarios esperando	Autoescalado horizontal; migración a cualquier host de contenedores

Componente	**Tecnología / herramienta**	**Uso en la solución**	**Justificación técnica**	**Necesidad futura considerada**
Borde / red	Cloudflare DNS, proxy y WAF	TLS 1.3, reglas WAF administradas, *rate limiting*	Protección DDoS y WAF sin costo adicional; oculta el origen	Cloudflare Access para restringir `/admin`
CI/CD	GitHub Actions + GitHub Environments	Lint, pruebas, SAST, auditoría de dependencias, *build*, despliegue con aprobación manual a producción	Integrado al repositorio; 2.000 min/mes gratis en repositorios privados	Despliegues *blue-green*
Contenedores	Docker + Docker Compose	Ambiente local idéntico a producción; Testcontainers en CI	Elimina el "en mi máquina funciona"; pruebas de integración contra motores reales	Orquestación (Kubernetes) solo si la escala lo exige
Infraestructura como código (**ideal, fuera del alcance comprometido**)	**OpenTofu** (módulos en `infra/`) + Trivy (análisis de configuración)	DNS, WAF, hosting de la PWA, servicio de la API, bases de datos y *buckets* de *staging* y producción	Infraestructura reproducible y revisable en PR; los módulos por rol permiten cambiar de proveedor sin rediseñar (RNF-08); licencia abierta MPL 2.0, sin costo y compatible con los proveedores del ecosistema Terraform	Primera mejora tras el piloto; luego, cambio de proveedor módulo a módulo (Tabla 4.3)
Observabilidad	`pino` (logs JSON), Sentry (errores), UptimeRobot (disponibilidad)	Logs con `request_id`, alertas de error y caída	Planes gratuitos; diagnósticos sin exponer datos sensibles en logs	Métricas de negocio (tiempo por etapa) en un tablero
Gestión del proyecto	GitHub Projects (tablero Scrum)	Backlog, Sprints, tablero y burndown	Vincula HU ↔ PR ↔ CI en una sola herramienta	—
Diagramas	PlantUML	Todas las figuras del informe	Diagramas como texto, versionables y revisables en PR	—

Anexo J — Diccionario de datos

Detalla la Tabla 6.1 de la sección 6.2.

Tabla J.1 — Diccionario de datos (PK = clave primaria, FK = clave foránea, UQ = única, NN = no nulo, CK = restricción CHECK, DF = valor por defecto)

Tabla	Campo	Tipo	Clave / restricción	Descripción
rol	id	SMALLSERIAL	PK	Identificador
rol	codigo	VARCHAR(30)	NN, UQ, CK ∈ {ADMINISTRADOR, ENCARGADA_PRODUCCION, VENDEDOR, DISENADOR, OPERARIO}	Código usado por el mapa de permisos
rol	nombre / descripcion	VARCHAR(60) / VARCHAR(255)	nombre NN	Texto para la UI
area	id	SMALLSERIAL	PK	Identificador
area	nombre	VARCHAR(60)	NN, UQ	Ventas, Diseño, Corte, Confección, Bordado, Sublimación, Impresión gran formato, Grabado, Instalación, Control y despacho
area	activa	BOOLEAN	NN, DF TRUE	Baja lógica
usuario	id	UUID	PK, DF `gen_random_uuid()`	Identificador no enumerable
usuario	nombre_usuario	VARCHAR(40)	NN, UQ (insensible a mayúsculas), CK `^[a-z0-9._-]{3,40}$`	Credencial de acceso (los operarios pueden no tener correo)
usuario	nombre_completo	VARCHAR(100)	NN	Nombre visible
usuario	correo	VARCHAR(120)	UQ parcial, CK formato	Opcional; requerido para recuperación por enlace
usuario	password_hash	TEXT	NN	Hash Argon2id en formato PHC (incluye sal y parámetros)
usuario	debe_cambiar_clave	BOOLEAN	NN, DF TRUE	Fuerza el cambio tras alta o restablecimiento

Tabla	Campo	Tipo	Clave / restricción	Descripción
usuario	mfa_habilitado / mfa_secreto_cifrado	BOOLEAN / BYTEA	CK `mfa_habilitado` → `secreto NOT NULL`	Secreto TOTP cifrado con AES-256-GCM (clave en el gestor de secretos)
usuario	area_id	SMALLINT	FK → area(id) ON DELETE SET NULL	Área principal del trabajador
usuario	activo	BOOLEAN	NN, DF TRUE	Desactivar en vez de borrar (conserva la trazabilidad)
usuario	intentos_fallidos / bloqueado_hasta	SMALLINT / TIMESTAMPTZ	CK ≥ 0	Control de fuerza bruta
usuario	ultimo_acceso, creado_en, actualizado_en	TIMESTAMPTZ	NN (creado/actualizado), DF now()	Auditoría básica; `actualizado_en` lo mantiene el trigger R1
usuario_rol	usuario_id, rol_id	UUID, SMALLINT	PK compuesta; FK → usuario ON DELETE CASCADE; FK → rol ON DELETE RESTRICT	Asignación de roles
usuario_rol	asignado_por, asignado_en	UUID, TIMESTAMPTZ	FK → usuario ON DELETE SET NULL	Quién otorgó el rol
sesion_refresh	id	UUID	PK	Sesión
sesion_refresh	usuario_id	UUID	NN, FK → usuario ON DELETE CASCADE	Titular
sesion_refresh	familia_id	UUID	NN	Agrupa rotaciones para detectar reutilización
sesion_refresh	token_hash	CHAR(64)	NN, UQ	SHA-256 del token opaco (el token nunca se almacena)
sesion_refresh	emitido_en, expira_en, revocado_en	TIMESTAMPTZ	CK expira_en > emitido_en	Ciclo de vida
sesion_refresh	ip, user_agent	INET, VARCHAR(255)	—	Contexto para la revocación y la auditoría
tipo_trabajo	id	SMALLSERIAL	PK	Identificador
tipo_trabajo	nombre	VARCHAR(80)	NN, UQ	P. ej., “Uniforme deportivo sublimado”

Tabla	Campo	Tipo	Clave / restricción	Descripción
tipo_trabajo	requiere_diseno, activo, version	BOOLEAN, BOOLEAN, INTEGER	NN; CK version > 0	Configuración de la plantilla
plantilla_etapa	id	SERIAL	PK	Identificador
plantilla_etapa	tipo_trabajo_id	SMALLINT	NN, FK → tipo_trabajo ON DELETE CASCADE	Plantilla a la que pertenece
plantilla_etapa	secuencia	SMALLINT	NN, CK > 0, UQ (tipo_trabajo_id, secuencia)	Orden de ejecución sin duplicados
plantilla_etapa	nombre, instrucciones	VARCHAR(80), TEXT	NN	Conocimiento operativo estandarizado
plantilla_etapa	area_id	SMALLINT	NN, FK → area ON DELETE RESTRICT	Área que ejecuta
plantilla_etapa	duracion_estimada_horas	NUMERIC(5,1)	NN, CK > 0	Base para la fecha estimada
plantilla_etapa	es_etapa_diseno	BOOLEAN	NN, DF FALSE	Marca la etapa que requiere aprobación
insumo	id	SERIAL	PK	Identificador
insumo	codigo	VARCHAR(20)	NN, UQ	Código interno (p. ej., `TIN-PLT-K`)
insumo	nombre	VARCHAR(80)	NN	Descripción
insumo	unidad_medida	VARCHAR(10)	NN, CK ∈ {UNIDAD, METRO, M2, LITRO, ML, KG, PLIEGO, ROLLO}	Unidad controlada
insumo	stock_actual, stock_minimo	NUMERIC(12,2)	NN, CK ≥ 0	Disponibilidad y umbral de alerta
insumo	activo, actualizado_en	BOOLEAN, TIMESTAMPTZ	NN	Baja lógica y trazabilidad
plantilla_insumo	plantilla_etapa_id, insumo_id	INTEGER, INTEGER	PK compuesta; FK → plantilla_etapa CASCADE; FK → insumo RESTRICT	Insumo estándar por etapa
plantilla_insumo	cantidad_por_unidad	NUMERIC(10,3)	NN, CK > 0	Consumo por unidad producida
orden_trabajo	id	UUID	PK	Identificador
orden_trabajo	codigo	VARCHAR(20)	NN, UQ, CK `^OT-[0-9]{4}-[0-9]{4,6}$`	Código de negocio

Tabla	Campo	Tipo	Clave / restricción	Descripción
orden_trabajo	tipo_trabajo_id	SMALLINT	NN, FK → tipo_trabajo ON DELETE RESTRICT	Plantilla usada
orden_trabajo	titulo, referencia_cliente	VARCHAR(120), VARCHAR(80)	NN	Referencia = solo nombre de la institución (**sin datos de contacto**, FB-03)
orden_trabajo	descripcion	TEXT	—	Detalle libre
orden_trabajo	cantidad	INTEGER	NN, CK > 0	Unidades a producir
orden_trabajo	especificaciones	JSONB	NN, DF '{}', CK objeto	Tallas, medidas, colores
orden_trabajo	prioridad	VARCHAR(10)	NN, CK ∈ {BAJA, NORMAL, ALTA, URGENTE}	Priorización en el panel
orden_trabajo	estado	VARCHAR(20)	NN, CK ∈ {REGISTRADA, EN_DISENO, EN_PRODUCCION, DETENIDA, TERMINADA, ENTREGADA, ANULADA}; trigger R2	Ciclo de vida de la OT (Fig. 4.8)
orden_trabajo	fecha_compromiso, fecha_estimada_termino	DATE	compromiso NN	Comprometida y calculada
orden_trabajo	creado_por	UUID	NN, FK → usuario ON DELETE RESTRICT	Autor (trazabilidad)
orden_trabajo	terminada_en	TIMESTAMPTZ	CK presente ⇔ estado ∈ {TERMINADA, ENTREGADA}	Cierre
etapa_orden	id	UUID	PK	Identificador
etapa_orden	orden_id	UUID	NN, FK → orden_trabajo ON DELETE CASCADE	OT a la que pertenece
etapa_orden	plantilla_etapa_id	INTEGER	FK → plantilla_etapa ON DELETE SET NULL	Origen (nulo si es una etapa *ad hoc*)
etapa_orden	secuencia	SMALLINT	NN, CK > 0, UQ (orden_id, secuencia)	Orden dentro de la OT

Tabla	Campo	Tipo	Clave / restricción	Descripción
etapa_orden	nombre, instrucciones, duracion_ estimada_horas, es_etapa_diseno	—	NN	Instantánea de la plantilla
etapa_orden	area_id	SMALLINT	NN, FK → area RESTRICT	Área responsable
etapa_orden	responsable_id	UUID	FK → usuario ON DELETE SET NULL	Trabajador asignado
etapa_orden	estado	VAR-CHAR(12)	NN, CK ∈ {PENDIENTE, DISPONIBLE, EN_CURSO, BLOQUEADA, TERMINADA, OMITIDA}; trigger R2	Estado de la máquina (Fig. 4.7)
etapa_orden	motivo_bloqueo	VAR-CHAR(255)	CK presente ⇔ estado = BLOQUEADA	Causa del bloqueo
etapa_orden	iniciada_en, terminada_en	TIMESTAM-PTZ	CK terminada ≥ iniciada; terminada ⇔ TERMINADA	Tiempos reales (análisis futuro)
etapa_orden	version	INTEGER	NN, DF 1 (incrementa el trigger R2)	Concurrencia optimista
requerimiento_ insumo	id	SERIAL	PK	Identificador
requerimiento_ insumo	etapa_orden_id, insumo_id	UUID, INTEGER	NN; FK CASCADE / RESTRICT; UQ (etapa, insumo)	Insumo por etapa concreta
requerimiento_ insumo	cantidad_ requerida, cantidad_ consumida	NUME-RIC(12,2)	NN; CK > 0 / ≥ 0	Plan contra consumo real
movimiento_ insumo	id	BIGSERIAL	PK	Identificador (alto volumen)
movimiento_ insumo	insumo_id	INTEGER	NN, FK → insumo RESTRICT	Insumo afectado
movimiento_ insumo	tipo, cantidad	VAR-CHAR(10), NUME-RIC(12,2)	CK tipo ∈ {INGRESO, CONSUMO, AJUSTE}; CK signo (ingreso > 0, consumo < 0, $\neq 0$)	Movimiento con signo
movimiento_ insumo	etapa_orden_id	UUID	FK SET NULL; CK obligatorio si es CONSUMO	Trazabilidad del consumo

Tabla	Campo	Tipo	Clave / restricción	Descripción
movimiento_insumo	usuario_id, comentario, creado_en	UUID, VARCHAR, TIMESTAMPTZ	usuario NN FK RESTRICT	Autor; **inmutable** (sin UPDATE/DELETE para la aplicación)
alerta_insumo	id	SERIAL	PK	Identificador
alerta_insumo	insumo_id, etapa_orden_id	INTEGER, UUID	insumo NN FK RESTRICT; etapa FK SET NULL	Insumo y etapa afectada
alerta_insumo	origen	VARCHAR(10)	NN, CK ∈ {AUTOMATICA, MANUAL}; CK MANUAL ⇒ reportada_por NN	Trigger o reporte del operario
alerta_insumo	estado	VARCHAR(12)	NN, CK ∈ {ABIERTA, EN_GESTION, CERRADA}; CK CERRADA ⇔ cerrada_en	Gestión de la reposición
alerta_insumo	cantidad_faltante, comentario	NUMERIC, VARCHAR	CK > 0	Detalle
alerta_insumo	reportada_por, cerrada_por, creada_en, cerrada_en	UUID, UUID, TIMESTAMPTZ	FK → usuario SET NULL	Responsables
aprobacion_diseno	id	SERIAL	PK	Identificador
aprobacion_diseno	orden_id, version	UUID, SMALLINT	NN; FK CASCADE; UQ (orden_id, version); UQ parcial: una PENDIENTE por OT	Versionado
aprobacion_diseno	evidencia_ref	CHAR(24)	NN	`_id` del documento en `evidencias_etapa` (MongoDB)
aprobacion_diseno	estado	VARCHAR(10)	NN, CK ∈ {PENDIENTE, APROBADO, RECHAZADO}; RECHAZADO ⇒ comentario NN	Resultado
aprobacion_diseno	subido_por, resuelto_por, resuelto_en	UUID, UUID, TIMESTAMPTZ	FK RESTRICT; CK resuelto_por ≠ subido_por; PENDIENTE ⇔ sin resolución	**Segregación de funciones**
outbox_evento	id	BIGSERIAL	PK	Orden de publicación

Tabla	Campo	Tipo	Clave / restricción	Descripción
outbox_evento	tipo, agregado, agregado_id	VARCHAR	NN	Qué ocurrió y sobre qué entidad
outbox_evento	actor_id	UUID	FK → usuario SET NULL	Quién lo hizo
outbox_evento	payload	JSONB	NN	Estado anterior/nuevo y datos del evento
outbox_evento	creado_en, procesado_en, intentos	TIMESTAMPTZ, TIMESTAMPTZ, SMALLINT	índice parcial sobre pendientes	Control del relay

Anexo K — Modelo de amenazas y patrones de codificación segura

Detalla las Tablas 7.2 y 7.3 del capítulo 7. El riesgo se calcula como probabilidad × impacto (sección 7.4).

Tabla K.1 — Matriz de modelado de amenazas

ID	Amenaza (STRIDE / OWASP)	Activo / componente	Escenario de riesgo	Prob.	Impacto	Riesgo	Medida de mitigación	Verificación
AM-01	Inyección SQL (T / A03)	PostgreSQL vía repositorios	Un parámetro manipulado (`?estado=' OR 1=1--`) altera la consulta y expone o modifica OT	Media	Alto	**Alto**	Consultas 100 % parametrizadas (`$1…$n`) en la capa *repository*; validación Zod con listas blancas; usuario de BD sin DDL; regla ESLint/Semgrep que prohíbe SQL concatenado	CP-24, SAST en CI
AM-02	Inyección NoSQL (T / A03)	MongoDB	Un objeto `{"$ne": null}` en lugar de un *string* evade un filtro	Baja	Medio	Medio	Zod exige tipos primitivos (`z.string().uuid()`); `sanitizeFilter` del driver; los filtros nunca se construyen directamente desde `req.body`	CP-24
AM-03	XSS almacenado (T / A03)	PWA: notas de evidencia, motivos, instrucciones	Un usuario guarda `<script>` en una nota que roba el token al visualizarla la Encargada	Media	Alto	**Alto**	React escapa por defecto; prohibido `dangerouslySetInnerHTML` (regla ESLint); CSP estricta `script-src 'self'`; token de acceso solo en memoria; refresh `HttpOnly`	CP-25
AM-04	CSRF (S / A01)	Endpoint `/auth/refresh` (único que usa cookie)	Un sitio externo fuerza un refresco o una acción con la cookie de la víctima	Baja	Medio	Medio	Las acciones de negocio usan *Bearer* (no son vulnerables a CSRF); cookie `SameSite=Strict`; verificación del encabezado `Origin` contra una lista permitida; encabezado personalizado `X-Requested-With` obligatorio; CORS restringido a `https://sgp.kotoduo.cl`	CP-26

ID	Amenaza (STRIDE / OWASP)	Activo / componente	Escenario de riesgo	Prob.	Impacto	Riesgo	Medida de mitigación	Verificación
AM-05	Acceso no autorizado horizontal / IDOR (E / A01)	Etapas, evidencias	El Operario A cambia el UUID en `/etapas/{id}/terminar` y avanza la etapa del Operario B	Media	Alto	**Alto**	Verificación de propiedad en el servicio y filtro SQL por `responsable_id`; UUID no secuenciales; respuesta 404 para recursos ajenos (no revela su existencia)	CP-27
AM-06	Escalada vertical de privilegios (E / A01)	API de plantillas, usuarios	Un Operario invoca `POST /plantillas` o `PATCH /usuarios/{id}/roles`	Media	Alto	**Alto**	`requierePermiso()` en cada ruta con denegación por defecto; prueba automatizada de inventario de rutas; roles solo modificables por Administrador con MFA	CP-28
AM-07	Fuerza bruta / relleno de credenciales (S / A07)	`/auth/login`	Un bot prueba contraseñas filtradas contra cuentas de operarios	Alta	Medio	**Alto**	Bloqueo progresivo; *rate limit* por IP y por cuenta; WAF de Cloudflare; lista de contraseñas comprometidas; MFA en cuentas privilegiadas	CP-29
AM-08	Robo o reutilización de sesión (S / A07)	Tokens	Robo del refresh desde un teléfono compartido o perdido	Media	Alto	**Alto**	Refresh `HttpOnly`, rotación y detección de reutilización; acceso de 10 min; revocación remota por el Administrador; expiración por inactividad	CP-30
AM-09	Subida de archivo malicioso (T / A04)	R2, PWA	Se sube un `.html`/`.svg` con script, o un archivo de 2 GB	Media	Medio	Medio	URL prefirmada con `Content-Type` y `Content-Length` firmados; lista blanca (jpg, png, webp, pdf) $\leq$ 10 MB; *bucket* privado; descarga con `Content-Disposition: attachment` y `X-Content-Type-Options: nosniff`; el SHA-256 se registra	CP-31

ID	Amenaza (STRIDE / OWASP)	Activo / componente	Escenario de riesgo	Prob.	Impacto	Riesgo	Medida de mitigación	Verificación
AM-10	Exposición de secretos (I / A05)	Credenciales de BD, claves JWT y VAPID	Se publica un `.env` en GitHub o se filtra un *stack trace*	Media	Alto	**Alto**	Secretos solo en el gestor de Render/GitHub; `.env` en `.gitignore` + *secret scanning* y *push protection* de GitHub; manejador de errores sin detalles internos; rotación documentada	CP-32, revisión de PR
AM-11	Configuración insegura / cabeceras (A05)	API, Pages	Falta de HSTS, CSP o `frame-ancestors` permite *clickjacking* o degradación	Media	Medio	Medio	`helmet` con CSP, HSTS, `frame-ancestors 'none'`, `Referrer-Policy`; TLS mínimo 1.2 en Cloudflare	CP-32
AM-12	Componentes vulnerables (A06)	Dependencias npm, imagen Docker	Una librería con CVE crítico explotable	Media	Alto	**Alto**	`npm audit --audit-level=high` y Dependabot en CI; imagen base `node:24-slim` actualizada; `package-lock` con integridad	CI (cada PR)
AM-13	Repudio de acciones (R / A09)	Bitácora	Un usuario niega haber terminado una etapa o aprobado un diseño	Media	Medio	Medio	Outbox en la misma transacción; bitácora inmutable (solo `insert`); eventos con actor, IP, `request_id` y marca de tiempo del servidor	CP-11
AM-14	Denegación de servicio (D)	API, BD	Una ráfaga de solicitudes o consultas costosas del panel dejan sin servicio al taller	Baja	Alto	Medio	Protección DDoS y *rate limiting* de Cloudflare; límites por usuario en la API; paginación obligatoria (máx. 50); índices y `statement_timeout = 5s`	CP-20, CP-23

ID	Amenaza (STRIDE / OWASP)	Activo / componente	Escenario de riesgo	Prob.	Impacto	Riesgo	Medida de mitigación	Verificación
AM-15	Pérdida de datos (D / integridad)	PostgreSQL, MongoDB	Borrado accidental, error de migración o caída del proveedor	Baja	Alto	Medio	PITR de Neon + `pg_dump` cifrado diario en R2 (RPO 24 h); copias de Atlas; migraciones probadas en staging; restauración ensayada trimestralmente (RTO 4 h)	Ensayo de restauración en A29
AM-16	Exposición de datos personales (I / Ley 19.628/21.719)	BD, logs	Se registran datos personales innecesarios o en logs	Baja	Medio	Bajo	Minimización: sin contactos de clientes; logs sin cuerpos de solicitud ni tokens (redacción en `pino`); cifrado en reposo de los proveedores	Revisión de logs (CP-32)

Tabla K.2 — Patrones de codificación segura y su verificación

#	Patrón obligatorio	Amenaza que mitiga	Documentación de referencia	Verificación
1	**Consultas parametrizadas.** Los valores viajan separados del texto SQL y nunca se concatenan. Solo la capa *repository* accede a la base de datos. Las lecturas del operario filtran por responsable en la propia consulta	Inyección SQL (A03); IDOR (A01)	OWASP *Query Parameterization Cheat Sheet*: `https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html`; node-postgres, *Queries*: `https://node-postgres.com/features/queries`	Regla Semgrep contra la concatenación; CP-24
2	**Validación estricta de entradas.** Cada ruta declara un esquema de lista blanca (tipos, rangos y longitudes) que rechaza campos no declarados. El mismo esquema se comparte con los formularios de la PWA, y los controladores solo reciben datos ya validados	Inyección (A03); asignación masiva	OWASP *Input Validation Cheat Sheet*: `https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html`; *Mass Assignment Cheat Sheet*: `https://cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment_Cheat_Sheet.html`; Zod: `https://zod.dev`	Prueba de inventario: toda ruta declara un esquema; CP-06, CP-24
3	**Autorización en tres niveles.** (a) Permiso por rol desde un mapa único compartido por la API y la PWA, con denegación por defecto; (b) verificación de **propiedad** del recurso en el servicio, usando la identidad del token y nunca la del cuerpo de la solicitud; (c) validación del **estado** del recurso. Si el recurso no pertenece al actor, se responde 404 para no revelar su existencia	Control de acceso roto (A01); IDOR; escalada de privilegios	OWASP *Authorization Cheat Sheet*: `https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html`; *IDOR Prevention Cheat Sheet*: `https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html`	CP-27, CP-28
4	**Almacenamiento de contraseñas y login uniforme.** Hash Argon2id con los parámetros mínimos recomendados por OWASP. El login responde con el mismo mensaje y un tiempo similar exista o no el usuario, y aplica bloqueo progresivo tras intentos fallidos	Robo de credenciales; enumeración de usuarios; fuerza bruta (A07)	OWASP *Password Storage Cheat Sheet*: `https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html`; *Authentication Cheat Sheet*: `https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html`; NIST SP 800-63B-4	CP-07, CP-29; revisión con Anexo E (ítems 1–3)

#	Patrón obligatorio	Amenaza que mitiga	Documentación de referencia	Verificación
5	**Sesión con dos tokens.** Token de acceso de vida corta guardado solo en memoria. Token de refresco opaco, aleatorio, almacenado con hash y enviado en una cookie `__Host-` con `HttpOnly`, `Secure` y `SameSite=Strict`. El refresco rota el token en cada uso y, si detecta la reutilización de uno ya rotado, revoca toda la familia	Secuestro de sesión; XSS que roba tokens; reutilización de tokens	OWASP *Session Management Cheat Sheet*: `https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html`; IETF *OAuth 2.0 Security Best Current Practice* (RFC 9700), sección sobre rotación de *refresh tokens*: `https://www.rfc-editor.org/rfc/rfc9700`	CP-09, CP-30
6	**Cabeceras, CORS, CSRF y límites de tasa.** CSP restrictiva sin `unsafe-inline`, HSTS, `nosniff` y política de *referrer*; CORS limitado al dominio de la PWA; tamaño máximo del cuerpo; límites de tasa más estrictos en autenticación. El único endpoint que usa cookie exige un `Origin` válido y un encabezado personalizado	XSS (A03); CSRF; abuso y fuerza bruta; configuración insegura (A05)	OWASP *HTTP Headers Cheat Sheet*: `https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html`; *Content Security Policy Cheat Sheet*: `https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html`; *CSRF Prevention Cheat Sheet*: `https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html`; Helmet: `https://helmetjs.github.io`	CP-25, CP-26, CP-29, CP-32
7	**Manejo centralizado de errores.** Un único manejador traduce los errores esperados a respuestas tipadas y los inesperados a un mensaje genérico con un identificador de solicitud. El detalle técnico (traza, SQL, versiones) va solo al log, con redacción de datos sensibles	Fuga de información (A05, A09)	OWASP *Error Handling Cheat Sheet*: `https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html`; *Logging Cheat Sheet*: `https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html`	CP-24 (sin fugas); revisión de logs

#	Patrón obligatorio	Amenaza que mitiga	Documentación de referencia	Verificación
8	**Subida segura de archivos.** El cliente sube directamente al almacenamiento mediante una URL prefirmada de corta vida que emite la API tras verificar la propiedad de la etapa. Se valida una lista blanca de tipos y un tamaño máximo; el servidor genera el nombre del archivo; el *bucket* es privado	Archivos maliciosos; IDOR; agotamiento de almacenamiento	OWASP *File Upload Cheat Sheet*: `https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html`; Cloudflare R2, *Presigned URLs*: `https://developers.cloudflare.com/r2/api/s3/presigned-urls/`	CP-31

Anexo L — Especificación de los casos de prueba

Detalla la Tabla 8.1 del capítulo 8. La columna “Resultado obtenido” se completa al ejecutar cada caso en su Sprint; el procedimiento de registro se describe en la sección 8.8.

Tabla L.1 — Pruebas unitarias (Vitest)

ID	HU / CU	Descripción	Precondición	Pasos / datos	Resultado esperado	Resultado obtenido	Estado
CP-01	HU-02 / CU-06	Transiciones válidas de la etapa	—	`EtapaOrden.puedeTransicionar` para DISPONIBLE→EN_CURSO, EN_CURSO→TERMINADA, EN_CURSO→BLOQUEADA, BLOQUEADA→EN_CURSO, DISPONIBLE→OMITIDA; `OrdenTrabajo.puedeTransicionar` para EN_PRODUCCION→DETENIDA, DETENIDA→EN_DISENO	`true` en los 7 casos	[Completar tras ejecución]	Pendiente
CP-02	HU-02 / CU-06	Transiciones inválidas rechazadas	—	Etapa: PENDIENTE→TERMINADA, TERMINADA→EN_CURSO, DISPONIBLE→TERMINADA, EN_CURSO→OMITIDA con la OT activa, DISPONIBLE→EN_CURSO con la OT DETENIDA; OT: TERMINADA→EN_PRODUCCION, DETENIDA→TERMINADA, EN_PRODUCCION→TERMINADA con etapas abiertas, DETENIDA→EN_PRODUCCION con una etapa BLOQUEADA	`false`; en BD, el trigger lanza `check_violation`	[Completar]	Pendiente
CP-03	HU-07 / CU-04	Generación de etapas desde la plantilla	Plantilla “Uniforme” con 6 etapas	`generarEtapas(tipo, cantidad=30)`	6 etapas con secuencia 1..6; la primera DISPONIBLE y el resto PENDIENTE; requerimientos = cantidad × cantidad_por_unidad (p. ej., tela 36 m)	[Completar]	Pendiente

ID	HU / CU	Descripción	Precondición	Pasos / datos	Resultado esperado	Resultado obtenido	Estado
CP-04	HU-05 / CU-04	Fecha estimada de término	Etapas con 23 h totales; registro el lunes 09:00	`calcularFechaEstimada(jornada=8)`	Miércoles de la misma semana (3 jornadas hábiles); omite sábado y domingo	[Completar]	Pendiente
CP-05	HU-09 / CU-08	Umbral de stock mínimo	—	`estaBajoMinimo` con stock 2/mín 2; 2,01/2; 0/0	`true`, `false`, `true` (la regla es $\leq$)	[Completar]	Pendiente
CP-06	HU-05 / CU-04	Validación de la OT	—	`NuevaOrdenSchema` con cantidad 0; título de 200 caracteres; campo extra `precio`; caso válido	Error en los 3 primeros (incluido el campo no permitido); éxito en el válido	[Completar]	Pendiente
CP-07	HU-11 / CU-01	Política de contraseñas	Lista de claves comprometidas cargada	`validarPoliticaClave`: “abc12345678” (11), “contraseña123” (comprometida), “tinta plotter azul 2026”	Rechaza, rechaza, acepta	[Completar]	Pendiente
CP-08	HU-11 / CU-02	Mapa de permisos	—	Verificar `PERMISOS_POR_ROL` contra la Tabla 7.1 (prueba tabular de 25 permisos $\times$ 5 roles)	125 aserciones coinciden; Operario sin `ordenes.crear`; Diseñador sin `diseno.resolver`	[Completar]	Pendiente

Tabla L.2 — Pruebas de integración (Supertest + Testcontainers)

ID	HU / CU	Descripción	Precondición	Pasos / datos	Resultado esperado	Resultado obtenido	Estado
CP-09	HU-11 / CU-01	Login correcto de operario	Usuario `operario.sub` activo	`POST /api/v1/auth/login` con credenciales válidas	200 con `accessToken` (JWT, `exp` = 10 min); cookie `__Host-sgp_rt` con `HttpOnly`, `Secure`, `SameSite=Strict`; evento LOGIN_EXITOSO en la bitácora	[Completar]	Pendiente
CP-10	HU-11 / CU-01	MFA obligatorio para la Encargada	`encargada.prod` con MFA	Login → `POST /auth/mfa` con código TOTP válido y luego inválido	Primer paso: 200 `{mfaRequerido: true}` sin tokens; código válido: 200 con tokens; inválido: 401	[Completar]	Pendiente
CP-11	HU-05, HU-12 / CU-04	Registro de OT end-to-end en datos	Vendedor autenticado	`POST /api/v1/ordenes` (tipo 1, cantidad 30)	201 con `codigo` OT-AAAA-NNNN; 6 etapas en PostgreSQL; ≤ 5 s después, documento ORDEN_REGISTRADA en `bitacora_eventos` con `_id = evt-n`	[Completar]	Pendiente
CP-12	HU-02 / CU-06	Terminar etapa habilita la siguiente	Etapa 2 EN_CURSO asignada al operario	`POST /api/v1/etapas/{id}/terminar { version, consumos:[{insumoId:3, cantidad:0.6}]}`	200 `{siguienteEtapa:"Sublimado", area:"Sublimación"}`; etapa 3 DISPONIBLE; stock descontado; notificación ETAPA_DISPONIBLE para el responsable de la etapa 3; variante con la etapa 3 OMITIDA: se habilita la etapa 4	[Completar]	Pendiente
CP-13	HU-02 / CU-06	Terminar la última etapa cierra la OT	Etapa 6 EN_CURSO	Terminar la etapa 6	OT en estado TERMINADA con `terminada_en`; evento ORDEN_ESTADO_CAMBIADO	[Completar]	Pendiente
CP-14	HU-09 / CU-08	Alerta automática única	Papel: stock 4, mínimo 2	Dos consumos: $-2{,}5$ y $-0{,}5$; luego un consumo de -2	Una sola alerta AUTOMATICA abierta; el tercer consumo (stock negativo) devuelve 409 y revierte la transacción completa	[Completar]	Pendiente

ID	HU / CU	Descripción	Precondición	Pasos / datos	Resultado esperado	Resultado obtenido	Estado
CP-15	HU-03 / CU-09	Reporte manual de faltante	Operario con etapa EN_CURSO	`POST /api/v1/alertas {insumoId, etapaId, comentario}`	201; alerta MANUAL con `reportada_por`; notificación ALERTA_INSUMO a los usuarios Encargada	[Completar]	Pendiente
CP-16	HU-06 / CU-07	Aprobación de diseño y segregación	Diseño v1 PENDIENTE subido por el Diseñador	(a) El Vendedor rechaza sin comentario; (b) rechaza con comentario; (c) el Diseñador intenta aprobar su v2; (d) el Vendedor aprueba la v2	(a) 400; (b) 200, RECHAZADO; (c) 403; (d) 200, APROBADO y la primera etapa de producción pasa a DISPONIBLE	[Completar]	Pendiente
CP-17	HU-04 / CU-06	Evidencia con URL prefirmada	Etapa propia; emulador S3 activo	Solicitar URL (image/jpeg, 300 KB) → PUT al emulador → `POST /evidencias` con metadatos	URL con expiración de 300 s; documento en `evidencias_etapa` con `object_key` y `sha256` válidos	[Completar]	Pendiente
CP-18	HU-10 / CU-10, CU-14	Panel identifica atrasadas y en riesgo	4 OT: compromiso ayer (activa), mañana, ayer (TERMINADA) y una activa con fecha estimada posterior al compromiso	`GET /api/v1/panel/ordenes?filtro=atrasadas`; luego ejecutar dos veces el Verificador de atrasos	Atrasadas: solo la primera, con `porcentaje_avance` correcto; la cuarta queda `en_riesgo` y la Encargada recibe una sola notificación (operación idempotente)	[Completar]	Pendiente
CP-19	HU-02 / CU-06	Concurrencia sobre la misma etapa	Etapa EN_CURSO, version = 3	Dos `POST /terminar` simultáneos (`Promise.all`) con version 3	Una respuesta 200 y otra 409; la siguiente etapa se habilita una sola vez	[Completar]	Pendiente

Tabla L.3 — Pruebas de rendimiento (k6 / Lighthouse)

ID	HU / CU	Descripción	Precondición	Pasos / datos	Resultado esperado	Resultado obtenido	Estado
CP-20	HU-01 / CU-05	Carga de Mi Trabajo	Staging con 5.000 OT; 40 usuarios de prueba	k6: rampa 0→40 VU en 1 min, 5 min sostenido, `GET /mi-trabajo` cada 3 s por VU	p95 < 500 ms, p99 < 1 s, errores < 1 %	[Completar]	Pendiente
CP-21	HU-02 / CU-06	Escrituras concurrentes de avance	20 VU, cada uno con etapas propias	k6: 20 VU × 3 min, iniciar + terminar etapa	p95 < 800 ms; 0 errores 5xx; outbox drenado en < 30 s tras la prueba	[Completar]	Pendiente
CP-22	RNF-02, RNF-03	Carga inicial de la PWA en el móvil	Build de producción en staging	Lighthouse móvil, “Slow 4G”, 3 ejecuciones (mediana)	LCP < 2,5 s; TBT < 200 ms; PWA instalable (manifest y SW válidos)	[Completar]	Pendiente
CP-23	HU-10 / CU-10	Panel con volumen	5.000 OT, 30.000 etapas	`EXPLAIN ANALYZE` de `v_panel_ordenes` paginada (50) + k6 a 10 VU	Uso de `ix_ot_estado_compromiso`; p95 < 300 ms	[Completar]	Pendiente

Tabla L.4 — Pruebas de seguridad (ZAP, Postman, Semgrep, lista de cotejo)

ID	Amenaza	Descripción	Precondición	Pasos / datos	Resultado esperado	Resultado obtenido	Estado
CP-24	AM-01, AM-02	Inyección SQL / NoSQL	Token de Encargada	`GET /ordenes?estado=' OR 1=1--`; `GET /ordenes/{id}` con `id = 1;DROP TABLE x`; body `{"insumoId": {"$ne": null}}`; escaneo activo de ZAP	400 por validación en todos; sin errores SQL ni *stack traces* en la respuesta; ZAP sin alertas de inyección	[Completar]	Pendiente
CP-25	AM-03	XSS almacenado	Operario autenticado	Guardar la nota `<img src=x onerror=alert(1)>` en una evidencia; abrirla como Encargada (Playwright)	El texto se muestra literal; no se ejecuta ningún script; la CSP bloquea la ejecución en línea (sin diálogo)	[Completar]	Pendiente
CP-26	AM-04	CSRF en el refresh	Sesión válida en el navegador	Página en `https://evil.test` que hace `fetch(.../auth/refresh, {credentials: 'include'})`; petición sin `X-Requested-With`	403 por origen no permitido o encabezado ausente; la cookie no viaja (SameSite=Strict)	[Completar]	Pendiente
CP-27	AM-05	IDOR horizontal	Operarios A y B con etapas propias	A envía `POST /etapas/{idDeB}/terminar` y `GET /evidencias?etapa={idDeB}`	404 en ambas; la etapa de B no cambia; evento de seguridad registrado	[Completar]	Pendiente
CP-28	AM-06	Escalada vertical	Token de Operario	`POST /plantillas`, `PATCH /usuarios/{id}/roles`, `GET /bitacora`	403 en los tres; prueba de inventario: 0 rutas sin permiso declarado	[Completar]	Pendiente
CP-29	AM-07	Fuerza bruta	Usuario activo	6 logins fallidos seguidos; luego uno correcto; 11 solicitudes en 60 s desde la misma IP	Desde el 5.º fallo: bloqueo de 15 min (el login correcto también falla con mensaje genérico); la solicitud 11.ª recibe 429	[Completar]	Pendiente
CP-30	AM-08	Reutilización de un refresh rotado	Sesión con refresh R1	Refrescar con R1 (obtiene R2); volver a usar R1; luego usar R2	Uso de R1 repetido: 401 y revocación de la familia; R2 también 401; evento SESIONES_REVOCADAS	[Completar]	Pendiente

ID	Amenaza	Descripción	Precondición	Pasos / datos	Resultado esperado	Resultado obtenido	Estado
CP-31	AM-09	Archivo no permitido	Etapa propia	Solicitar URL para `text/html`; para jpeg de 20 MB; PUT con `Content-Type` distinto al firmado	400, 400 y 403 (firma inválida en R2)	[Completar]	Pendiente
CP-32	AM-10, AM-11, AM-12	Configuración, cabeceras y dependencias	Staging desplegado	ZAP *baseline*; `npm audit --audit-level=high`; revisión con la lista de cotejo ASVS (Anexo E); revisión de logs sin secretos; con el rol `sgp_app`, intentar modificar `movimiento_insumo`, borrar `outbox_evento` y crear una tabla	ZAP sin High/Medium; CSP, HSTS, `X-Content-Type-Options`, `frame-ancestors` presentes; 0 vulnerabilidades altas; ítems ASVS L2 seleccionados cumplidos; los tres intentos con `sgp_app` fallan por falta de privilegios (R7, R8)	[Completar]	Pendiente

Tabla L.5 — Pruebas de usabilidad y adaptabilidad

ID	HU / CU	Descripción	Precondición	Pasos / datos	Resultado esperado	Resultado obtenido	Estado
CP-33	HU-01, HU-02, HU-03 / CU-05, CU-06, CU-09	Autonomía del operario (criterio de éxito)	5 participantes; staging con OT asignadas	Tareas T1–T3 sin ayuda, en el propio teléfono, cronometradas	$\geq 80\,\%$ de éxito sin ayuda; T1 en ≤ 3 toques y < 30 s	[Completar]	Pendiente
CP-34	RNF-02	Diseño responsive	Build en staging	Playwright: capturas de P-01 a P-12 en 360×800, 390×844, 768×1024 y 1366×768; aserción de ausencia de *scroll* horizontal	0 desbordes horizontales; navegación inferior en $<$ 600 px y lateral en $\geq$ 1024 px	[Completar]	Pendiente
CP-35	RNF-01	Satisfacción percibida	Tras CP-33	Cuestionario SUS (10 ítems)	Puntaje SUS promedio ≥ 70	[Completar]	Pendiente
CP-36	RNF-01	Accesibilidad automatizada	Build en staging	`@axe-core/playwright` en P-01 a P-12 (reglas WCAG 2.2 A/AA)	0 violaciones *serious* o *critical*	[Completar]	Pendiente